



ESCUELA TÉCNICA SUPERIOR DE INGENIERÍA INFORMÁTICA
UNIVERSIDAD DE SEVILLA

Trabajo Fin de Grado

Grado en Ingeniería Informática — Tecnologías Informáticas

**Detección de fraude mediante agentes LLM locales sobre
un detector de anomalías**

Realizado por:

Adrián Muriel Bautista

Dirigido por:

Alfonso Márquez

Departamento:

Lenguajes y Sistemas Informáticos

Sevilla, octubre de 2026

Resumen

La detección de fraude en transacciones financieras se aborda habitualmente con modelos de aprendizaje automático que asignan a cada operación una puntuación de riesgo. Estos modelos ordenan bien las operaciones, pero su salida es un número acompañado de la contribución de cada variable a ese número, algo difícil de interpretar para un analista sin formación técnica.

Este Trabajo Fin de Grado tiene un doble objetivo. El primero es implementar y evaluar un detector de fraude en transacciones con tarjeta basado en XGBoost, acompañado de un Isolation Forest como referencia no supervisada, con una metodología de evaluación que evita la fuga de información entre los datos de entrenamiento y los de prueba. El segundo es estudiar si una capa de agentes basados en modelos de lenguaje, ejecutados en local, mejora el sistema en dos sentidos concretos: la fiabilidad de la predicción, es decir, que se detecte más fraude o se generen menos falsas alarmas, y la explicabilidad, es decir, que el analista reciba una explicación comprensible de cada caso. El sistema completo se ejecuta en una única máquina, sin servicios externos y sin coste de licencias.

El detector alcanza un área bajo la curva de precisión y exhaustividad (AUC-PR) de 0,883 cuando los datos se dividen al azar y de 0,797 cuando se entrena con las transacciones más antiguas y se evalúa con las más recientes, valores dentro del rango publicado para este conjunto de datos. Entre los cincuenta casos que el detector considera más sospechosos, el 100 % y el 98 % respectivamente son fraude real.

La capa de agentes produce informes legibles y verificados automáticamente, pero no mejora la fiabilidad de la predicción. Cuando se le permite descartar casos elevados por el detector, reduce la fiabilidad de la detección, sobre todo con el modelo de lenguaje más pequeño evaluado, de 8.000 millones de parámetros. Los modelos de mayor tamaño descartan menos fraude real, pero la diferencia no es estadísticamente significativa. Un modelo de 72.000 millones de parámetros, ejecutado en el mismo equipo cargando sus capas desde disco, emite exactamente las mismas decisiones que uno de 14.000 millones con un coste de cómputo 1.790 veces mayor. El trabajo documenta además los errores detectados durante el desarrollo que no generan ningún aviso y producen informes de apariencia correcta.

Por último, cuando se advierte al modelo de que un importe bajo no descarta el fraude, su explicación escrita incorpora esa advertencia, pero su decisión no cambia. En el modelo de 14.000 millones de parámetros, la explicación contradice a la decisión en el

61 % de los casos en que menciona el patrón. Por tanto, una explicación bien redactada no garantiza que la decisión se haya tomado por los motivos que expone.

Abstract

Fraud detection in financial transactions is commonly addressed with machine learning models that assign a risk score to each operation. These models rank operations well, but their output is a number together with the contribution of each feature to that number, which is hard to interpret for an analyst without technical training.

This Bachelor's Thesis has a twofold objective. The first is to implement and evaluate a card fraud detector based on XGBoost, with an Isolation Forest as an unsupervised reference, using an evaluation methodology that prevents information leakage between training and test data. The second is to study whether a layer of agents based on locally executed language models improves the system in two specific ways: prediction reliability, that is, detecting more fraud or raising fewer false alarms, and explainability, that is, giving the analyst an understandable explanation of each case. The whole system runs on a single machine, with no external services and no licensing costs.

The detector reaches an area under the precision-recall curve (AUC-PR) of 0.883 when the data are split at random and 0.797 when it is trained on the oldest transactions and evaluated on the most recent ones, both within the range reported for this dataset. Among the fifty cases the detector ranks as most suspicious, 100 % and 98 % respectively are actual fraud.

The agent layer produces readable, automatically verified reports, but it does not improve prediction reliability. When it is allowed to discard cases raised by the detector, it reduces detection reliability, especially with the smallest language model evaluated, with 8 billion parameters. Larger models discard less actual fraud, but the difference is not statistically significant. A 72-billion-parameter model, run on the same computer by loading its layers from disk, issues exactly the same decisions as a 14-billion-parameter model at 1,790 times the computing cost. The thesis also documents the errors found during development that raise no warning and produce reports that look correct.

Finally, when the model is told that a low amount does not rule out fraud, its written explanation incorporates that warning, but its decision does not change. In the 14-billion-parameter model, the explanation contradicts the decision in 61 % of the cases in which it mentions the pattern. A well-written explanation therefore does not guarantee that the decision was made for the reasons it states.

Agradecimientos

Dedicado a mis padres y a mi pareja que me han apoyado en todo el camino. Cada suspenso, cada aprobado, todo el esfuerzo y todas las alegrías se las dedico a ellos. Gracias por el apoyo.

Índice general

Resumen.....	I
Abstract.....	III
Agradecimientos.....	IV
Índice general.....	V
Índice de figuras.....	IX
Índice de tablas.....	XI
1. Introducción.....	1
1.1 Contexto y motivación.....	1
1.2 Objetivos.....	2
1.3 Restricción de partida.....	2
1.4 Tecnologías utilizadas.....	2
1.5 Estructura del documento.....	3
2. Estado del arte.....	4
2.1 El problema de la detección de fraude.....	4
2.2 Detección de fraude con aprendizaje automático.....	5
2.2.1 La cuestión de la fuga de información.....	6
2.2.2 Decisiones metodológicas de este trabajo.....	7
2.3 Conceptos básicos sobre modelos de lenguaje y agentes.....	8
2.4 Modelos de lenguaje sobre datos tabulares.....	10
2.5 Modelos de lenguaje para explicar salidas de modelos.....	11
2.6 Sistemas multiagente en el ámbito financiero.....	12
2.7 Síntesis.....	13
3. Planificación y gestión del proyecto.....	14
3.1 Marco de gestión.....	14
3.2 Alcance del proyecto.....	14
3.3 Planificación y recursos.....	15
3.4 Análisis de riesgos.....	16
3.5 Ejecución y seguimiento.....	17
3.6 Esfuerzo previsto frente a real.....	17
3.7 Lecciones aprendidas.....	18
4. Ejecución y desarrollo del producto.....	20
4.1 Arquitectura general.....	20
4.2 El conjunto de datos.....	25
4.2.1 Origen y composición.....	25
4.2.2 Ingeniería de variables.....	26
4.3 Capa 1: el detector.....	27
4.3.1 Elección de los algoritmos.....	27
4.3.2 Tratamiento del desbalanceo.....	28

4.3.3 Las dos particiones.....	28
4.3.4 Explicabilidad sin dependencias adicionales.....	29
4.4 Capa 2: el departamento de agentes.....	29
4.4.1 Marco de orquestación.....	29
4.4.2 Los seis roles.....	31
4.4.3 El expediente determinista.....	32
4.4.4 Los dos modos de autoridad.....	33
4.5 El escenario de evaluación.....	33
4.6 Instrumentación de la evaluación.....	35
4.6.1 Auditoría de fiabilidad.....	35
4.6.2 Evaluación contra la clase real.....	37
4.6.3 Experimentación reproducible.....	37
4.7 Demostrador web.....	37
4.8 Seguimiento del desarrollo.....	38
5. Implementación: tecnologías utilizadas.....	41
5.1 Lenguaje de programación y entorno de ejecución.....	42
5.1.1 Python.....	42
5.1.2 Hardware y CUDA.....	43
5.2 Bibliotecas de datos y aprendizaje automático.....	44
5.2.1 pandas y NumPy.....	44
5.2.2 scikit-learn.....	44
5.2.3 XGBoost.....	45
5.2.4 imbalanced-learn.....	45
5.2.5 SciPy.....	45
5.2.6 Matplotlib.....	45
5.3 Modelos de lenguaje y agentes.....	46
5.3.1 Ollama.....	46
5.3.2 CrewAI y LiteLLM.....	46
5.3.3 PyTorch, Transformers y bitsandbytes.....	47
5.3.4 AirLLM.....	47
5.3.5 PEFT y Datasets.....	47
5.4 Aplicación web.....	48
5.4.1 FastAPI y Uvicorn.....	48
5.4.2 HTML, CSS y JavaScript.....	48
5.4.3 JSON.....	49
5.5 Herramientas de desarrollo.....	49
5.5.1 Git y GitHub.....	49
5.5.2 Graphviz.....	49
6. Resultados.....	50

6.1 Diseño experimental.....	50
6.1.1 Métricas empleadas.....	50
6.1.2 Definición de los experimentos.....	50
6.2 E1. Comparación de detectores.....	53
6.2.1 Objetivo y diseño.....	53
6.2.2 Resultados.....	53
6.2.3 Discusión.....	55
6.3 E2. Ablación de variables.....	56
6.3.1 Objetivo y diseño.....	56
6.3.2 Resultados.....	56
6.3.3 Discusión.....	56
6.4 E3. Tratamiento del desbalanceo y fuga de información.....	57
6.4.1 Objetivo y diseño.....	57
6.4.2 Resultados.....	57
6.4.3 Discusión.....	57
6.5 E4. Punto de operación y capacidad de revisión.....	58
6.5.1 Objetivo y diseño.....	58
6.5.2 Resultados.....	58
6.5.3 Discusión.....	59
6.6 E5. Comparativa de cinco modelos en lotes sintéticos.....	60
6.6.1 Objetivo y diseño.....	60
6.6.2 Resultados.....	60
6.6.3 Discusión.....	60
6.7 E6. Comparativa de tres modelos en turnos reales.....	61
6.7.1 Objetivo y diseño.....	61
6.7.2 Resultados.....	61
6.7.3 Discusión.....	63
6.7.4 Resultados corregidos durante el trabajo.....	64
6.8 E7. Efecto de la capa 2 y ablación del expediente.....	65
6.8.1 Objetivo y diseño.....	65
6.8.2 Resultados.....	65
6.8.3 Discusión.....	66
6.9 E8. Operación en continuo.....	67
6.9.1 Objetivo y diseño.....	67
6.9.2 Resultados.....	67
6.9.3 Discusión.....	69
6.10 E9. Comparación con un modelo de mayor tamaño.....	69
6.10.1 Objetivo y diseño.....	69
6.10.2 Resultados.....	70

6.10.3 Discusión.....	72
6.11 E10. Corrección de la premisa del importe.....	73
6.11.1 Objetivo y diseño.....	73
6.11.2 Resultados.....	73
6.11.3 Discusión.....	75
6.12 E11. Extensión exploratoria con variables de negocio.....	76
6.12.1 Objetivo y diseño.....	76
6.12.2 Resultados.....	77
6.12.3 Discusión.....	79
6.13 E12. Ajuste fino con QLoRA.....	79
6.13.1 Objetivo y diseño.....	79
6.13.2 Resultados.....	80
6.13.3 Discusión.....	81
6.14 Errores silenciosos detectados.....	81
7. Conclusiones.....	85
7.1 Grado de consecución de los objetivos.....	85
7.2 Conclusiones principales.....	86
7.2.1 El detector es la contribución principal.....	86
7.2.2 El tamaño del modelo de lenguaje no es el factor limitante.....	86
7.2.3 Corregir la premisa cambia la explicación, no la decisión.....	87
7.2.4 La puntuación del detector aporta confianza, no criterio.....	88
7.2.5 El cuello de botella operativo es la capacidad de revisión.....	88
7.2.6 Los errores del sistema no generan avisos.....	88
7.3 Posicionamiento respecto a la literatura.....	89
7.4 Limitaciones.....	91
7.5 Trabajo futuro.....	92
Bibliografía.....	94
Apéndices.....	98
A. Manual de instalación y reproducción.....	98
B. Código fuente.....	99
C. Manual de usuario del demostrador.....	101
C.1 Puesta en marcha.....	101
C.2 Pantalla 1: construir un turno.....	102
C.3 Pantalla 2: inspeccionar un caso.....	103
C.4 Pantalla 3: investigar un caso.....	104
C.5 Pantalla 4: generar y auditar el informe.....	105
C.6 Problemas frecuentes.....	107
D. Glosario.....	107
E. Uso de herramientas de inteligencia artificial.....	109

Índice de figuras

Figura 1. Planificación y ejecución del proyecto. Las barras con trama corresponden a tareas planificadas y la línea discontinua marca la fecha de esta versión de la memoria.....	15
Figura 2. Arquitectura del sistema y recorrido completo de un turno. En azul, los componentes deterministas; en rojo, los estocásticos; en verde, la comprobación automática.....	22
Figura 3. Recorrido de un caso individual desde la transacción hasta sus dos salidas: el párrafo explicativo y el veredicto.....	24
Figura 4. El caso 55 en el demostrador: a la izquierda, lo que calcula el detector; a la derecha, el dossier literal que recibe el modelo.....	25
Figura 5. Distribución de importes por clase sobre el conjunto completo. A la izquierda, proporción acumulada; a la derecha, porcentaje de cada clase por debajo de varios importes.....	26
Figura 6. Definición de los agentes en CrewAI dentro del entorno de desarrollo: parámetros comunes, agente Coordinador y ensamblado del equipo con proceso secuencial.....	30
Figura 7. Traza de CrewAI durante una ejecución: el Coordinador entrega su resumen del turno y el Analista de datos comienza su tarea.....	32
Figura 8. Construcción y puntuación del turno de evaluación en el demostrador: 32.791 transacciones y 54 fraudes, de los que el detector eleva 55 casos con 44 fraudes.....	35
Figura 9. Auditoría de un informe en modo revisa: cobertura del 98 % y aviso de que 21 casos quedaron sin veredicto y conservan la decisión del detector.....	36
Figura 10. Pantalla inicial del demostrador, con los indicadores de entorno y las cuatro etapas del recorrido.....	38
Figura 11. Historial del repositorio: cada fase del desarrollo corresponde a un registro trazable.....	39
Figura 12. Organización de las tecnologías empleadas en el sistema.....	41
Figura 13. Verificación del entorno: versión de Python, GPU, servidor de modelos y conjunto de datos.....	44
Figura 14. Mapa del diseño experimental: fuentes de datos, experimentos sobre cada capa y experimentos que se comparan con la línea base.....	52
Figura 15. Métricas de los tres detectores en ambas particiones, tal como las imprime el guion a partir del fichero de resultados del entrenamiento.....	54
Figura 16. Curvas de precisión frente a exhaustividad de los tres detectores en la partición aleatoria, con la línea base en la prevalencia del fraude.....	54
Figura 17. Precisión en los N primeros casos y fracción del fraude capturado frente al número de casos revisados (partición aleatoria), con una capacidad de revisión de diez casos marcada.....	59
Figura 18. Fraude destruido por cada modelo en dieciséis ejecuciones: total por modelo (izquierda) y distribución por ejecución (derecha).....	62
Figura 19. Efecto del nivel de información del expediente sobre el volumen de descartes, su acierto y la reproducibilidad entre ejecuciones.....	66
Figura 20. Espera media hasta aparecer en un informe frente al tamaño de ventana (izquierda) y ocupación de cómputo medida en las ejecuciones registradas (derecha).	69

Figura 21. Ejecución con AirLLM junto a nvidia-smi: memoria de vídeo casi vacía (1,7 de 16 GB) y utilización de GPU de un solo dígito mientras el modelo de 72B procesa un caso.....	71
Figura 22. Investigación del caso 55 sin la corrección de dominio: el modelo usa el importe bajo como argumento y descarta el fraude.....	74
Figura 23. Investigación del caso 55 con la corrección de dominio: la explicación reconoce el patrón del cargo de prueba y el veredicto descarta igualmente el fraude.	74
Figura 24. Error silencioso reproducido en el demostrador: el modelo de razonamiento agota los 220 tokens pensando y devuelve una respuesta vacía.....	82
Figura 25. Estructura del repositorio. En azul, el código determinista de la capa 1; en rojo, el estocástico de la capa 2; en verde, los guiones de experimentación; en gris, los datos y la documentación.....	98
Figura 26. Función de selección de casos del detector, con la documentación de sus decisiones de diseño.....	100
Figura 27. Inicio del servidor del demostrador desde la consola.....	102
Figura 28. Pantalla inicial del demostrador con los indicadores del entorno.....	102
Figura 29. Pantalla 1: construcción y puntuación de un turno.....	103
Figura 30. Pantalla 2: inspección del caso 1 con el expediente completo.....	104
Figura 31. Pantalla 3: investigación del caso 1, confirmado por el modelo.....	105
Figura 32. Pantalla 4: informe marcado como fiable por el auditor, con las métricas del turno y del sistema.....	106
Figura 33. Pantalla 4: tabla de casos ensamblada sin modelo y comienzo del informe redactado.....	107

Índice de tablas

Tabla 1. Resultados de la propuesta MASCA frente a sus líneas base, según sus autores.....	12
Tabla 2. Rasgos del proyecto que condicionan su gestión.....	14
Tabla 3. Recursos del proyecto.....	16
Tabla 4. Registro de riesgos del proyecto.....	16
Tabla 5. Hitos del proyecto y evidencia asociada.....	17
Tabla 6. Esfuerzo dedicado por tramos, reconstruido a partir de la evidencia del proyecto.....	18
Tabla 7. Comparación entre las dos capas del sistema.....	23
Tabla 8. Algoritmos implementados en el detector.....	27
Tabla 9. Agentes de la capa 2, su función y sus herramientas.....	31
Tabla 10. Turnos de evaluación y composición de su expediente con umbral 0,05.....	34
Tabla 11. Fases del desarrollo y entregable de cada una.....	39
Tabla 12. Tecnologías principales y su función en el sistema.....	42
Tabla 13. Modelos de lenguaje empleados en el trabajo.....	46
Tabla 14. Rutas de la API del demostrador web.....	48
Tabla 15. Definición de los doce experimentos del trabajo y objetivo específico al que responde cada uno.....	51
Tabla 16. Métricas de los tres detectores en las dos particiones.....	53
Tabla 17. Precisión en los N primeros casos de XGBoost y Random Forest (partición temporal).....	55
Tabla 18. Ablación de grupos de variables del detector (partición temporal).....	56
Tabla 19. AUC-PR de XGBoost con cuatro estrategias de tratamiento del desbalanceo.	57
Tabla 20. Casos elevados, fraude capturado, recall y precisión según el umbral (turno 0 h).....	58
Tabla 21. Fraude capturado y recall según la capacidad de revisión (turno 0 h).....	59
Tabla 22. Comparativa de cinco modelos de lenguaje sobre lotes sintéticos con umbral estricto (12 ejecuciones por modelo).....	60
Tabla 23. Comparativa de tres modelos sobre turnos reales con umbral 0,05 (16 ejecuciones por modelo).....	61
Tabla 24. Distribución del número de fraudes destruidos por ejecución.....	62
Tabla 25. Valores p de los contrastes estadísticos entre pares de modelos.....	62
Tabla 26. Recall del sistema en las cuatro repeticiones de cada turno.....	63
Tabla 27. Recall del detector con umbral estricto frente al sistema con la capa 2 y umbral 0,05.....	65
Tabla 28. Ablación del expediente: descartes y su acierto según la información disponible (medias por ejecución).....	66
Tabla 29. Turnos con las cuatro repeticiones idénticas según el nivel de información.....	66
Tabla 30. Transacciones y casos elevados según el tamaño de ventana.....	68
Tabla 31. Espera media hasta la alerta según el tamaño de ventana.....	68
Tabla 32. Veredictos de Qwen2.5 de 14B y de 72B sobre los cuatro casos seleccionados del turno 0 h.....	71

Tabla 33. Coste de ejecución de los modelos de 14B y 72B.....	71
Tabla 34. Distribución de importes por clase en el turno 0 h.....	72
Tabla 35. Efecto de la corrección de dominio sobre 192 veredictos (48 por modelo y condición).....	75
Tabla 36. Composición del conjunto Sparkov en entrenamiento y prueba.....	77
Tabla 37. Métricas del detector XGBoost sobre el conjunto Sparkov.....	77
Tabla 38. Contribuciones por variable de un caso de fraude del conjunto Sparkov.....	78
Tabla 39. Veredictos del Investigador con el dossier completo y con el reducido sobre los mismos nueve casos.....	78
Tabla 40. Veredictos del modelo base, con corrección en el enunciado y con el adaptador QLoRA sobre doce casos.....	80
Tabla 41. Errores silenciosos detectados durante el desarrollo y su corrección.....	82
Tabla 42. Cobertura de la investigación y del informe final según el tamaño del expediente.....	84
Tabla 43. Grado de consecución de los objetivos específicos.....	85
Tabla 44. Posicionamiento del trabajo respecto a la literatura revisada.....	89
Tabla 45. Directorios principales del repositorio.....	98
Tabla 46. Guiones del sistema en funcionamiento.....	100
Tabla 47. Guiones de experimentos y ablaciones.....	101
Tabla 48. Pruebas que no invocan al modelo de lenguaje.....	101
Tabla 49. Herramientas de inteligencia artificial empleadas durante el trabajo.....	109

1. Introducción

1.1 Contexto y motivación

El fraude con tarjetas de pago consiste en utilizar una tarjeta, o sus datos, sin el consentimiento de su titular: una tarjeta robada con la que se hacen compras, o un número de tarjeta obtenido en una filtración que se usa para pagar en una tienda en línea. Es un problema de gran volumen económico. Según el informe conjunto de la Autoridad Bancaria Europea y el Banco Central Europeo, el fraude en pagos dentro del Espacio Económico Europeo alcanzó los 4.200 millones de euros en 2024, de los cuales 1.329 millones corresponden a tarjetas [1].

Al mismo tiempo, el fraude es un fenómeno muy poco frecuente en proporción al total de operaciones. En el conjunto de datos empleado en este trabajo [2], las operaciones fraudulentas representan el 0,173 % de las transacciones y el 0,239 % del importe total. Esa desproporción define el problema: un sistema que clasificara todas las operaciones como legítimas acertaría el 99,83 % de las veces y sería completamente inútil.

Los departamentos antifraude no revisan todas las alertas que produce un modelo, sino solo las que les da tiempo a revisar. Por eso, el valor de un sistema de detección no está en su tasa de acierto global, sino en la calidad de los primeros puestos de su lista de casos sospechosos: cuántas de las primeras cincuenta operaciones señaladas son realmente fraude.

A esa dificultad se añade otra. Los modelos que mejor funcionan en este dominio trabajan sobre datos tabulares, es decir, datos organizados en forma de tabla en la que cada fila es una transacción y cada columna una variable (importe, hora, etc.). Estos modelos, basados en conjuntos de árboles de decisión, devuelven para cada transacción una probabilidad de fraude y la contribución de cada variable a esa probabilidad. Para un analista sin formación en aprendizaje automático, un valor como «V14 = -9,151 con aporte +5,100» no es una explicación razonable sobre la que pueda actuar.

La aparición de modelos de lenguaje capaces de ejecutarse en un ordenador personal abre una posibilidad: emplearlos como capa de interpretación sobre el detector, para que traduzcan su salida a lenguaje natural. Este trabajo estudia si esa capa aporta algo medible y en qué condiciones.

1.2 Objetivos

El trabajo tiene un doble objetivo general. El primero, y principal, es implementar y evaluar un detector de fraude en transacciones con tarjeta basado en XGBoost, con una metodología de evaluación rigurosa. El segundo es estudiar si una capa de agentes basados en modelos de lenguaje, ejecutados íntegramente en local y sin coste de licencias, mejora la fiabilidad de la predicción del detector o la explicabilidad de sus resultados.

Se concreta en cinco objetivos específicos:

OE1. Construir y evaluar un detector de fraude sobre transacciones con tarjeta, comparando algoritmos supervisados y no supervisados, ajustando sus hiperparámetros y tratando el desbalanceo de clases sin introducir fuga de información.

OE2. Diseñar e implementar una capa multiagente basada en modelos de lenguaje ejecutados en local que interprete la salida del detector y genere informes de revisión.

OE3. Comparar experimentalmente varios modelos de lenguaje locales de distinto tamaño, midiendo cuál es el tamaño mínimo necesario para que permitirles descartar casos no reduzca la fiabilidad de la detección de fraude.

OE4. Cuantificar qué se pierde por emplear modelos que caben en la memoria de una tarjeta gráfica de consumo frente a un modelo de mayor tamaño, sin abandonar la ejecución local.

OE5. Validar de forma automática las salidas generadas por la capa de agentes y documentar los errores observados.

1.3 Restricción de partida

El sistema debe ejecutarse por completo en una sola máquina, sin depender de servicios de terceros, sin licencias y sin coste. Esta restricción coincide con la situación real de una entidad financiera, que no puede enviar datos de tarjetas de sus clientes a un servicio externo por motivos de confidencialidad y de cumplimiento normativo. Tiene además una ventaja práctica: el funcionamiento del sistema no depende de la disponibilidad de ningún proveedor externo.

1.4 Tecnologías utilizadas

Todo el sistema está escrito en Python. El detector se apoya en las bibliotecas XGBoost [3] y scikit-learn [4], la capa de agentes se construye con el marco CrewAI [5] y los modelos de lenguaje, todos de pesos abiertos, se ejecutan en local con Ollama [6]

sobre una tarjeta gráfica NVIDIA RTX 5060 Ti de 16 GB. El demostrador web utiliza FastAPI [7]. El capítulo 5 describe con detalle cada una de las tecnologías empleadas y su función en el sistema.

1.5 Estructura del documento

El capítulo 2 explica el problema de la detección de fraude, introduce los conceptos básicos sobre modelos de lenguaje y agentes, y revisa el estado del arte. El capítulo 3 documenta la planificación y la gestión del proyecto. El capítulo 4 describe el diseño y el desarrollo del sistema. El capítulo 5 describe las tecnologías con las que se ha implementado. El capítulo 6 define los doce experimentos realizados y presenta sus resultados. El capítulo 7 recoge las conclusiones, el posicionamiento del trabajo respecto a la literatura, las limitaciones y las líneas de trabajo futuro. Los apéndices incluyen el manual de instalación, la descripción del código fuente, el manual de usuario del demostrador, un glosario y la declaración del uso de herramientas de inteligencia artificial.

2. Estado del arte

Este capítulo comienza describiendo el problema de la detección de fraude y las técnicas de aprendizaje automático que se aplican a él, que es el ámbito de la contribución principal del trabajo. A continuación introduce los conceptos básicos sobre modelos de lenguaje y agentes necesarios para seguir el resto de la memoria, y revisa tres líneas de investigación relacionadas con la segunda parte del trabajo: el uso de modelos de lenguaje sobre datos tabulares, su uso para explicar las salidas de otros modelos y los sistemas multiagente aplicados al ámbito financiero.

2.1 El problema de la detección de fraude

Un sistema de detección de fraude recibe las transacciones de los clientes de una entidad y debe decidir cuáles son sospechosas. Un ejemplo típico es el siguiente: un delincuente obtiene los datos de una tarjeta, realiza primero un cargo de uno o dos euros para comprobar que la tarjeta funciona y, si el cargo se acepta, realiza a continuación una compra de importe elevado. Si el sistema detecta el primer cargo a tiempo, la tarjeta puede bloquearse antes de que se produzca el segundo.

Las cifras del informe de la Autoridad Bancaria Europea y el Banco Central Europeo dan la medida del problema. En 2024 el fraude en pagos alcanzó los 4.200 millones de euros en el Espacio Económico Europeo, frente a 3.500 millones en 2023. El fraude con tarjetas emitidas en la región ascendió a 1.329 millones, un 29 % más que el año anterior. Aun así, el fraude representa en torno al 0,002 % del valor total de las operaciones de pago, y el fraude con tarjeta es unas diecisiete veces mayor cuando el comercio está fuera del Espacio Económico Europeo, donde la autenticación reforzada del cliente no es obligatoria [1].

Estas cifras resumen las tres dificultades del problema. La primera es el desbalanceo de clases: los casos de interés son una fracción mínima del total, de modo que la exactitud global no sirve para medir el rendimiento. La segunda es el cambio a lo largo del tiempo: los defraudadores adaptan su comportamiento, por lo que un modelo debe evaluarse sobre datos posteriores a los que usó para aprender. La tercera es la capacidad limitada de revisión: la entidad solo puede investigar manualmente un número reducido de alertas al día, de modo que lo que importa es que las primeras alertas sean correctas.

2.2 Detección de fraude con aprendizaje automático

El conjunto de datos empleado en este trabajo es el publicado por el grupo de aprendizaje automático de la Universidad Libre de Bruselas junto con Worldline [2, 8]. Es el conjunto de referencia en la literatura de detección de fraude con tarjeta, lo que permite comparar las métricas obtenidas con resultados publicados. Los valores de AUC-PR reportados sobre él se sitúan aproximadamente entre 0,78 y 0,97, según el modelo y el tratamiento que cada trabajo aplique al desbalanceo de clases. La AUC-PR es el área bajo la curva que relaciona la precisión con la exhaustividad del modelo; se define con detalle en el capítulo 6 y es la métrica recomendada cuando las clases están muy desbalanceadas [9].

Se revisan a continuación tres detectores representativos de la literatura.

FraudX AI [10] es un modelo de conjunto, es decir, un modelo que no decide con un solo algoritmo sino combinando las predicciones de varios, de forma que los errores de uno puedan compensarse con los aciertos de otro. En su caso promedia las probabilidades que asignan Random Forest y XGBoost y optimiza el umbral de decisión. Se evalúa sobre el conjunto de la Universidad Libre de Bruselas, el mismo que emplea este trabajo, y reporta un 95 % de exhaustividad y un 97 % de AUC-PR. Para interpretar sus predicciones aplica atribución de Shapley, una técnica que reparte la predicción del modelo entre las variables de entrada según lo que aporta cada una; procede de la teoría de juegos cooperativos [11] y se aplica al aprendizaje automático en el trabajo de Lundberg y Lee [12]. Este trabajo emplea la misma técnica, que se describe en el apartado 4.3.4. FraudX AI no emplea remuestreo sintético: conserva el desbalanceo natural del conjunto y asigna mayor peso a la clase minoritaria durante el entrenamiento, que es la misma decisión que se ha tomado en este trabajo. Sus autores argumentan que los conjuntos equilibrados artificialmente producen resultados que no reflejan las condiciones reales de operación. Dividen los datos en un 80 % para entrenamiento y un 20 % para prueba, manteniendo la proporción de clases.

Taha y Malebary [13] proponen un detector basado en LightGBM, otro algoritmo de refuerzo de gradiente sobre árboles, cuyos hiperparámetros se ajustan mediante optimización bayesiana. Sobre el conjunto de la Universidad Libre de Bruselas reportan un 98,40 % de exactitud, un 92,88 % de área bajo la curva ROC y un 97,34 % de precisión, pero un valor de F1 de solo el 56,95 %. Esa combinación de cifras es por sí misma ilustrativa del problema: la exactitud es altísima porque el 99,83 % de las transacciones son legítimas y basta clasificarlas bien para acercarse al 100 %, mientras que el valor de F1, que resume el equilibrio entre precisión y exhaustividad sobre la clase

de fraude, queda por debajo del 60 %. Es la razón por la que este trabajo no utiliza la exactitud como métrica. Su trabajo ilustra además que la familia de algoritmos de refuerzo de gradiente, a la que pertenece XGBoost, es una de las más empleadas en este dominio.

Carcillo y otros [14] estudian la combinación de técnicas no supervisadas y supervisadas. Calculan puntuaciones de anomalía sin usar etiquetas y las añaden como variables de entrada a un clasificador supervisado. Su evaluación no se hace sobre el conjunto público anterior, sino sobre 76 millones de transacciones reales de Worldline correspondientes a 334 días de 2016, con un 0,36 % de fraude. Sus resultados son matizados y conviene retenerlos: las puntuaciones de anomalía calculadas sobre el conjunto completo no mejoran la detección de forma apreciable, las calculadas sobre el entorno inmediato de cada transacción la empeoran, y solo las calculadas por grupos de clientes producen una mejora significativa, que además aparece en AUC-PR pero no en la precisión de los primeros casos. Este enfoque está relacionado con la decisión de este trabajo de entrenar un Isolation Forest junto al detector supervisado, y su conclusión anticipa la que se obtiene en el experimento E1: una puntuación de anomalía sin etiquetas aporta mucho menos de lo que su nombre sugiere.

El detector de FraudX AI supera los 0,883 de AUC-PR obtenidos en este trabajo con la partición aleatoria, y lo hace sin remuestreo sintético. El detector construido aquí no pretende ser el mejor publicado: sus cifras se sitúan dentro del rango de la literatura y son suficientes para sostener el resto del sistema. Su aportación está en la metodología de evaluación, que se describe en los apartados siguientes.

2.2.1 La cuestión de la fuga de información

El desbalanceo extremo de este dominio obliga a tratarlo de forma explícita. La técnica más extendida en la literatura es el remuestreo, que consiste en modificar el conjunto de entrenamiento para equilibrar las clases. Una variante muy habitual es SMOTE [15], que genera ejemplos sintéticos de la clase minoritaria interpolando entre ejemplos reales cercanos.

Dos trabajos recientes, el de Kabane [16] y el de Magnier [17], han examinado cómo se aplica ese remuestreo en la literatura publicada sobre el conjunto de la Universidad Libre de Bruselas, que es el que emplea esta memoria. Sus conclusiones pueden resumirse en tres puntos. Primero, el remuestreo mejora el rendimiento del modelo. Segundo, la fiabilidad de los resultados depende del momento en que se aplica: si el remuestreo se hace antes de dividir los datos en entrenamiento y prueba, los ejemplos sintéticos se generan a partir de observaciones que después formarán parte del conjunto

de prueba. Se produce así una fuga de información, es decir, el modelo recibe durante el entrenamiento información del conjunto con el que después se le evalúa, y las métricas resultan infladas. Tercero, los modelos que aplican el remuestreo solo sobre el conjunto de entrenamiento obtienen resultados claramente inferiores a los que lo aplican antes de dividir, pero sus resultados sí son válidos.

Kabane [16] compara el mismo modelo de refuerzo de gradiente, XGBoost, sobre ese mismo conjunto de la Universidad Libre de Bruselas en tres situaciones: sin tratar el desbalanceo, con remuestreo aplicado solo al conjunto de entrenamiento y con remuestreo aplicado antes de dividir. En el último caso el modelo alcanza un 99,969 % de exactitud, un 99,938 % de precisión y un 100 % de exhaustividad. Sus experimentos sin fuga, en cambio, se sitúan en torno al 96,5 % de precisión y el 91,5 % de exhaustividad.

Magnier [17] lleva el argumento más lejos. Construye una red neuronal de tipo perceptrón multicapa y la simplifica hasta dejarla en una única neurona de salida, manteniendo la fuga por remuestreo previo a la división. Esa configuración mínima obtiene un 94 % de exhaustividad y un 97,6 % de precisión, y al añadir una capa oculta de hasta dieciséis neuronas alcanza el 99,9 % de exhaustividad. Su conclusión es que la fuga de información aporta más rendimiento aparente que la complejidad del modelo. El mismo trabajo identifica cuatro problemas recurrentes en esta literatura: la fuga derivada del orden del preprocesamiento, la descripción poco precisa de la metodología, la falta de validación temporal y la optimización de la exhaustividad a costa de la precisión. Los cuatro se han tenido en cuenta en el diseño de este trabajo.

La consecuencia práctica es que una parte de los valores publicados en el extremo alto del rango no es comparable con valores obtenidos sin fuga, y que no tiene sentido comparar cifras entre trabajos sin revisar el orden en que se aplicó cada paso.

2.2.2 Decisiones metodológicas de este trabajo

De lo anterior se derivan cuatro decisiones, que se exponen aquí para que el lector pueda interpretar las métricas del capítulo 6.

La primera es emplear XGBoost como detector principal. XGBoost [3] es un algoritmo de refuerzo de gradiente que construye árboles de decisión en secuencia, de forma que cada árbol corrige los errores de los anteriores. Es el algoritmo sobre el que se apoya FraudX AI, que obtiene con él una exhaustividad del 95 % sobre el mismo conjunto de datos que emplea este trabajo, y pertenece a la familia de modelos más

utilizada en el dominio. Además, calcula de forma nativa y exacta la contribución de cada variable a cada predicción, que es la información que necesita la capa de agentes.

La segunda es no emplear remuestreo sintético. El desbalanceo se trata aumentando el peso de los errores cometidos sobre la clase minoritaria durante el entrenamiento. Este mecanismo no genera observaciones nuevas y, por tanto, no puede provocar fuga de información. Como la literatura indica que el remuestreo bien aplicado mejora el rendimiento, cabía esperar que esta elección rindiera algo menos; el experimento E3 del capítulo 6 comprueba que ambas estrategias rinden de forma equivalente sobre estos datos.

La tercera es reportar también una partición temporal. La mayor parte de la literatura evalúa con una partición aleatoria, que reparte las transacciones al azar entre entrenamiento y prueba. Este trabajo reporta además una partición cronológica, que entrena con las transacciones más antiguas y evalúa con las más recientes. Es la única que reproduce las condiciones reales de uso y, como es de esperar, produce métricas inferiores.

La cuarta es reportar la precisión en los N primeros casos junto a las métricas agregadas. Esta métrica, que en inglés se denomina Precision@N o P@N , es la proporción de fraudes reales entre las N transacciones con mayor puntuación. Por ejemplo, una P@50 de 0,98 significa que 49 de los 50 casos más sospechosos son fraude. Se emplea porque un departamento antifraude no revisa todo lo que el modelo señala, sino los primeros casos de la lista. Dos modelos pueden diferir de forma estadísticamente significativa en una métrica agregada y ser indistinguibles en los casos que alguien llega a revisar, como se muestra en el experimento E1.

2.3 Conceptos básicos sobre modelos de lenguaje y agentes

La segunda parte del trabajo utiliza modelos de lenguaje y agentes. Este apartado define los conceptos que aparecen en el resto de la memoria.

Un modelo de lenguaje de gran tamaño (LLM, por sus siglas en inglés) es una red neuronal, normalmente basada en la arquitectura Transformer [18], entrenada para predecir la siguiente palabra de un texto a partir de enormes cantidades de datos. El texto se procesa dividido en tokens, fragmentos de palabra de unos pocos caracteres; la longitud de la entrada y de la salida se mide en tokens. El texto que se entrega al modelo para pedirle una tarea se denomina enunciado o prompt. Al generar, el modelo elige cada token de forma aleatoria entre los más probables; la temperatura es el parámetro que controla esa aleatoriedad, de modo que con una temperatura baja, como la de 0,1

empleada en este trabajo, las respuestas varían poco entre ejecuciones, aunque no son idénticas.

El tamaño de un modelo se expresa por su número de parámetros, los valores numéricos que se ajustan durante el entrenamiento. En esta memoria se emplea la notación habitual: 8B equivale a 8.000 millones de parámetros, 14B a 14.000 millones, 20B a 20.000 millones y 72B a 72.000 millones. Los modelos evaluados son Llama 3.1 de 8B [19], Qwen2.5 de 14B y 72B [20], Qwen3 de 14B y gpt-oss de 20B [21]. A igualdad de familia y generación, un modelo con más parámetros suele obtener mejores resultados, y por eso en la literatura se habla de modelos de mayor o menor capacidad. La capacidad no es una magnitud única: se mide con bancos de pruebas específicos para cada habilidad. Por ejemplo, el Berkeley Function Calling Leaderboard [22] mide el porcentaje de llamadas a herramientas que un modelo formula correctamente, y otros bancos de pruebas miden la capacidad de colaborar con otros agentes [23] o de completar tareas de varios pasos [24]. En este trabajo se usa el tamaño en parámetros como aproximación a la capacidad y se mide directamente su efecto sobre la tarea concreta de detección de fraude.

La cuantización reduce la memoria que ocupa un modelo representando sus parámetros con menos bits, por ejemplo 4 en lugar de 16, a costa de una pequeña pérdida de precisión. Es lo que permite ejecutar modelos de 14B o 20B en una tarjeta gráfica de 16 GB.

Hay varias formas de pedir una tarea a un modelo. En modo cero disparos (zero-shot) se le pide directamente, sin ningún ejemplo resuelto [25]. La cadena de pensamiento (chain of thought) consiste en pedirle que razone paso a paso antes de responder [26, 27]. Algunos modelos recientes, denominados modelos de razonamiento, generan siempre ese razonamiento intermedio antes de la respuesta; en este trabajo es el caso de Qwen3 y gpt-oss.

Un agente es un modelo de lenguaje al que se le asigna un papel, un objetivo y un conjunto de herramientas. Una herramienta es una función del programa que el modelo puede pedir que se ejecute, por ejemplo para consultar las estadísticas de un lote de transacciones, y cuyo resultado recibe a continuación. Un sistema multiagente es un conjunto de agentes con papeles distintos que colaboran en una tarea. Para coordinarlos se emplea un marco de orquestación, una biblioteca que define los agentes y sus tareas, decide en qué orden intervienen y hace llegar a cada uno el resultado de los anteriores.

Por último, el ajuste fino consiste en continuar el entrenamiento de un modelo ya entrenado con ejemplos de una tarea concreta. LoRA [28] es una técnica de ajuste fino que no modifica los parámetros originales, sino que entrena un pequeño adaptador que se suma a ellos; QLoRA [29] aplica la misma idea sobre un modelo cuantizado a 4 bits, lo que permite realizar el ajuste en una tarjeta de consumo.

2.4 Modelos de lenguaje sobre datos tabulares

La detección de anomalías consiste en encontrar observaciones que no se ajustan al comportamiento habitual de los datos [30]. Se distinguen tres tipos. Las anomalías puntuales son observaciones individuales muy distintas del resto, como un cargo de 20.000 euros en una tarjeta cuyos pagos no superan normalmente los 100. Las anomalías contextuales son valores normales en general pero extraños en su contexto, como una compra habitual realizada a las cuatro de la madrugada. Las anomalías colectivas son grupos de observaciones que resultan sospechosos en conjunto aunque cada una por separado no lo sea, como varios cargos pequeños seguidos en pocos minutos. El fraude con tarjeta suele manifestarse como anomalías de los tres tipos.

Existe una línea de investigación que estudia si un modelo de lenguaje puede detectar anomalías directamente sobre datos tabulares. Para ello, cada fila de la tabla se convierte en texto y se entrega al modelo.

Li y otros [31] muestran que un modelo de lenguaje preentrenado, sin ningún entrenamiento adicional, puede recibir un conjunto de observaciones y señalar cuáles se apartan del resto. Tras un ajuste fino sobre datos sintéticos, su rendimiento es comparable al de los métodos especializados de aprendizaje transductivo, que son aquellos que aprenden directamente sobre el conjunto concreto de datos que deben clasificar, en lugar de construir un modelo general aplicable a datos nuevos [32]. AnoLLM [33] ajusta un modelo de lenguaje para que asigne una puntuación de anomalía a cada fila convertida en texto, y obtiene los mejores resultados en conjuntos de datos que mezclan variables numéricas y textuales, y resultados competitivos en conjuntos mayoritariamente numéricos. FinFRE-RAG [34] aplica modelos de lenguaje de pesos abiertos a la detección de fraude financiero en cuatro conjuntos de datos públicos. Selecciona las variables más importantes, las convierte en texto y añade al enunciado ejemplos parecidos ya etiquetados. Mejora claramente a la consulta directa al modelo, aunque los clasificadores tabulares especializados siguen obteniendo mejores resultados.

La diferencia con este trabajo está en el planteamiento. En esa línea, el modelo de lenguaje compite con el modelo tabular en la misma tarea de clasificación. En este trabajo, el modelo de lenguaje no clasifica todas las transacciones: actúa por encima de un detector ya entrenado, solo sobre los casos que este ha señalado, y lo que se mide es si su intervención mejora o empeora el resultado.

De esa diferencia se deriva la principal línea de trabajo futuro propuesta en el capítulo 7: aplicar el sistema a dominios con más contenido textual, como las transferencias o la prevención del blanqueo de capitales. En el conjunto de datos de este trabajo, compuesto por veintiocho variables anonimizadas, un importe y una hora, hay poco que interpretar más allá de lo que el detector ya calcula; en cambio, un ordenante, un concepto o una cadena de operaciones relacionadas sí admiten interpretación. Es una hipótesis del autor, no verificada en este trabajo.

2.5 Modelos de lenguaje para explicar salidas de modelos

Bilal, Ebert y Lin [35] revisan las propuestas que emplean modelos de lenguaje para explicar las salidas de otros modelos de aprendizaje automático, así como las técnicas para evaluar esas explicaciones. Su aportación más útil para este trabajo es la distinción entre dos propiedades de una explicación.

La fidelidad es el grado en que una explicación representa el proceso de decisión que el modelo ha seguido realmente. Una explicación fiel debe ser coherente con la lógica interna del modelo y debe permitir reproducir la decisión a partir de los factores que menciona.

La plausibilidad mide si la explicación es lógica y coherente con el conocimiento del dominio: que se apoye en principios aceptados, que no se contradiga y que no haga afirmaciones imposibles.

Los autores subrayan que una explicación puede ser plausible y no ser fiel: puede tener sentido para quien la lee y no representar el razonamiento real del modelo. Por eso recomiendan evaluar las explicaciones con métodos cuantitativos. En esta memoria ambas propiedades se utilizan para analizar los resultados de los experimentos E10 y E12 del capítulo 6, y se retoman en las conclusiones.

En los trabajos recogidos en esa revisión, el modelo de lenguaje se limita a traducir a lenguaje natural la salida de un método de atribución, como los valores de Shapley empleados en este trabajo, y no toma ninguna decisión. Esa limitación es una práctica común de esos trabajos, no una recomendación explícita de la revisión. Este trabajo va

un paso más allá: además de pedir al modelo que explique, en uno de sus modos de funcionamiento le permite descartar casos señalados por el detector, y mide el efecto de esa decisión.

2.6 Sistemas multiagente en el ámbito financiero

Existen varias propuestas de sistemas multiagente basados en modelos de lenguaje para decisiones financieras.

MASCA [36] es un sistema jerárquico de agentes para evaluar el riesgo de crédito. Organiza agentes especializados en capas (ingesta y contextualización de datos, evaluación desde varios puntos de vista, optimización de la estrategia y un orquestador que toma la decisión final) y lo evalúa sobre doscientas solicitudes de un conjunto de datos de crédito. FinCon [37] es un sistema jerárquico para decisiones de inversión en el que un agente gestor coordina a varios agentes analistas y actualiza sus criterios a partir de la revisión de sus propios errores. TradingAgents [38] reproduce la estructura de una empresa de inversión, con agentes que analizan los fundamentales, el sentimiento del mercado y los indicadores técnicos, agentes que debaten posiciones alcistas y bajistas, y un equipo de gestión del riesgo.

La Tabla 1 recoge los resultados que MASCA compara. Para interpretarla conviene aclarar que una línea base es una configuración de referencia, normalmente sencilla, contra la que se compara la propuesta para saber si esta aporta una mejora. MASCA se compara con tres líneas base: el mismo modelo de lenguaje consultado en modo cero disparos, el mismo modelo con cadena de pensamiento y un único agente que realiza todas las subtareas.

Configuración	Exactitud	Precisión	Exhaustividad	F1
Cero disparos	45,5 %	33,3 %	67,7 %	44,7 %
Cadena de pensamiento	36,0 %	37,1 %	52,1 %	43,4 %
Un solo agente multitarea	42,5 %	28,8 %	64,4 %	39,8 %
Sistema multiagente jerárquico	60,0 %	65,5 %	83,3 %	73,3 %

Tabla 1. Resultados de la propuesta MASCA frente a sus líneas base, según sus autores.

El sistema jerárquico mejora a las tres líneas base en todas las métricas. Se observa también que la cadena de pensamiento empeora la exactitud respecto a la consulta directa (36 % frente a 45,5 %); los autores lo atribuyen a que los errores se propagan a lo

largo del razonamiento. Es coherente con lo observado en el capítulo 6, donde los modelos de razonamiento tampoco mejoran el resultado.

Todas las líneas base de MASCA son configuraciones del propio modelo de lenguaje. No se compara con un modelo estadístico clásico entrenado sobre los mismos datos, de modo que el trabajo responde a si un sistema multiagente funciona mejor que un único agente, pero no a si el modelo de lenguaje aporta algo frente a no utilizarlo. Este trabajo plantea precisamente esa segunda pregunta: su línea base es el detector funcionando solo, sin capa de agentes. Además, los resultados de MASCA son estimaciones puntuales sobre doscientas muestras, sin pruebas de significación estadística, y se obtienen con modelos comerciales de una sola familia. Este trabajo emplea exclusivamente modelos abiertos ejecutados en local y acompaña sus comparaciones de contrastes estadísticos.

2.7 Síntesis

La revisión muestra que la detección de fraude con modelos tabulares está bien estudiada, pero que una parte de los resultados publicados está afectada por la fuga de información. También muestra que se han propuesto sistemas multiagente para tareas financieras y que la literatura distingue entre explicaciones plausibles y explicaciones fieles. Este trabajo se sitúa en la intersección de esas líneas: construye un detector evaluado sin fuga y con partición temporal, y mide qué aporta sobre él una capa de agentes locales, tanto en la fiabilidad de la detección como en la relación entre sus explicaciones y sus decisiones. La comparación detallada con la literatura se presenta en el capítulo 7, una vez expuestos los resultados.

3. Planificación y gestión del proyecto

Este capítulo describe cómo se ha planificado y gestionado el proyecto: el marco de gestión adoptado, el alcance, la planificación temporal y los recursos, el análisis de riesgos, el seguimiento realizado, la comparación entre el esfuerzo previsto y el real, y las lecciones aprendidas.

3.1 Marco de gestión

El proyecto presenta tres rasgos que condicionan su gestión, resumidos en la Tabla 2. El equipo lo forma una sola persona, de modo que planificar consiste en repartir sus horas disponibles. El tutor valida el alcance y los cambios importantes, pero no participa en la ejecución. Y el alcance es de carácter investigador: el resultado de los experimentos no se conoce de antemano, por lo que la planificación debe admitir revisiones.

Rasgo	Implicación
Equipo de una sola persona	El único recurso es la disponibilidad del alumno; planificar consiste en asignar sus horas
Dirección académica	El tutor valida el alcance y los cambios importantes, pero no participa en la ejecución
Alcance investigador	El resultado no se conoce de antemano; la planificación debe admitir revisión

Tabla 2. Rasgos del proyecto que condicionan su gestión.

La carga oficial del Trabajo Fin de Grado es de 12 créditos ECTS, equivalentes a unas 300 horas de dedicación. Esa cifra es la referencia con la que se compara el esfuerzo real en el apartado 3.6.

Se adoptó un ciclo de vida incremental [39], con fases cerradas y verificables antes de pasar a la siguiente. Cada fase produce un resultado comprobable, como un entorno que responde, un modelo con sus métricas o un experimento con sus resultados, de modo que un fallo queda acotado a la fase en curso.

3.2 Alcance del proyecto

El alcance acordado con el tutor comprende:

- La construcción de un detector de fraude sobre un conjunto de datos público de transacciones con tarjeta, con comparación de algoritmos y ajuste de hiperparámetros.
- La construcción de una capa de agentes basados en modelos de lenguaje locales que interprete la salida del detector y redacte un informe de revisión.

- Un conjunto de experimentos que mida el efecto de esa capa sobre la detección, con contrastes estadísticos.
- Un demostrador web local que permita ejecutar el sistema y observar sus resultados.
- La presente memoria.

Quedan fuera del alcance la integración con sistemas reales de una entidad financiera, el uso de datos reales de clientes y la evaluación con analistas humanos.

3.3 Planificación y recursos

El proyecto se desarrolla a lo largo de dos cuatrimestres, desde la definición del trabajo con el tutor a finales de abril hasta la entrega y la defensa previstas en octubre. La planificación se organizó en bloques de trabajo sucesivos, que se muestran en el diagrama de Gantt de la Figura 1. El diagrama representa cada bloque como una barra horizontal entre su fecha de inicio y de fin. En gris aparecen las tareas de documentación y estudio, tanto la definición inicial del trabajo como el estudio de las técnicas de detección de anomalías; en azul, la preparación del entorno de desarrollo, los primeros desarrollos y la construcción del sistema; en verde, la experimentación, el estudio del estado del arte, las extensiones exploratorias y el demostrador web; en rojo, la redacción y la relectura de la memoria; y con trama, los bloques todavía planificados a la fecha de esta versión, marcada con una línea discontinua: la incorporación de las correcciones del tutor y la preparación de la defensa.

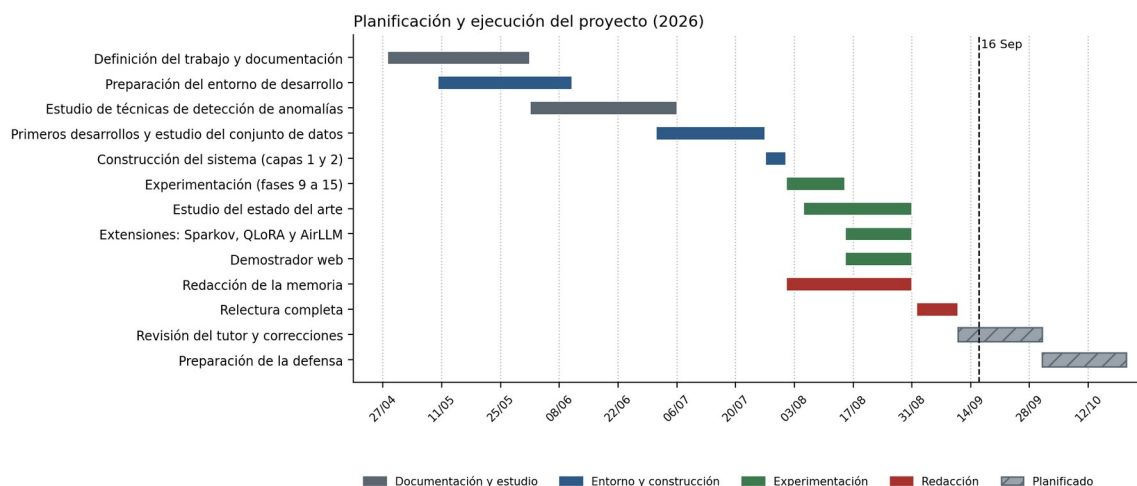


Figura 1. Planificación y ejecución del proyecto. Las barras con trama corresponden a tareas planificadas y la línea discontinua marca la fecha de esta versión de la memoria.

El reparto del tiempo es desigual a propósito. La etapa de documentación se solapa con el periodo de exámenes y avanza despacio; la construcción del sistema ocupa poco tiempo en proporción al total, porque todos sus componentes son locales y no dependen

de terceros; y el grueso de las horas se concentra en la experimentación, el estudio de la literatura y la redacción. El plan de las últimas semanas da más peso a la redacción que a la experimentación de forma deliberada: con la fase experimental cerrada, el mayor riesgo pasa a ser subestimar el tiempo de escritura.

La Tabla 3 recoge los recursos empleados. Todos los programas son libres o gratuitos, y el único equipo es el ordenador personal del alumno.

Tipo	Recurso
Humano	Un alumno, con dedicación variable según la carga académica y laboral
Hardware	Equipo personal con GPU NVIDIA RTX 5060 Ti de 16 GB
Software	Python, bibliotecas libres (XGBoost, scikit-learn, CrewAI, FastAPI), Ollama y Git
Coste económico	Nulo, por requisito del propio proyecto

Tabla 3. Recursos del proyecto.

3.4 Análisis de riesgos

La Tabla 4 recoge los riesgos identificados, con una estimación cualitativa de su probabilidad y su impacto y la medida de mitigación prevista para cada uno. El riesgo de dependencia de servicios externos se eliminó desde el diseño, al exigir que todo el sistema se ejecute en local. Los riesgos restantes se refieren a la validez de los resultados, a la pérdida del equipo de trabajo y al tiempo disponible.

Riesgo	Prob.	Impacto	Mitigación
Dependencia de un servicio externo que deje de estar disponible	Baja	Alto	Ejecución íntegramente local; ningún componente depende de terceros
Un experimento invalida una conclusión	Media	Medio	Todos los resultados se recalculan mediante guiones a partir de ficheros registrados
Fallo del equipo de cómputo	Baja	Alto	Repositorio remoto; los modelos se reentrenan en minutos
Subestimación del tiempo de redacción	Media	Alto	Se redacta en paralelo a la experimentación
Carga laboral superior a la prevista	Media	Medio	El plan admite recortar experimentos secundarios

Tabla 4. Registro de riesgos del proyecto.

Dos de estos riesgos llegaron a materializarse de forma parcial. Varias conclusiones intermedias tuvieron que corregirse al repetir o ampliar experimentos, como se documenta en el capítulo 6; la mitigación prevista permitió rehacer las tablas sin trabajo manual. Y el tiempo de redacción superó lo previsto, lo que se refleja en el apartado 3.6.

3.5 Ejecución y seguimiento

La Tabla 5 resume los hitos del proyecto, con la evidencia que respalda cada uno. Al tratarse de un proyecto unipersonal, el seguimiento se apoyó en indicadores objetivos del propio trabajo: fases cerradas con su registro en el repositorio, ejecuciones experimentales guardadas en fichero y auditorías automáticas superadas.

Fecha	Hito	Evidencia
28 abr	Definición del trabajo y adjudicación	Acta de la reunión
Mayo	Entorno de desarrollo operativo	Metadatos del proyecto
Jun – jul	Estudio de las técnicas de detección de anomalías y del conjunto de datos	Notas de trabajo
27 jul	Inicio del desarrollo del sistema definitivo	Primer registro en el repositorio
27 – 29 jul	Construcción completa del sistema en ocho fases	Ocho registros en el repositorio
1 – 14 ago	Experimentación (fases 9 a 15)	Catorce registros y 64 ficheros de resultados
15 – 30 ago	Extensiones exploratorias, demostrador web, estado del arte y redacción	Capturas, ficheros de resultados y borradores
1 – 10 sep	Relectura completa de la memoria	Versión revisada de la memoria
Septiembre	Revisión del tutor e incorporación de sus correcciones	Comentarios del tutor y esta versión
Octubre	Entrega y defensa	Planificado

Tabla 5. Hitos del proyecto y evidencia asociada.

3.6 Esfuerzo previsto frente a real

El esfuerzo no se registró mediante un parte de horas diario. Se reconstruye en la Tabla 6 a partir de la evidencia que el propio proyecto ha dejado: las fechas y horas de los registros en el repositorio, las marcas de tiempo de los ficheros de resultados, que indican el instante de cada ejecución, y las reuniones mantenidas. Para cada tramo se indica el número de días de trabajo efectivo, la dedicación media en esos días, el total de horas y la evidencia que lo respalda. Los días en los que no hubo dedicación, principalmente durante el periodo de exámenes, no se cuentan.

Tramo	Días	h/día	Total	Evidencia
28 abr – 31 may	10	2,0	20 h	Definición del trabajo con el tutor y documentación inicial
1 jun – 15 jul	12	2,9	35 h	Preparación del entorno de desarrollo y primeros desarrollos
16 – 26 jul	8	5,0	40 h	Estudio del conjunto de datos y diseño del sistema
27 – 31 jul	5	4,0	20 h	8 registros y 44 ficheros de resultados
1 – 14 ago	14	4,8	67 h	14 registros, 64 ficheros de resultados, 13.831 líneas
15 – 30 ago	16	6,8	110 h	Demostrador web, capturas, conjunto Sparkov, ajuste QLoRA, comparativa con AirLLM, estado del arte y redacción de los capítulos 4, 5 y 6
1 – 10 sep	10	2,0	20 h	Relectura completa de la memoria
Total a 10 sep	75	4,2	312 h	104 % de las 300 h nominales

Tabla 6. Esfuerzo dedicado por tramos, reconstruido a partir de la evidencia del proyecto.

El tramo del 1 al 14 de agosto fue el más intenso en experimentación: se cerró el objetivo específico OE4, incluida una ejecución desatendida de siete horas y cuarenta minutos, se construyeron dos guiones de experimentación completos y se leyeron cinco trabajos de la literatura.

El tramo del 15 al 30 de agosto agrupa trabajos distintos realizados en paralelo: la construcción del demostrador y sus capturas, la preparación del conjunto Sparkov, el ajuste fino con QLoRA, la comparación con AirLLM, la búsqueda y lectura de la literatura del capítulo 2 y la redacción de los capítulos 4, 5 y 6. A diferencia de los tramos anteriores, las horas de este tramo proceden de la estimación del propio autor y no de sumar registros o ficheros.

El proyecto supera ya la dedicación nominal de 300 horas, repartidas entre finales de abril y septiembre. La desviación se explica porque el tiempo que no fue necesario para construir el sistema se dedicó a ampliar la experimentación y a redactar y revisar la memoria. Al total indicado se sumarán las horas de incorporación de las correcciones del tutor y de preparación de la defensa.

3.7 Lecciones aprendidas

Las dependencias externas deben evitarse o verificarse antes que nada. La decisión de ejecutar todo en local eliminó el principal riesgo de disponibilidad y permitió construir el sistema en pocos días.

El análisis de riesgos no es un trámite. Identificar desde el principio que los resultados podían cambiar al ampliar los experimentos llevó a generar todas las tablas con guiones, lo que permitió corregirlas sin esfuerzo cuando fue necesario.

Dividir el trabajo en fases cerradas permite localizar los fallos. Cada vez que algo falló, se supo de inmediato en qué fase buscar.

Los resultados intermedios deben tratarse con cautela. Varias afirmaciones basadas en muestras pequeñas no se sostuvieron al ampliar los experimentos, lo que obligó a declarar el tamaño de muestra junto a cada conclusión.

4. Ejecución y desarrollo del producto

Este capítulo describe el sistema desarrollado. Empieza por su arquitectura general y por el conjunto de datos empleado. Después detalla las dos capas del sistema: el detector (capa 1), que es la contribución principal del trabajo, y el conjunto de agentes basados en modelos de lenguaje (capa 2). A continuación explica cómo se construye el escenario de evaluación y cómo se verifica automáticamente cada resultado. Termina con el demostrador web y con el seguimiento del desarrollo por fases.

4.1 Arquitectura general

El sistema separa la decisión de la explicación en dos capas con responsabilidades distintas. La capa 1 es un detector entrenado sobre los datos, que puntúa cada transacción y decide qué casos se revisan. La capa 2 es un conjunto de agentes basados en modelos de lenguaje que interpretan esa salida y redactan un informe de revisión y que, además, en uno de sus dos modos de funcionamiento deciden qué casos se descartan. Esa segunda atribución es la que hace que la capa 2 pueda alterar la fiabilidad de la detección, para bien o para mal, y no solo describirla; los dos modos se detallan en el apartado 4.4.4. Esta separación entre decidir y explicar es la hipótesis que el trabajo pone a prueba: si la segunda capa mejora o no lo que hace la primera.

El sistema procesa las transacciones por turnos. Un turno es el conjunto de todas las transacciones ocurridas en una ventana de tiempo, de cuatro horas salvo que se indique otra cosa, que el sistema analiza de una vez para producir un único informe, de forma parecida a como un equipo de analistas revisa las alertas de su turno de trabajo.

La Figura 2 representa la arquitectura y el recorrido completo de un turno. Los componentes deterministas, que ante la misma entrada producen siempre la misma salida, aparecen en azul; los estocásticos, cuya salida puede variar entre ejecuciones, en rojo; y la comprobación automática, en verde. El recorrido tiene cinco fases:

- Datos. El sistema recibe el turno, en el ejemplo de la figura 32.791 transacciones.
- Capa 1 (detector). XGBoost asigna a cada transacción una probabilidad de fraude. A continuación se calcula la contribución de cada variable a esa probabilidad mediante valores de Shapley (SHAP). Por último, se seleccionan los casos que superan el umbral de probabilidad, con un máximo fijado por la capacidad de revisión del equipo.
- Expediente. Los casos seleccionados, 55 en el ejemplo, forman el expediente: un documento que contiene, para cada caso, su identificador, su probabilidad, su

importe, su hora y las contribuciones de sus variables. Se construye sin intervención de ningún modelo de lenguaje.

- Capa 2 (agentes). El Modelador presenta los casos, el Investigador explica cada uno y emite un veredicto, y el Reportero redacta el informe. La tabla de casos del informe se ensambla directamente desde el expediente, sin pasar por el modelo, para que los identificadores no puedan alterarse. La figura muestra los tres agentes que intervienen sobre cada caso; los otros tres roles, descritos en el apartado 4.4.2, aportan contexto general al principio de la ejecución.
- Informe y auditoría. El informe resultante pasa por un auditor automático que comprueba que cubre todos los casos, que no inventa datos y que está redactado en el idioma pedido, y lo marca como fiable o no fiable.

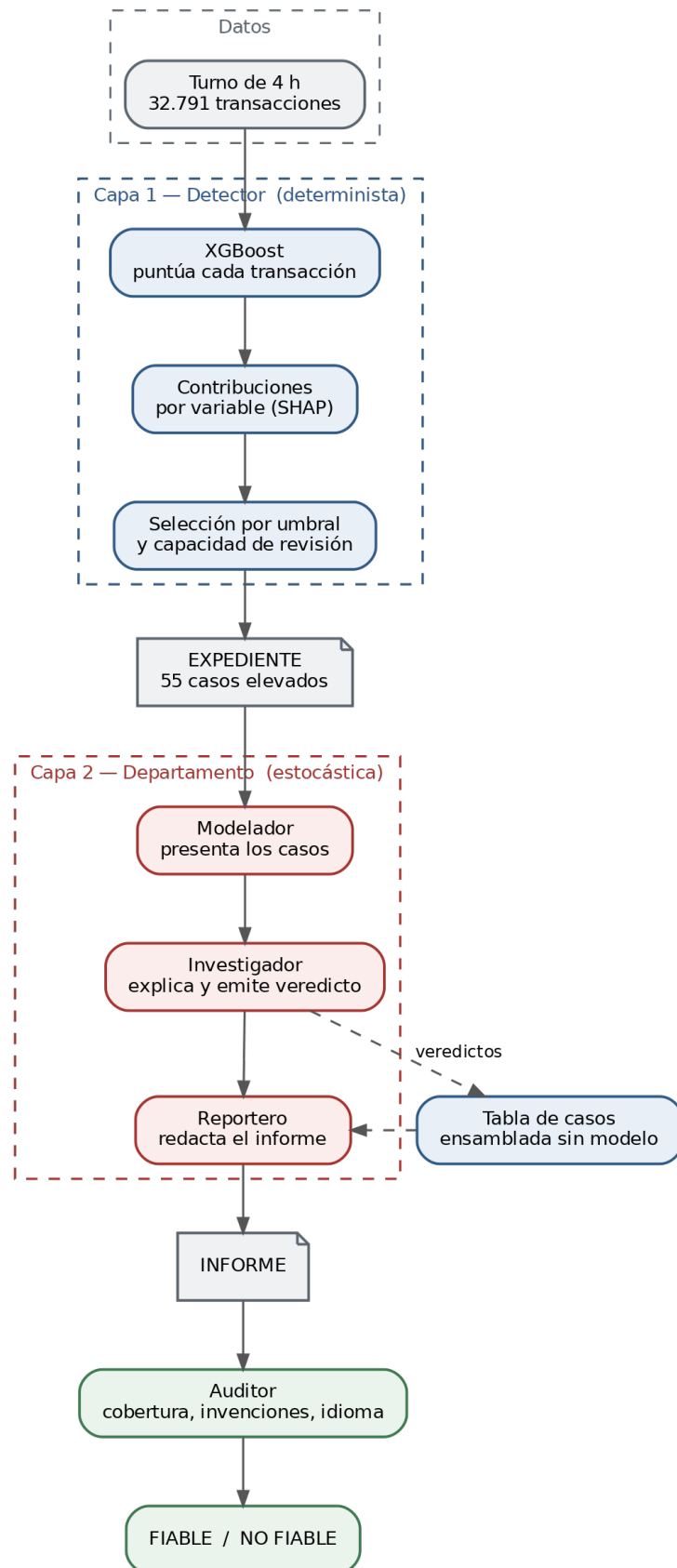


Figura 2. Arquitectura del sistema y recorrido completo de un turno. En azul, los componentes deterministas; en rojo, los estocásticos; en verde, la comprobación automática.

La Tabla 7 compara las dos capas. La capa 1 es un detector implementado con XGBoost, se entrena con los datos etiquetados, puntúa y prioriza, es determinista y procesa un turno completo en milisegundos. La capa 2 está formada por seis agentes en CrewAI, emplea modelos preentrenados que nunca ven las etiquetas, interpreta y redacta, es estocástica y tarda entre cuatro y cinco minutos por turno.

	Capa 1: detector	Capa 2: agentes
Qué es	Detector implementado con XGBoost	Seis agentes en CrewAI
Entrenamiento	Entrenado sobre los datos	Preentrenado; nunca ve las etiquetas
Función	Puntuar y priorizar	Interpretar y redactar
Determinismo	Determinista	Estocástico
Coste por turno	Milisegundos	Cuatro a cinco minutos

Tabla 7. Comparación entre las dos capas del sistema.

Como la capa 2 solo recibe el expediente, su coste no depende del volumen de transacciones sino del número de casos elevados. Por eso el sistema puede aplicarse a volúmenes de operaciones reales.

La Figura 3 detalla el recorrido de un caso individual. A partir de las variables de la transacción, el detector calcula la probabilidad de fraude y las contribuciones por variable. De la probabilidad se deriva además un nivel de riesgo (alto, medio o bajo) según el umbral aplicado. Los tres elementos forman el dossier del caso, que recibe el Investigador. El Investigador produce dos salidas: un párrafo explicativo, que va al texto del informe, y un veredicto, confirmado o descartado, que determina las métricas del sistema. La línea de puntos roja indica que ambas salidas no siempre se corresponden, uno de los resultados principales del trabajo (experimento E10). La flecha discontinua indica que el nivel de riesgo se retira en uno de los experimentos (E7).

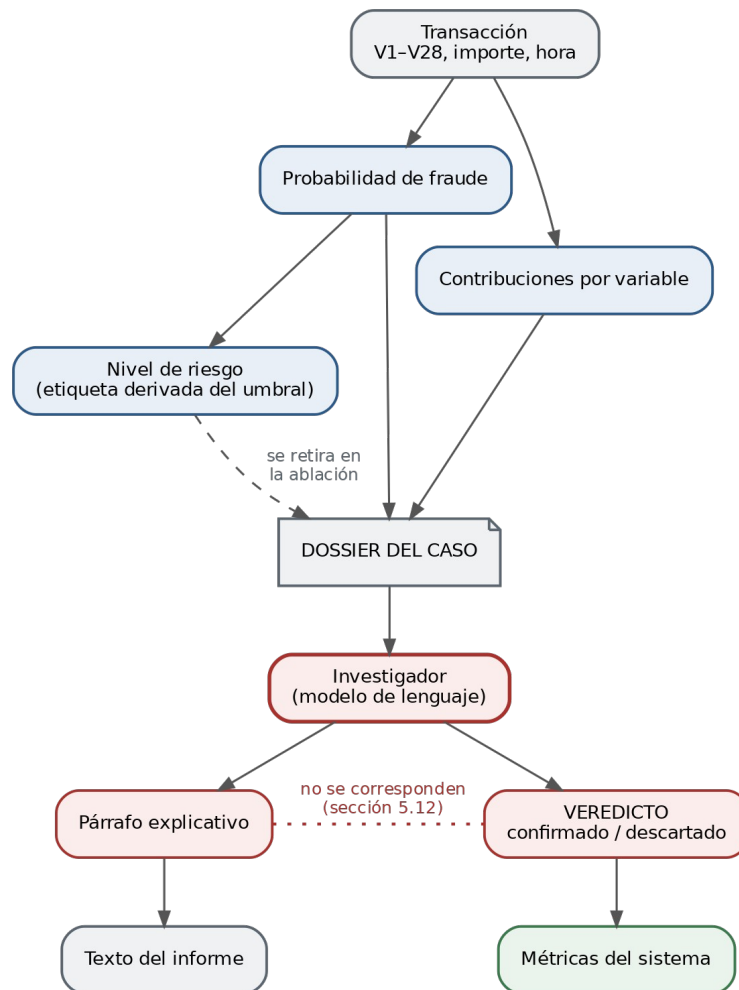


Figura 3. Recorrido de un caso individual desde la transacción hasta sus dos salidas: el párrafo explicativo y el veredicto.

La Figura 4 muestra un caso real tal como lo presenta el demostrador web. A la izquierda aparece lo que calcula el detector: una probabilidad de 0,0673, un importe de 2,22 euros, la hora y las contribuciones de las variables, en rojo las que empujan hacia fraude y en azul las que empujan hacia legítima. A la derecha aparece el texto literal que recibe el modelo de lenguaje. Aunque la probabilidad es baja, la clase real de esta transacción es fraude.

Departamento antifraude

Detector tabular con una capa de agentes encima. Todo se ejecuta en local.

✓ conjunto de datos ✓ detector entrenado ✓ Ollama (10 modelos)

Turno activo: 4 h desde +0 h umbral 0.05 capacidad 60 55 casos en el expediente (numerados del 1 al 55) 44 son fraude real

1 Turno 2 **Caso** 3 Investigación 4 Informe

2. El caso, tal como lo ve el modelo

A la izquierda, las cifras que calcula el detector. A la derecha, el texto exacto que se incrusta en el enunciado. El desplegable reproduce la ablación de la sección 5.7: retira información y observa qué deja de ver el modelo.

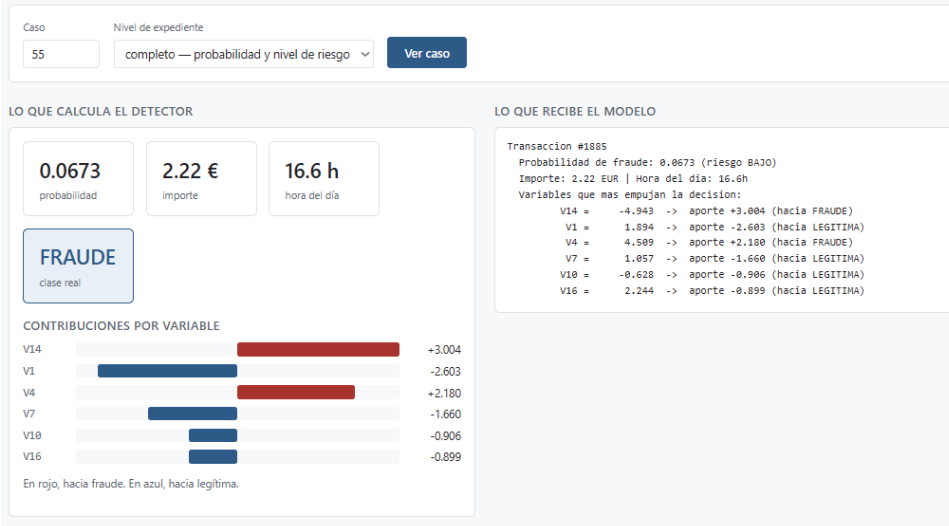


Figura 4. El caso 55 en el demostrador: a la izquierda, lo que calcula el detector; a la derecha, el dossier literal que recibe el modelo.

La capa 1 no depende de ningún modelo de lenguaje y puede ejecutarse y evaluarse sin cargar ninguno. Esta independencia resultó útil durante todo el desarrollo: cuando algo fallaba, se sabía de inmediato en qué capa buscar.

4.2 El conjunto de datos

4.2.1 Origen y composición

Se emplea el conjunto Credit Card Fraud Detection, publicado por el grupo de aprendizaje automático de la Universidad Libre de Bruselas (ULB) junto con la empresa Worldline [2, 8]. Contiene 284.807 transacciones realizadas por titulares de tarjetas europeos durante dos días de septiembre de 2013, de las cuales 492 son fraudulentas, un 0,173 %.

Las variables V1 a V28 se obtuvieron mediante análisis de componentes principales (PCA), una transformación que combina las variables originales en otras nuevas. Los publicadores la aplicaron para proteger la confidencialidad de los datos, de modo que estas variables no tienen un significado de negocio conocido. Solo el tiempo y el importe conservan su significado original.

Se ha elegido porque es el conjunto de referencia en la literatura de detección de fraude, lo que permite comparar las métricas obtenidas con resultados publicados, y porque su desbalanceo extremo es representativo del problema real.

Tiene una limitación importante: al estar anonimizado, las explicaciones no pueden referirse a conceptos de negocio. El sistema puede afirmar que la variable V14 tiene un valor muy poco habitual, pero no que la compra se hizo en un comercio inusual. Es una limitación del conjunto de datos, no del sistema, y la auditoría automática está diseñada para detectar cualquier explicación que invente ese tipo de información. El experimento E11 del capítulo 6 explora un conjunto con variables de negocio.

La Figura 5 muestra la distribución de los importes de cada clase sobre el conjunto completo. El panel izquierdo representa, para cada importe, la proporción de transacciones de cada clase que no lo superan. El salto vertical de la curva roja en un euro indica que muchas transacciones fraudulentas son exactamente de ese importe. El panel derecho muestra el porcentaje de cada clase por debajo de 1, 2, 5 y 10 euros: el 45 % de los fraudes no supera los 5 euros, frente al 24 % de las transacciones legítimas. Los importes bajos son, por tanto, más frecuentes en el fraude que en las operaciones legítimas, un dato que resulta clave en los experimentos E9 y E10.

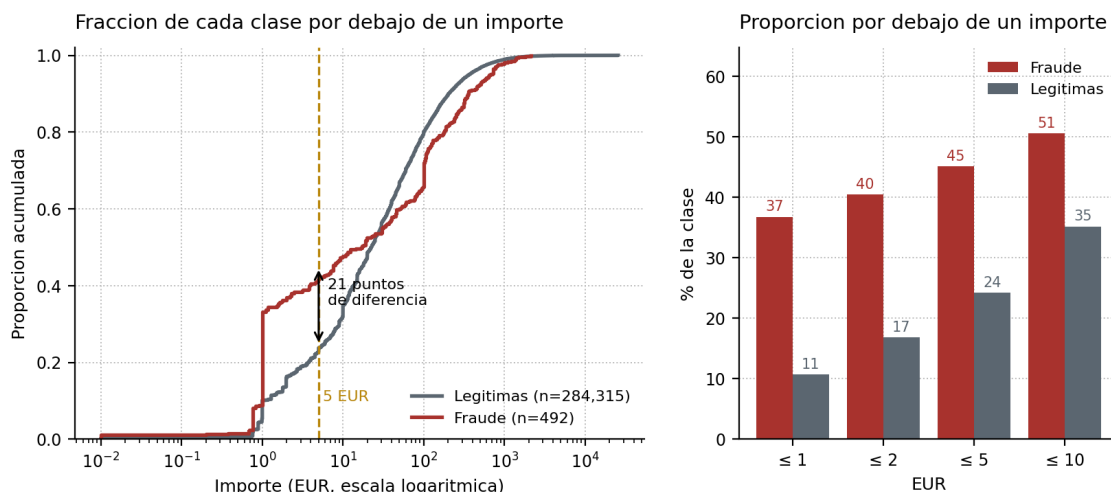


Figura 5. Distribución de importes por clase sobre el conjunto completo. A la izquierda, proporción acumulada; a la derecha, porcentaje de cada clase por debajo de varios importes.

4.2.2 Ingeniería de variables

El detector emplea las veintiocho componentes principales, el importe y una variable horaria derivada. Dos decisiones requieren justificación.

El tiempo no se usa directamente como variable. La variable original indica los segundos transcurridos desde la primera transacción del periodo. En una partición temporal, los valores del conjunto de prueba quedan fuera del rango visto durante el

entrenamiento, y un modelo basado en árboles no sabe extrapolar a esos valores, de modo que la variable no generalizaría a datos nuevos.

La hora del día sí se usa. Se obtiene como el resto de dividir entre 24 el número de horas transcurridas. Se incluyó porque el fraude suele concentrarse de madrugada y porque su rango es siempre el mismo. El experimento E2 comprueba que, en este conjunto de datos, su aportación es nula.

4.3 Capa 1: el detector

4.3.1 Elección de los algoritmos

Se implementan tres modelos, dos supervisados, que aprenden a partir de ejemplos etiquetados como fraude o legítima, y uno no supervisado, que no utiliza las etiquetas. La Tabla 8 resume cómo se construye cada uno y qué busca.

Modelo	Aprendizaje	Construcción	Qué busca
XGBoost	Supervisado	Árboles en secuencia; cada uno corrige los errores de los anteriores	La frontera entre clases
Random Forest	Supervisado	Árboles independientes entrenados en paralelo; se promedia su voto	La frontera entre clases
Isolation Forest	No supervisado	Cortes aleatorios; mide cuánto cuesta aislar cada punto	Las observaciones poco frecuentes

Tabla 8. Algoritmos implementados en el detector.

XGBoost [3] es el detector principal. Se eligió por tres motivos. El primero, su rendimiento en el dominio: es el algoritmo en el que se apoya FraudX AI [10], que obtiene con él una exhaustividad del 95 % sobre el mismo conjunto de datos que emplea este trabajo, y pertenece a la familia de algoritmos de refuerzo de gradiente, la más empleada en detección de fraude. El segundo, su tratamiento nativo del desbalanceo mediante la ponderación de la clase minoritaria. El tercero, que calcula de forma exacta y sin bibliotecas adicionales la contribución de cada variable a cada predicción, como se explica en el apartado 4.3.4.

Random Forest [40] se añadió a propuesta del tutor para comparar con datos la elección de XGBoost. El resultado de esa comparación, en el experimento E1, muestra que ambos modelos son prácticamente equivalentes en los primeros casos de la lista.

Isolation Forest [41] se incluye como referencia no supervisada. Detecta anomalías midiendo cuántos cortes aleatorios hacen falta para aislar cada observación: las observaciones raras quedan aisladas con pocos cortes. Resulta útil cuando no hay

etiquetas disponibles, y su comparación con los modelos supervisados permite cuantificar cuánto se gana al disponer de ellas.

En el sistema en funcionamiento solo opera XGBoost. Es el modelo que puntúa cada transacción y el que calcula las contribuciones por variable que se incluyen en el expediente. El Isolation Forest se calcula en cada puntuación, pero su resultado no llega a la capa 2. Random Forest no se utiliza en el flujo operativo: existe únicamente como término de comparación.

4.3.2 Tratamiento del desbalanceo

Con una prevalencia del 0,173 %, un modelo que prediga siempre la clase mayoritaria acierta el 99,83 % de las veces, así que el desbalanceo debe tratarse de forma explícita.

Existen cuatro familias de soluciones: eliminar ejemplos de la clase mayoritaria (submuestreo), repetir ejemplos de la minoritaria (sobremuestreo), generar ejemplos sintéticos de la minoritaria con técnicas como SMOTE, o dar más peso a los errores cometidos sobre la clase minoritaria. Se eligió la última, mediante el parámetro `scale_pos_weight` de XGBoost, que multiplica el peso de cada fraude en el entrenamiento por la proporción entre transacciones legítimas y fraudulentas, unas 578 veces.

Esta opción no descarta información, a diferencia del submuestreo, y no inventa observaciones, a diferencia del sobremuestreo sintético. Además, como se explicó en el apartado 2.2.1, no puede provocar fuga de información.

4.3.3 Las dos particiones

Todos los modelos se evalúan sobre dos particiones distintas de los datos.

La partición aleatoria reparte las transacciones al azar entre entrenamiento (80 %) y prueba (20 %), conservando la proporción de fraude en ambos. Es la que emplea la mayor parte de la literatura y permite la comparación con ella.

La partición temporal ordena las transacciones por tiempo y reserva el primer 80 % para entrenamiento y el 20 % final para prueba: se entrena con el pasado y se evalúa con el futuro. Es la que reproduce las condiciones reales de uso y, por eso, es la que se usa para el modelo que alimenta a los agentes.

Reportar ambas no es redundante. Como muestra el experimento E1, la conclusión sobre qué modelo es mejor depende de la partición elegida.

4.3.4 Explicabilidad sin dependencias adicionales

La capa 2 necesita saber por qué el detector considera sospechosa cada transacción. Para ello, cada predicción se descompone en contribuciones por variable mediante valores de Shapley. Este concepto procede de la teoría de juegos cooperativos [11] y reparte el resultado de un juego entre los jugadores según lo que aporta cada uno; aplicado a un modelo, reparte la predicción entre las variables de entrada [12]. Una contribución positiva empuja la predicción hacia fraude y una negativa, hacia legítima.

XGBoost calcula estos valores de forma exacta para sus árboles, sin instalar bibliotecas adicionales. La suma de todas las contribuciones más un valor base reproduce exactamente la puntuación del modelo antes de convertirla en probabilidad. Por tanto, la descomposición es fiel al modelo y no una aproximación. Existen implementaciones exactas equivalentes para Random Forest en bibliotecas externas, de modo que esta propiedad no es exclusiva de XGBoost, pero sí evita añadir dependencias.

4.4 Capa 2: el departamento de agentes

La capa 2 simula un pequeño departamento antifraude formado por seis agentes, cada uno con un papel. Este apartado explica qué marco se ha usado para coordinarlos, qué hace cada agente, cómo se les entrega la información del detector y qué grados de autoridad pueden tener sobre las decisiones.

4.4.1 Marco de orquestación

Como se explicó en el apartado 2.3, un marco de orquestación es la biblioteca que define los agentes, les asigna tareas, fija el orden en que intervienen y hace llegar a cada uno lo que han producido los anteriores. Este trabajo emplea CrewAI [5], un marco de código abierto para Python. En CrewAI, cada agente se define con un papel (role), un objetivo (goal), un trasfondo (backstory) que describe su perfil y las herramientas que puede usar. Las tareas se definen por separado, indicando qué agente las realiza, qué resultado se espera y de qué tareas anteriores reciben contexto. Por último, un equipo (Crew) reúne agentes y tareas y fija el proceso de ejecución. La Figura 6 muestra cómo se definen en el código del proyecto los parámetros comunes a todos los agentes, uno de los seis agentes y el equipo completo. El modelo de lenguaje de cada agente se ejecuta en local a través de Ollama, y la telemetría de CrewAI está desactivada para que ningún dato salga de la máquina.

```

src/agents/crew.py — parámetros comunes y agente Coordinador
114     comun = dict(
115         llm=llm,
116         allow_delegation=False,
117         verbose=verbose,
118         max_iter=3,
119         max_execution_time=180, # segundos por agente
120     )
121
122     coordinador = Agent(
123         role="Coordinador del departamento antifraude",
124         goal=(
125             "Encuadrar el analisis del lote: cuantas transacciones hay, cuanto "
126             "dinero esta en juego y que volumen de casos exige revision manual."
127         ),
128         backstory=(
129             "Diriges un equipo de analistas en un banco. No revisas casos uno a "
130             "uno: tu trabajo es dimensionar el problema y decidir donde poner la "
131             "atencion del equipo, que siempre es un recurso escaso."
132         ) + IDIOMA,
133         tools=[EstadisticasLoteTool()],
134         **comun,
135     )

src/agents/crew.py — ensamblado del equipo (Crew) con proceso secuencial
477     return Crew(
478         agents=list(agentes.values()),
479         tasks=tareas,
480         process=Process.sequential,
481         verbose=verbose,
482         # Sin telemetria. CrewAI ofrece enviar trazas de ejecucion a
483         # app.crewai.com, lo que incluiria prompts, respuestas del LLM y
484         # salidas de las herramientas: es decir, datos de transacciones
485         # saliendo de la maquina. El TFG sostiene que el sistema es
486         # enteramente local, asi que esto queda desactivado por diseno.
487         tracing=False,
488     )

```

Figura 6. Definición de los agentes en CrewAI dentro del entorno de desarrollo: parámetros comunes, agente Coordinador y ensamblado del equipo con proceso secuencial.

CrewAI ofrece dos procesos de ejecución. En el proceso secuencial, las tareas se ejecutan en un orden fijo definido por el programador. En el proceso jerárquico, un agente gestor adicional decide en cada momento a qué agente asigna cada tarea, de forma parecida al responsable de un departamento real.

Este trabajo emplea el proceso secuencial, y el proceso jerárquico se descartó por dos motivos. El primero es práctico: el proceso jerárquico exige que el agente gestor formule correctamente muchas llamadas a herramientas para delegar las tareas, y con el modelo más pequeño evaluado, Llama 3.1 de 8B, el gestor entraba en bucles de delegación hasta agotar el número máximo de intentos. El segundo es propio del dominio: en un contexto que debe poder auditarse, un orden de intervención fijo es una ventaja, porque ante un regulador la forma en que se generó un informe se explica con un diagrama y no con una descripción probabilística. El proceso jerárquico solo se probó con Llama 3.1 de 8B, por lo que esta conclusión no puede extenderse a modelos de mayor tamaño.

4.4.2 Los seis roles

La Tabla 9 describe los seis agentes, su función y las herramientas que tiene disponibles cada uno. Intervienen en el orden de la tabla. Las herramientas son funciones del programa que consultan el detector: las estadísticas del turno, la lista priorizada de casos sospechosos y el rendimiento medido del detector.

Agente	Función	Herramientas
Coordinador	Dimensiona el turno recibido	Estadísticas
Analista de datos	Describe la composición del turno y sus patrones	Estadísticas, priorización
Modelador	Presenta los casos elevados	Priorización
Evaluador	Explica cuánta confianza merece el detector	Rendimiento
Investigador	Explica cada caso y emite su veredicto	Ninguna
Reportero	Redacta el informe final	Ninguna

Tabla 9. Agentes de la capa 2, su función y sus herramientas.

Cada agente recibe únicamente las herramientas propias de su papel. Una lista corta de herramientas reduce la probabilidad de que el modelo use la herramienta equivocada, un problema frecuente en modelos pequeños.

El Investigador no tiene herramientas por un motivo concreto. Inicialmente disponía de la herramienta de explicación, que devolvía una tabla ya formateada, y el agente se limitaba a copiarla en lugar de redactar el análisis pedido. Al retirársela, y como el expediente completo ya figura en su enunciado, la única respuesta posible pasó a ser el texto solicitado. La medida ahorra además cuatro llamadas al modelo por ejecución.

La Figura 7 muestra la traza que CrewAI imprime durante una ejecución. En ella se ve cómo el Coordinador termina su tarea con un resumen del turno y cómo el Analista de datos recibe a continuación la suya, con la instrucción de describir la composición del turno.

```
Riesgo MEDIO (verde) : 2
Riesgo BAJO (verde) : 32747
Probabilidad maxima : 1.0000
Probabilidad media : 0.0014
Importe total del lote : 2,762,649.67 EUR
Importe medio : 84.25 EUR
Franja horaria : 16.3h - 20.3h

[Finalize] todos_count=0, todos_with_results=0
Agent: Coordinador del departamento antifraude
Final Answer:
El lote contiene un total de 22,791 transacciones, con 42 (0.1%) clasificadas como riesgo alto y 2 (0.0%) en riesgo medio. El importe total del lote es de 2,762,649.67 EUR, siendo el importe asociado a las transacciones de riesgo alto significativamente más elevado que la media.

Task Completion
Task Completed
Name: Consulta las estadísticas del lote con la herramienta disponible. Resume en 3 o 4 frases: cuantas transacciones hay, cuantas de riesgo alto, el importe total y el importe en riesgo.
Agent: Coordinador del departamento antifraude

Task Started
Task Started
Name: Describe la composición del lote. Comenta el reparto de importes y la franja horaria. Si observas que las transacciones de riesgo alto tienen importes llamativamente bajos, dilo explícitamente.
Recuerda: V1 a V26 son componentes PCA sin significado de negocio.
ID: e777f215-8799-40b4-bb33-4f08fb5986c9

Agent Started
Agent: Analista de datos
Task: Describe la composición del lote. Comenta el reparto de importes y la franja horaria. Si observas que las transacciones de riesgo alto tienen importes llamativamente bajos, dilo explícitamente.
Recuerda: V1 a V26 son componentes PCA sin significado de negocio.
```

Figura 7. Traza de CrewAI durante una ejecución: el Coordinador entrega su resumen del turno y el Analista de datos comienza su tarea.

4.4.3 El expediente determinista

El expediente es el documento que la capa 1 entrega a la capa 2, descrito en el apartado 4.1: la lista de casos elevados con, para cada uno, su identificador, su probabilidad de fraude, su importe, su hora y la contribución de cada variable. Se le llama determinista porque se construye íntegramente por programa, sin que ningún modelo de lenguaje intervenga en su contenido, y se inserta literalmente en el enunciado de cada agente que lo necesita; ante el mismo turno es siempre idéntico, carácter que da nombre al apartado. Esta es la decisión de diseño más importante del sistema, y surgió de un error.

En la primera versión, los identificadores de las transacciones pasaban de un agente a otro a través del texto generado por el modelo. En una ejecución con Llama 3.1 de 8B, un agente omitió la tabla de casos en su respuesta. El agente siguiente se quedó sin identificadores y se los inventó; la herramienta devolvió errores al no encontrarlos, y el agente respondió describiendo cinco transacciones inexistentes, con importes en dólares y variables que nunca había consultado. El informe final tenía un formato correcto, pero su contenido era falso.

La conclusión es de diseño: un dato que debe llegar intacto no debe pasar por un componente estocástico. Los identificadores proceden del detector, y hacerlos pasar por un modelo de lenguaje para que lleguen al siguiente agente es arriesgarse a que se pierdan o se alteren.

La corrección consiste en calcular el expediente completo (identificadores, probabilidades, importes y contribuciones por variable) sin intervención del modelo e

insertarlo literalmente en el enunciado de cada agente que lo necesita. Al modelo se le pide que describa datos que tiene delante, no que los recuerde.

Una segunda corrección surgió al pasar de lotes sintéticos pequeños a turnos reales. Los casos se numeran del uno al total del expediente, en lugar de usar su índice original en el conjunto de datos. Con lotes pequeños los índices tenían una o dos cifras y no causaban problemas; con turnos reales pasaron a tener cuatro o cinco, y el modelo empezó a copiarlos mal, cambiando un dígito por otro. Un número largo se divide en varios tokens y es fácil que el modelo altere uno, mientras que un número pequeño suele ser un único token. La correspondencia entre el número de caso y el índice real se mantiene fuera del alcance del modelo.

4.4.4 Los dos modos de autoridad

El experimento central de la capa 2 consiste en variar cuánta autoridad se concede a los agentes sobre la lista de casos. Se han definido dos modos de autoridad, es decir, dos formas de repartir la última palabra sobre qué casos aparecen en el informe.

En el modo interpreta, el detector decide qué casos se elevan y los agentes se limitan a explicarlos y a redactar el informe, sin poder quitar ninguno. La fiabilidad de la detección del sistema es, por construcción, la misma que la del detector.

En el modo revisa, el Investigador emite un veredicto para cada caso, confirmado o descartado, y los casos descartados se retiran del informe. La fiabilidad del sistema puede disminuir respecto a la del detector, si se descarta un fraude real, pero nunca puede aumentar.

Esta asimetría refleja la realidad del problema: una capa posterior no puede recuperar un fraude que el detector no elevó, porque nunca llega a verlo. En modo revisa, lo único que la capa 2 puede mejorar es la precisión, retirando falsas alarmas.

4.5 El escenario de evaluación

Los primeros experimentos empleaban lotes sintéticos de 500 transacciones con cinco fraudes añadidos. Ese diseño tiene dos problemas. Ni el tamaño ni la proporción de fraude se corresponden con la realidad, y con cinco fraudes la exhaustividad solo puede tomar seis valores (0, 0,2, 0,4 y así hasta 1), de modo que cualquier diferencia parece enorme.

El escenario definitivo extrae turnos reales del periodo de prueba de la partición temporal: todas las transacciones de un intervalo de cuatro horas, con la proporción de

fraude que tenían realmente. Un turno contiene unas treinta mil transacciones y alrededor de medio centenar de fraudes.

Cada turno se identifica por su desplazamiento respecto al inicio del periodo de prueba: el turno de desplazamiento 0 h cubre las cuatro primeras horas del periodo, el de 1 h va de la hora 1 a la 5, el de 2 h de la 2 a la 6, y así sucesivamente. Como el periodo de prueba dura menos de ocho horas, turnos consecutivos comparten aproximadamente la mitad de sus transacciones.

La Tabla 10 describe los cuatro turnos empleados: el número de transacciones y de fraudes de cada uno, el número de casos que el detector eleva al expediente con un umbral de 0,05, y cuántos de esos casos son falsos positivos, es decir, transacciones legítimas señaladas como sospechosas.

Desplazamiento	Transacciones	Fraudes	Expediente	Falsos positivos
0 h	32.791	54	55	11
1 h	31.981	51	53	12
2 h	31.327	38	42	15
3 h	28.854	26	32	14

Tabla 10. Turnos de evaluación y composición de su expediente con umbral 0,05.

La Figura 8 muestra la construcción del turno de desplazamiento 0 h en el demostrador: se indican la duración, el desplazamiento, el umbral y la capacidad de revisión, y el detector puntúa las 32.791 transacciones y eleva 55 casos, de los cuales 44 son fraude, en 264 milisegundos. La columna de clase real solo se muestra para poder juzgar las decisiones del sistema; ningún componente la consulta.

Departamento antifraude

Detector tabular con una capa de agentes encima. Todo se ejecuta en local.

✓ conjunto de datos ✓ detector entrenado ✓ Ollama (10 modelos)

Turno activo: 4 h desde +0 h umbral 0,05 capacidad 60 55 casos en el expediente (numerados del 1 al 55) 44 son fraude real

1 Turno 2 Caso 3 Investigación 4 Informe

1. Construir un turno y detectar

La capa 1 puntúa todas las transacciones del periodo y eleva a revisión las que superan el umbral, hasta agotar la capacidad del equipo.

Duración (h)	Desplazamiento (h)	Umbral	Capacidad (casos)	Detectar
4	0	0,05	60	

32.791

transacciones

54

fraudes en el turno

55

casos elevados

44

de ellos, fraude

264.2 ms

detección

#	IDENTIFICADOR	PROBABILIDAD	RIESGO	IMPORTE	HORA	CLASE REAL
1	15904	1.0000	ALTO	1.00 €	18.2 h	FRAUDE
2	1867	1.0000	ALTO	1.18 €	16.6 h	FRAUDE
3	15854	1.0000	ALTO	1.00 €	18.2 h	FRAUDE
4	24929	1.0000	ALTO	0.77 €	19.3 h	FRAUDE
5	2631	1.0000	ALTO	94.82 €	16.7 h	FRAUDE
6	16003	1.0000	ALTO	1.00 €	18.2 h	FRAUDE
7	20150	1.0000	ALTO	51.37 €	18.7 h	FRAUDE
8	23632	1.0000	ALTO	0.77 €	19.2 h	FRAUDE
9	24279	1.0000	ALTO	0.77 €	19.2 h	FRAUDE
10	7799	1.0000	ALTO	122.68 €	17.2 h	FRAUDE
11	16159	1.0000	ALTO	1.00 €	18.3 h	FRAUDE

La columna de clase real no la ve el sistema en ningún momento: se muestra sólo para poder juzgar sus decisiones. Pulsa una fila para inspeccionar el caso.

Figura 8. Construcción y puntuación del turno de evaluación en el demostrador: 32.791 transacciones y 54 fraudes, de los que el detector eleva 55 casos con 44 fraudes.

Al solaparse, los cuatro turnos no son muestras independientes entre sí. A ello se suma que las repeticiones dentro de cada turno, con temperatura 0,1, son casi idénticas. Por tanto, el número de observaciones realmente independientes en las comparativas del capítulo 6 está más cerca de cuatro que del número total de ejecuciones. Esta limitación se recoge en el capítulo 7.

4.6 Instrumentación de la evaluación

La instrumentación es el conjunto de mecanismos que verifican y miden automáticamente cada ejecución. Es lo que convierte el sistema en un experimento evaluable y no en una simple demostración. Opera en tres niveles.

4.6.1 Auditoría de fiabilidad

El informe se genera en la última fase del recorrido de un turno descrito en el apartado 4.1: el agente Reportero lo redacta a partir de las explicaciones y los veredictos que el Investigador ha producido para cada caso del expediente, y su tabla de casos se ensambla directamente desde el expediente, sin pasar por el modelo. Es ese informe, ya terminado, el que recibe el auditor automático.

Cada informe generado se verifica automáticamente en dos planos. El primero comprueba que todos los identificadores citados existen en el expediente y qué porcentaje de los casos del expediente cubre el informe. El segundo detecta la invención de conceptos: menciones a países, comercios, cuentas, titulares o historial del cliente, datos que no existen en el conjunto empleado.

El segundo plano es necesario porque el primero no basta. En una ejecución real, el sistema citó correctamente los cuatro identificadores y, aun así, el informe era falso, porque atribuía a las transacciones un país de origen de alto riesgo que no figura en los datos.

La Figura 9 muestra el resultado de la auditoría en el demostrador para un informe en modo revisa. El auditor marca el informe como incompleto: cubre el 98 % del expediente, pero 21 casos se quedaron sin veredicto y, por diseño, conservan la decisión del detector. Debajo aparecen las métricas del turno y la tabla de casos ensamblada sin intervención del modelo.

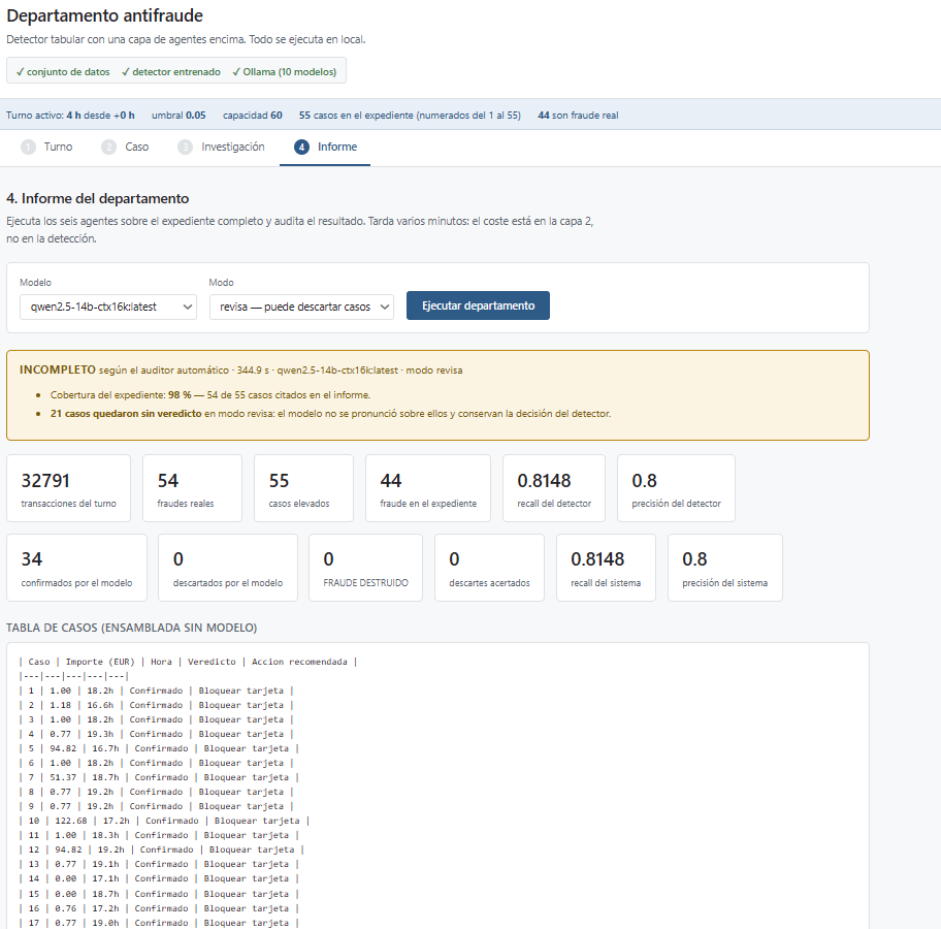


Figura 9. Auditoría de un informe en modo revisa: cobertura del 98 % y aviso de que 21 casos quedaron sin veredicto y conservan la decisión del detector.

4.6.2 Evaluación contra la clase real

Cada transacción del turno conserva su clase real (fraude o legítima), pero ningún componente la consulta durante la ejecución: ni el detector al puntuar ni los agentes en ningún momento. Solo se utiliza al final, para calcular las métricas del turno: la precisión y la exhaustividad del detector y las del sistema completo, el número de fraudes que la capa 1 eleva al expediente, el número que permanece tras la capa 2 y el fraude destruido, es decir, los fraudes que el detector había elevado y que la capa 2 descartó. Todas ellas se definen en el apartado 6.1.1. Es, por tanto, una evaluación a ciegas.

La medición distingue tres niveles. El primero es el fraude que eleva la capa 1, que marca el máximo alcanzable por el sistema. El segundo es el fraude que permanece tras la capa 2. El tercero son los casos concretos que se perdieron entre ambos. Este último nivel es el más informativo: saber que se descartaron dos casos aporta poco, mientras que saber que uno de ellos era fraude real es lo que permite valorar el sistema.

4.6.3 Experimentación reproducible

Los dos niveles anteriores verifican una ejecución concreta. El tercero persigue algo distinto: que el conjunto de los experimentos del capítulo 6 pueda repetirse y produzca las mismas cifras. Para ello hacen falta tres cosas, que se resuelven como sigue: recorrer de forma sistemática todas las combinaciones que compara cada experimento, no perder el trabajo ya hecho si una ejecución larga se interrumpe, y calcular las métricas con el mismo código tanto en los experimentos como en una ejecución individual del sistema.

Un guion recorre todas las combinaciones de modelo, modo de autoridad, turno y repetición. Guarda el resultado tras cada ejecución, para que una interrupción no haga perder el trabajo anterior, y genera tablas agregadas. Toda la lógica de medida está en un único módulo que comparten los experimentos y la ejecución individual, de modo que los informes concretos y las tablas de la memoria no pueden contradecirse. Sobre los datos agregados se aplican los contrastes estadísticos descritos en el capítulo 6.

4.7 Demostrador web

El sistema incluye una aplicación web que se ejecuta en el propio ordenador del usuario, construida con FastAPI [7]. Cumple dos funciones. La primera es permitir usar el sistema sin escribir ninguna orden en la consola. La segunda es permitir observar por separado cada fase del recorrido de un turno, desde la puntuación del detector hasta la auditoría del informe, que de otro modo solo quedarían reflejadas en los ficheros de resultados.

El demostrador no contiene una segunda implementación del sistema: llama a las mismas funciones que los guiones de experimentación. Lo que aparece en pantalla es, por tanto, el mismo sistema que produce las cifras del capítulo 6, y no una maqueta. Su uso se describe paso a paso en el manual de usuario del apéndice C.

Consta de cuatro pantallas que siguen el recorrido de un turno:

- Turno: construye el turno, lo puntúa y muestra el expediente, con una columna adicional con la clase real de cada caso, que el sistema nunca consulta y que solo sirve para juzgar sus decisiones.
- Caso: muestra un caso tal como lo calcula el detector y, al lado, el texto exacto que se inserta en el enunciado del modelo, con un selector para retirar información del dossier como en el experimento E7.
- Investigación: ejecuta el Investigador sobre un caso y presenta en columnas separadas el párrafo explicativo y el veredicto, porque uno de los resultados principales del trabajo es que no siempre se corresponden.
- Informe: ejecuta el departamento completo y somete el informe al auditor.

La Figura 10 muestra la pantalla inicial. En la parte superior aparecen los indicadores del entorno (conjunto de datos cargado, detector entrenado y número de modelos disponibles en Ollama) y debajo las cuatro pantallas del recorrido.

Figura 10. Pantalla inicial del demostrador, con los indicadores de entorno y las cuatro etapas del recorrido.

4.8 Seguimiento del desarrollo

El desarrollo se organizó en fases cerradas, cada una con su registro en el control de versiones y un resultado verificable. La Figura 11 muestra el historial del repositorio, en el que cada línea corresponde a un registro y los mensajes identifican la fase a la que pertenece.

```
(.venv) PS C:\Users\Adrián\Desktop\UNI\tfg-multiagente-fraude> git --no-optional-locks log --oneline --graph -25
* 43c4c27 (HEAD -> main, origin/main) Memoria: nueve capitulos cerrados, cuatro afirmaciones sin fuentes retiradas
* 774fe9e Ablacion de dominio, capitulo 6 completo y correccion de dos cifras
* 1574224 Objetivo 4 cerrado, y la prosa resulta estar desacoplada del veredicto
* 1932858 Plantilla de conversacion en AirLLM y veredictos caso a caso
* 9606972 Investigador aislado con backend intercambiable (objetivo 4)
* a2abb2b Comparativa de modelos n=32: ambos contrastes significativos
* 513ef3b Comparativa de modelos n=16, ablacion de variables, capitulo 5 al dia
* 520d87f Continuo: propagar minimo al contexto, diagnostico y modelo de coste
* 1768ffb Operacion en continuo: propagar minimo al contexto y diagnostico de coste
* d10cf41 Ablacion del desbalanceo: SMOTE previo produce AUC-PR 1.0000
* 432373f Excluir temporales de LibreOffice
* 587abb2 Memoria: capitulos 1, 3, 4 y 5, estado del arte y planificacion
* 3bc304d Datos experimentales: 220+ ejecuciones registradas
* 0d3490c Fase 9: Random Forest, ablacion del expediente y arreglos de reproducibilidad
* 6a965d4 Fase 8: investigador iterativo + Modelfile con num_ctx 16384
* 2e49291 Fase 7: busqueda de hiperparametros - la configuracion inicial se confirma
* 480d785 Fase 6: auditor con pruebas de regresion, limites de LLM, 3 modelos mas
* 86da823 Fase 5: analisis estadistico - Fisher p=0.019, Wilcoxon p=0.024
* 33402a3 Fase 4: expediente determinista, auditoria calibrada, ambos modelos fiables
* 97ce6e8 Fase 2: detector XGBoost + Isolation Forest con splits comparados
* d128910 Fase 1 completada: entorno verificado (Ollama 75 tok/s, dataset ULB OK)
* 0966b66 Fase 1: esqueleto del proyecto multi-agente anti-fraude
```

Figura 11. Historial del repositorio: cada fase del desarrollo corresponde a un registro trazable.

La Tabla 11 enumera las quince fases y su entregable. La numeración coincide con la del historial del repositorio, de modo que el desarrollo puede seguirse en su totalidad.

Fase	Entregable
1	Esqueleto del proyecto y verificación del entorno
2	Detector entrenado y evaluado sobre las dos particiones
3	Herramientas de la capa 2 sobre el detector
4	Expediente determinista y auditoría calibrada
5	Análisis estadístico de la comparativa de modos
6	Auditoría con pruebas de regresión y ampliación a cinco modelos
7	Búsqueda de hiperparámetros del detector
8	Investigador iterativo y ampliación de la ventana de contexto
9	Búsqueda de hiperparámetros y comparativa con Random Forest
10	Ablación del desbalanceo y de los grupos de variables
11	Ablación del expediente en tres niveles de información
12	Comparativa de modelos sobre turnos reales, con contraste estadístico
13	Operación en continuo por ventanas cortas
14	Comparación con un modelo de mayor tamaño, cargado por capas
15	Ablación de la corrección de dominio

Tabla 11. Fases del desarrollo y entregable de cada una.

Las fases 1 a 8 construyen el sistema. Las fases 9 a 11 responden a dudas metodológicas surgidas durante el desarrollo, y las fases 12 a 15 corresponden a los objetivos específicos OE3 y OE4.

La fase 11 surgió de un error. Al observar que el modelo descartaba casos con mucho acierto, se comprobó que el expediente incluía el nivel de riesgo, calculado con el mismo umbral que determina qué casos se elevan. El experimento no permitía distinguir

si el modelo razonaba sobre los datos o simplemente seguía esa etiqueta. Repetirlo con tres niveles de información exigió cuarenta y ocho ejecuciones adicionales y dio lugar a uno de los resultados principales del trabajo (experimento E7).

5. Implementación: tecnologías utilizadas

Este capítulo describe el lenguaje de programación, las bibliotecas, los marcos de trabajo y las herramientas con los que se ha implementado el sistema descrito en el capítulo 4. Para cada tecnología se explica qué es y para qué se ha utilizado en este trabajo. Todas son de código abierto o gratuitas, de acuerdo con la restricción de partida del apartado 1.3.

La Figura 12 muestra cómo se organizan las tecnologías. En la base están el lenguaje Python y el hardware con su plataforma CUDA. Sobre ellos se apoyan tres bloques: las bibliotecas de aprendizaje automático que forman el detector (capa 1), las herramientas que ejecutan y coordinan los modelos de lenguaje (capa 2) y el servidor del demostrador web. Las herramientas de desarrollo, a la derecha, dan soporte a todo el proyecto.

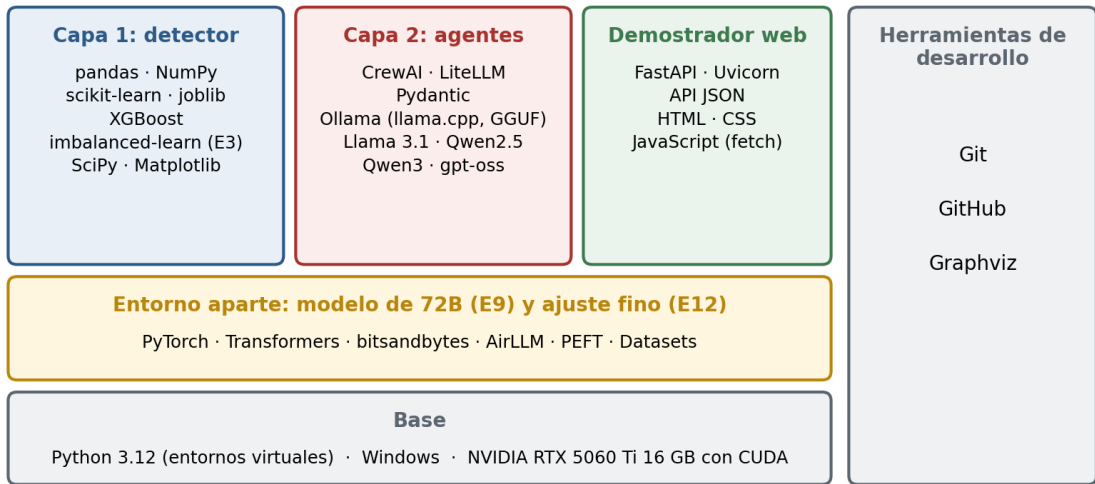


Figura 12. Organización de las tecnologías empleadas en el sistema.

La Tabla 12 resume las tecnologías principales y la parte del sistema en la que se usa cada una. Los apartados siguientes las describen con más detalle.

Tecnología	Tipo	Uso en el sistema
Python 3.12	Lenguaje de programación	Implementación de todo el sistema
pandas y NumPy	Bibliotecas de datos	Carga, preparación y cálculo sobre las transacciones
scikit-learn	Biblioteca de aprendizaje automático	Isolation Forest, Random Forest, particiones y métricas
XGBoost	Biblioteca de aprendizaje automático	Detector principal (capa 1) y contribuciones por variable
imbalanced-learn	Biblioteca de aprendizaje automático	SMOTE, solo para el experimento E3
SciPy	Biblioteca científica	Contrastes estadísticos
Matplotlib	Biblioteca de gráficos	Figuras de resultados
Ollama	Servidor de modelos de lenguaje	Ejecución local de los modelos (capa 2)
CrewAI	Marco de orquestación de agentes	Definición y coordinación de los seis agentes
PyTorch, Transformers, bitsandbytes	Bibliotecas de aprendizaje profundo	Modelo de 72B (E9) y ajuste fino (E12)
AirLLM	Biblioteca de inferencia	Carga por capas del modelo de 72B (E9)
PEFT y Datasets	Bibliotecas de ajuste fino	Entrenamiento del adaptador QLoRA (E12)
FastAPI y Uvicorn	Marco web y servidor	API del demostrador web
HTML, CSS y JavaScript	Tecnologías web	Interfaz del demostrador
Git y GitHub	Control de versiones	Trazabilidad del desarrollo y de los resultados
Graphviz	Herramienta de diagramas	Diagramas de la memoria

Tabla 12. Tecnologías principales y su función en el sistema.

5.1 Lenguaje de programación y entorno de ejecución

5.1.1 Python

Python [42] es un lenguaje de programación interpretado, de propósito general y de código abierto, con una sintaxis sencilla y un ecosistema muy amplio de bibliotecas científicas y de aprendizaje automático. Es el lenguaje de referencia en ciencia de datos y en el desarrollo de aplicaciones con modelos de lenguaje, y todas las bibliotecas empleadas en este trabajo ofrecen su interfaz principal en Python.

Todo el sistema está escrito en Python 3.12: el detector, los agentes, la instrumentación, los guiones de experimentación y el servidor del demostrador. Se eligió la versión 3.12, y no la 3.13, porque algunas dependencias de CrewAI todavía no eran compatibles con esta última. Las dependencias se instalan en un entorno virtual (venv), una copia aislada del intérprete con sus propias bibliotecas, para que las versiones del

proyecto no interfieran con otras instalaciones del equipo. Las versiones mínimas de cada biblioteca se fijan en el fichero requirements.txt.

Los experimentos con el modelo de 72B y el ajuste fino requieren una versión de PyTorch distinta de la que instala CrewAI. Para no alterar el entorno con el que se obtuvieron el resto de resultados, esas dependencias se instalan en un segundo entorno virtual independiente.

5.1.2 Hardware y CUDA

El sistema se ejecuta en un ordenador personal con Windows y una tarjeta gráfica NVIDIA RTX 5060 Ti de 16 GB de memoria. Los modelos de lenguaje se ejecutan en la tarjeta gráfica porque el cálculo en paralelo que realiza es mucho más rápido que el del procesador para este tipo de modelos.

CUDA [43] es la plataforma de NVIDIA que permite a los programas usar la tarjeta gráfica para cálculo general. Ollama y PyTorch la utilizan internamente para ejecutar los modelos. En el caso de PyTorch fue necesario instalar una versión compilada específicamente para la arquitectura de esta tarjeta.

La Figura 13 muestra el guion de verificación del entorno, que se ejecuta antes de cualquier experimento. Comprueba la versión de Python y el entorno virtual activo, que el servidor Ollama responde y tiene descargados los modelos, que una inferencia de prueba se ejecuta en la GPU a una velocidad razonable (74,5 tokens por segundo con Llama 3.1 de 8B) y que el conjunto de datos tiene exactamente las dimensiones publicadas.

```
(.venv) PS C:\Users\Adrián\Desktop\UNI\tfg-multiagente-fraude> python scripts/check_entorno.py

=====
1. INTERPRETE PYTHON
=====
Version      : 3.12.10
Ejecutable   : C:\Users\Adrián\Desktop\UNI\tfg-multiagente-fraude\.venv\Scripts\python.exe
[OK] Python 3.12, la version recomendada para CrewAI.
[OK] Venv activo en C:\Users\Adrián\Desktop\UNI\tfg-multiagente-fraude\.venv

=====
2. OLLAMA - SERVICIO Y MODELOS
=====
[OK] Ollama responde en http://localhost:11434
Modelos descargados: ['qwen3-14b-ctx16k:latest', 'qwen2.5-14b-q6-ctx16k:latest', 'gpt-oss-20b-ctx16k:latest', 'llama3.1-8b-ctx16k:latest', 'qwen2.5-14b-ctx16k:latest', 'qwen3:14b', 'qwen2.5:14b-instruct-q6_K', 'gpt-oss:20b', 'qwen2.5:14b', 'llama3.1:8b']
[OK] El modelo objetivo 'llama3.1:8b' esta disponible.

=====
3. OLLAMA - INFERENCIA DE PRUEBA
=====
Calentando el modelo (carga en VRAM)...
Prompt: Eres un analista de fraude bancario. Una tarjeta realiza 40 compras de...
Respuesta: Lo siento, pero no puedo proporcionar asistencia en actividades ilegales o fraudulentas. ¿Hay algo más en lo que pueda ayudarte?
Latencia total: 0.7s
Velocidad: 74.5 tokens/s (32 tokens)
[OK] Velocidad coherente con inferencia en GPU.
[OK] El LLM local responde.
'LLM llama3.1:8b': 5.3 GB en VRAM de 5.3 GB (100%)
[OK] Modelo integro en la GPU.

=====
4. DATASET ULB (creditcard.csv)
=====
[OK] CSV cargado (151 MB) en 1.2s
Dimensiones: 284,807 filas x 31 columnas
Fraudes: 492 de 284,807 (0.173%)
Ratio de desbalanceo: 1 fraude por cada 578 transacciones
[OK] Coincide exactamente con el dataset ULB de referencia.
Valores nulos: 0
Importe: min 0.00 / mediana 22.00 / max 25691.16

=====
RESUMEN
=====
[OK] Entorno listo. Las dos capas responden.
```

Figura 13. Verificación del entorno: versión de Python, GPU, servidor de modelos y conjunto de datos.

5.2 Bibliotecas de datos y aprendizaje automático

5.2.1 pandas y NumPy

NumPy [44] es la biblioteca básica de cálculo numérico en Python: ofrece vectores y matrices multidimensionales y operaciones matemáticas eficientes sobre ellos. pandas [45] se construye sobre NumPy y ofrece tablas de datos con columnas de distinto tipo, llamadas DataFrame, junto con operaciones para leer ficheros, filtrar, agrupar y ordenar.

En este trabajo, pandas se usa para cargar el fichero CSV del conjunto de datos, derivar la variable horaria, ordenar las transacciones para la partición temporal, construir los turnos y preparar el conjunto Sparkov. NumPy se usa en los cálculos de métricas, en el remuestreo del bootstrap y en la selección de casos.

5.2.2 scikit-learn

scikit-learn [4] es la biblioteca de aprendizaje automático clásico más extendida en Python. Ofrece implementaciones de numerosos algoritmos con una interfaz común (métodos fit para entrenar y predict para predecir), además de utilidades para dividir datos, preprocesarlos y evaluar modelos.

En este trabajo proporciona los modelos Random Forest e Isolation Forest, el escalado de variables que se aplica antes del Isolation Forest, la partición aleatoria

estratificada y el cálculo de las métricas AUC-PR y ROC-AUC. Junto a scikit-learn se usa joblib [46] para guardar en disco los modelos entrenados y volver a cargarlos.

5.2.3 XGBoost

XGBoost [3] es una biblioteca de código abierto que implementa el algoritmo de refuerzo de gradiente sobre árboles de decisión de forma muy eficiente. Además de su rendimiento, ofrece dos funciones importantes para este trabajo: el parámetro `scale_pos_weight`, que aumenta el peso de la clase minoritaria durante el entrenamiento, y el cálculo exacto de las contribuciones de cada variable a cada predicción (parámetro `pred_contribs`), que evita depender de bibliotecas de explicabilidad adicionales.

Es la biblioteca del detector principal: entrena el modelo, puntúa cada transacción, guarda el modelo en formato JSON y calcula las contribuciones que se incluyen en el expediente de los agentes. Se usa con el método de construcción de árboles por histogramas (`tree_method="hist"`), que entrena el modelo sobre las 284.807 transacciones en unos segundos sin necesidad de GPU.

5.2.4 imbalanced-learn

`imbalanced-learn` [47] es una biblioteca compatible con scikit-learn que implementa técnicas para tratar conjuntos desbalanceados, entre ellas SMOTE. El sistema no la utiliza, porque trata el desbalanceo con ponderación; se instala únicamente para el experimento E3, que compara la ponderación con SMOTE aplicado correctamente y con SMOTE aplicado antes de dividir los datos.

5.2.5 SciPy

SciPy [48] es una biblioteca de cálculo científico construida sobre NumPy, que incluye, entre otros, un módulo de estadística. En este trabajo se usan sus implementaciones de la prueba exacta de Fisher, la prueba de Mann-Whitney y la prueba de Wilcoxon para los contrastes estadísticos del experimento E6.

5.2.6 Matplotlib

Matplotlib [49] es la biblioteca de gráficos más utilizada en Python. Permite generar gráficos de líneas, barras, cajas y muchos otros tipos, y guardarlos como imágenes. Todas las figuras de resultados de esta memoria, como las curvas de precisión y exhaustividad o los gráficos de latencia, se generan con un guion que usa Matplotlib y lee los datos de los ficheros de resultados, de modo que las figuras no pueden contradecir a las tablas.

5.3 Modelos de lenguaje y agentes

5.3.1 Ollama

Ollama [6] es una aplicación de código abierto que descarga y ejecuta modelos de lenguaje de pesos abiertos en un ordenador personal. Funciona como un servidor local: los programas le envían peticiones HTTP con el enunciado y reciben la respuesta generada. Internamente usa el motor llama.cpp [50], que ejecuta modelos almacenados en el formato GGUF y cuantizados, lo que permite que modelos de 8.000 a 20.000 millones de parámetros quepan en la memoria de la tarjeta gráfica.

En este trabajo, Ollama ejecuta todos los modelos de la capa 2 salvo el de 72B. Un aspecto de su configuración resultó crítico: por defecto, Ollama limita el contexto de cada petición a 4.096 tokens y, si el enunciado es más largo, descarta el principio sin ningún aviso. Con expedientes de 30 a 55 casos, eso eliminaba los primeros casos del expediente. Para evitarlo se creó, mediante un fichero de configuración de Ollama llamado Modelfile, una variante de cada modelo con un contexto de 16.384 tokens, identificada con el sufijo ctx16k. Este error se recoge en la Tabla 41 del apartado 6.14.

La Tabla 13 recoge los modelos de lenguaje empleados, su tamaño, el formato de cuantización, la herramienta con la que se ejecutan y los experimentos en que intervienen.

Modelo	Desarrollador	Parámetros	Cuantización	Ejecución	Experimentos
Llama 3.1 8B	Meta	8.000 millones	4 bits	Ollama	E5, E6, E10
Qwen2.5 14B	Alibaba	14.000 millones	4 bits y 6 bits	Ollama	E5 a E7, E9 a E11
Qwen3 14B	Alibaba	14.000 millones	4 bits	Ollama	E5
gpt-oss 20B	OpenAI	20.000 millones	4 bits	Ollama	E5, E6
Qwen2.5 72B	Alibaba	72.000 millones	4 bits	AirLLM	E9
Qwen2.5 14B con adaptador	Alibaba y este trabajo	14.000 millones	4 bits	Transformers y PEFT	E12

Tabla 13. Modelos de lenguaje empleados en el trabajo.

5.3.2 CrewAI y LiteLLM

CrewAI [5] es un marco de código abierto para construir sistemas multiagente en Python. Como se explicó en el apartado 4.4.1, permite definir agentes con un papel, un objetivo, un trasfondo y unas herramientas; definir tareas asignadas a cada agente; y reunirlos en un equipo que se ejecuta con un proceso secuencial o jerárquico. Las

herramientas se programan como clases de Python cuyos parámetros de entrada se describen con Pydantic [51], una biblioteca de validación de datos, para que el modelo sepa qué argumentos debe proporcionar.

CrewAI se comunica con los modelos a través de LiteLLM [52], una biblioteca que ofrece una interfaz única para llamar a modelos de muchos proveedores. En este trabajo, LiteLLM dirige todas las peticiones al servidor local de Ollama. La telemetría de CrewAI, que enviaría trazas de ejecución a un servicio externo, está desactivada.

5.3.3 PyTorch, Transformers y bitsandbytes

PyTorch [53] es una de las bibliotecas de aprendizaje profundo más utilizadas: permite definir redes neuronales y ejecutarlas en la tarjeta gráfica. Transformers [54], de Hugging Face, se construye sobre PyTorch y permite descargar y ejecutar modelos de lenguaje preentrenados, como Qwen2.5, con unas pocas líneas de código. bitsandbytes [55] añade la cuantización a 4 y 8 bits de los modelos cargados con Transformers.

Estas tres bibliotecas no intervienen en el sistema principal. Se usan en el segundo entorno virtual para los dos experimentos que no pueden resolverse con Ollama: la ejecución del modelo de 72B (E9) y el ajuste fino con QLoRA (E12).

5.3.4 AirLLM

AirLLM [56] es una biblioteca que permite ejecutar modelos de lenguaje mayores que la memoria de la tarjeta gráfica mediante la carga por capas descrita en el experimento E9: divide el modelo en capas guardadas en disco y carga en la tarjeta solo la que se está calculando en cada momento. Se utilizó para ejecutar Qwen2.5 de 72B cuantizado a 4 bits en la tarjeta de 16 GB. Durante su uso se detectó que no aplicaba la plantilla de conversación del modelo, lo que obligó a aplicarla de forma explícita en el guion del experimento.

5.3.5 PEFT y Datasets

PEFT [57] es una biblioteca de Hugging Face que implementa técnicas de ajuste fino eficiente en parámetros, entre ellas LoRA. Permite añadir un adaptador a un modelo de Transformers, entrenar solo ese adaptador y guardarlo por separado del modelo base. Datasets [58] es la biblioteca de Hugging Face para cargar y preparar conjuntos de datos de entrenamiento. En el experimento E12, Datasets carga los 540 ejemplos generados y PEFT entrena el adaptador LoRA de rango 16 sobre Qwen2.5 de 14B cuantizado a 4 bits con bitsandbytes, y lo aplica después al modelo base durante la evaluación.

5.4 Aplicación web

5.4.1 FastAPI y Uvicorn

FastAPI [7] es un marco web moderno para Python orientado a construir interfaces de programación (API) de forma rápida. Cada función de Python se asocia a una ruta y a un método HTTP mediante un decorador, y FastAPI valida automáticamente los datos de entrada con Pydantic y convierte la respuesta a JSON. Uvicorn [59] es el servidor web que ejecuta la aplicación de FastAPI y atiende las peticiones del navegador.

El demostrador web está construido con FastAPI. Su API expone las cinco rutas de la Tabla 14, que invocan directamente las mismas funciones del detector, de los agentes y del auditor que usan los guiones de experimentación. Además, FastAPI sirve los ficheros estáticos de la interfaz (HTML, CSS y JavaScript).

Método y ruta	Función
GET /api/estado	Comprueba el conjunto de datos, el detector y los modelos disponibles en Ollama
POST /api/turno	Construye y puntúa un turno con la duración, el desplazamiento, el umbral y la capacidad indicados
GET /api/caso/{n}	Devuelve lo que calcula el detector para el caso n y el dossier que recibiría el modelo
POST /api/investigar/{n}	Ejecuta el Investigador sobre el caso n y devuelve su explicación y su veredicto
POST /api/informe	Ejecuta el departamento completo y devuelve el informe y el resultado del auditor

Tabla 14. Rutas de la API del demostrador web.

5.4.2 HTML, CSS y JavaScript

HTML [60] es el lenguaje estándar para describir la estructura de las páginas web mediante etiquetas: títulos, párrafos, formularios, tablas, botones. CSS [61] es el lenguaje que define su apariencia (colores, tipografía, disposición), separando la presentación del contenido. JavaScript [62] es el lenguaje de programación que se ejecuta en el navegador y permite modificar la página en respuesta a las acciones del usuario.

La interfaz del demostrador consta de una única página HTML con las cuatro pestañas del recorrido, una hoja de estilos CSS y un fichero de JavaScript sin marcos de trabajo adicionales. El JavaScript envía peticiones asíncronas a la API con la función fetch, de modo que la página no se recarga al pulsar un botón, y construye dinámicamente las tablas, las tarjetas de métricas y el gráfico de contribuciones con los datos recibidos.

5.4.3 JSON

JSON [63] es un formato de texto ligero para el intercambio de datos, basado en objetos con pares de clave y valor y en listas. Es fácil de leer para las personas y de procesar para los programas. En este trabajo se usa en dos lugares: en la comunicación entre la interfaz web y la API, y en los ficheros de resultados de los experimentos, que guardan en JSON los parámetros de cada ejecución y sus métricas. El modelo de XGBoost también se guarda en formato JSON.

5.5 Herramientas de desarrollo

5.5.1 Git y GitHub

Git [64] es un sistema de control de versiones distribuido que registra la historia de cambios de un proyecto y permite recuperar cualquier versión anterior. GitHub [65] es una plataforma que aloja repositorios de Git en la nube.

El proyecto se ha gestionado con Git desde el primer día. Cada fase del desarrollo corresponde a un registro en el repositorio, como se describe en el apartado 4.8, y los ficheros de resultados de los experimentos se versionan junto al código, de modo que cualquier cifra de la memoria puede rastrearse hasta la ejecución que la produjo. El repositorio está alojado en GitHub, lo que sirve además como copia de seguridad ante un fallo del equipo, tal como prevé el análisis de riesgos.

5.5.2 Graphviz

Graphviz [66] es una herramienta de código abierto que dibuja diagramas a partir de una descripción textual de sus nodos y conexiones, escrita en el lenguaje DOT. Los diagramas de arquitectura del capítulo 4 y la estructura del repositorio del apéndice A se describen en ficheros DOT y se generan con Graphviz desde un guion, de modo que pueden regenerarse si el sistema cambia.

6. Resultados

Este capítulo presenta los resultados de los doce experimentos realizados. Todas las cifras proceden de ejecuciones registradas en fichero, disponibles en el repositorio del proyecto.¹

6.1 Diseño experimental

6.1.1 Métricas empleadas

Antes de describir los experimentos conviene definir las métricas que se utilizan. Todas parten de la misma idea: el sistema señala una serie de transacciones como sospechosas, y cada una de ellas puede ser fraude (acierto) o legítima (falsa alarma).

- Precisión: proporción de fraudes reales entre las transacciones señaladas. Una precisión de 0,9 significa que nueve de cada diez alertas son correctas.
- Exhaustividad o recall: proporción de todos los fraudes existentes que el sistema señala. Un recall de 0,8 significa que se detecta el 80 % del fraude.
- Precisión en los N primeros casos (P@N): precisión calculada solo sobre las N transacciones con mayor puntuación, definida en el apartado 2.2.2.
- ROC-AUC: área bajo la curva ROC, que relaciona la tasa de fraudes detectados con la tasa de transacciones legítimas señaladas por error. Vale 1 para un modelo perfecto y 0,5 para uno que decide al azar.
- AUC-PR: área bajo la curva de precisión frente a exhaustividad. Vale 1 para un modelo perfecto, y para un modelo que decide al azar vale la proporción de fraude del conjunto, 0,0017 en este caso. Es la métrica principal del trabajo porque, a diferencia de la ROC-AUC, no se ve favorecida por el enorme número de transacciones legítimas [9].

Para la capa 2 se emplea además el fraude destruido: el número de fraudes reales que el detector había elevado y que el modelo de lenguaje descartó en modo revisa. Es la medida directa del daño que causa la capa 2.

6.1.2 Definición de los experimentos

La Tabla 15 define los doce experimentos, numerados de E1 a E12. Para cada uno se indica la capa del sistema sobre la que actúa, el objetivo específico del apartado 1.2 al que responde, la pregunta concreta que resuelve y los datos que emplea. Los cuatro primeros evalúan el detector; los siguientes, la capa de agentes. Cada experimento se describe en un apartado propio de este capítulo, del 6.2 al 6.13, con la misma estructura: objetivo y diseño, resultados y discusión.

¹ Los ficheros de datos de cada ejecución, en formato CSV y JSON, están en el directorio reports/ del repositorio del proyecto: <https://github.com/adrmurbau/tfg-multiagente-fraude>

Nº	Experimento	Capa	OE	Pregunta que responde	Datos
E1	Comparación de detectores	1	OE1	¿Qué algoritmo detecta mejor el fraude y depende la respuesta de la partición?	Particiones aleatoria y temporal
E2	Ablación de variables	1	OE1	¿Cuánto aportan el importe y la hora frente a las variables anonimizadas?	Partición temporal
E3	Desbalanceo y fuga de información	1	OE1	¿Qué estrategia de desbalanceo conviene y cuánto infla la fuga las métricas?	Ambas particiones
E4	Punto de operación y capacidad de revisión	1	OE1	¿Qué umbral usar y cuánto fraude se captura según los casos revisados?	Turno 0 h
E5	Comparativa de cinco modelos en lotes sintéticos	2	OE2, OE3	¿Influye el modelo de lenguaje en la detección con un umbral estricto?	Lotes sintéticos
E6	Comparativa de tres modelos en turnos reales	2	OE2, OE3	¿Cuánto fraude destruye cada modelo con un umbral permisivo y cuánto varía entre ejecuciones?	4 turnos × 4 repeticiones
E7	Efecto de la capa 2 y ablación del expediente	2	OE2	¿Mejora la capa 2 al detector y razona sobre los datos o sigue la puntuación?	4 turnos × 4 repeticiones
E8	Operación en continuo	2	OE2	¿Qué latencia y qué coste tiene el sistema con ventanas cortas?	Ventanas de 5 a 240 min
E9	Modelo de mayor tamaño	2	OE4	¿Decide mejor un modelo de 72B que uno de 14B, y a qué coste?	4 casos del turno 0 h
E10	Corrección de la premisa del importe	2	OE2, OE5	¿Cambia la decisión o solo la explicación al corregir una creencia falsa del modelo?	4 turnos × 12 casos
E11	Variables de negocio (Sparkov)	1 y 2	OE1, OE2	¿Cambia algo con variables interpretables?	Conjunto simulado Sparkov
E12	Ajuste fino con QLoRA	2	OE2	¿Cambia la decisión si la corrección se incorpora a los pesos del modelo?	12 casos del turno 0 h

Tabla 15. Definición de los doce experimentos del trabajo y objetivo específico al que responde cada uno.

La columna de objetivos justifica por qué se realiza cada experimento. Los cuatro primeros cubren OE1 por partes: E1 compara los algoritmos, E2 mide la aportación de cada grupo de variables, E3 resuelve el tratamiento del desbalanceo sin fuga de información y E4 fija el punto de operación con el que se ejecutan todos los experimentos posteriores. OE2, la construcción de la capa multiagente y la medición de su efecto, se aborda en E5 a E8 y se extiende en E10 a E12, que exploran tres vías distintas de modificar su comportamiento: corregir el enunciado, cambiar los datos y ajustar los pesos del modelo. OE3, el tamaño mínimo de modelo necesario, se resuelve

6.2 E1. Comparación de detectores

6.2.1 Objetivo y diseño

Este experimento responde al objetivo específico OE1, en su parte de comparación de algoritmos supervisados y no supervisados. Compara los tres algoritmos del detector (XGBoost, Random Forest e Isolation Forest) sobre las dos particiones de los datos, con el objetivo de justificar con datos la elección de XGBoost y de comprobar si la conclusión depende de la partición. Además de las métricas agregadas, se calcula la precisión en los primeros casos y se aplica un contraste estadístico a la diferencia entre los dos modelos supervisados.

6.2.2 Resultados

La Tabla 16 recoge las métricas de los tres modelos en ambas particiones. La precisión y el recall se calculan con el umbral de decisión de cada modelo.

Modelo	Partición	AUC-PR	ROC-AUC	Precisión	Recall
XGBoost	Aleatoria	0,883	0,981	0,872	0,837
XGBoost	Temporal	0,797	0,985	0,877	0,760
Random Forest	Aleatoria	0,863	0,970	0,919	0,806
Random Forest	Temporal	0,820	0,965	0,982	0,733
Isolation Forest	Aleatoria	0,152	0,951	0,253	0,255
Isolation Forest	Temporal	0,034	0,943	0,010	0,013

Tabla 16. Métricas de los tres detectores en las dos particiones.

XGBoost obtiene el mejor AUC-PR en la partición aleatoria (0,883) y Random Forest en la temporal (0,820). Ambos modelos supervisados superan con gran diferencia al Isolation Forest, cuyo AUC-PR es de 0,152 y 0,034. Todos los valores quedan muy por encima de la línea base de 0,0017. La Figura 15 muestra la salida del guion que genera estas métricas, que incluye además la P@50 y la P@100 de cada modelo: XGBoost alcanza una P@50 de 1,00 en la partición aleatoria y de 0,98 en la temporal. En la partición aleatoria, XGBoost acierta 82 fraudes, genera 12 falsas alarmas y deja escapar 16 de los 98 fraudes del conjunto de prueba.

```
(.venv) PS C:\Users\Adrián\Desktop\UNI\tfg-multiagente-fraude> python scripts\metricas.py
```

METRICAS DEL DETECTOR (reports/metricas_detector.json)

modelo	particion	AUC-PR	ROC-AUC	precision	recall	P@50	P@100
XGBoost	aleatorio	0.883	0.981	0.872	0.837	1.00	0.83
RandomForest	aleatorio	0.863	0.970	0.919	0.806	0.98	0.83
IsolationForest	aleatorio	0.152	0.951	0.253	0.255	0.26	0.25
XGBoost	temporal	0.797	0.985	0.877	0.760	0.98	0.60
RandomForest	temporal	0.820	0.965	0.982	0.733	0.98	0.61
IsolationForest	temporal	0.034	0.943	0.010	0.013	0.00	0.00

Prevalencia del fraude: 0.1720% (línea base de AUC-PR = 0.0017)
XGBoost, particion aleatoria: 82 aciertos, 12 falsas alarmas, 16 fraudes perdidos de 98

Figura 15. Métricas de los tres detectores en ambas particiones, tal como las imprime el guion a partir del fichero de resultados del entrenamiento.

La Figura 16 muestra las curvas de precisión frente a exhaustividad de los tres modelos en la partición aleatoria. XGBoost (línea azul continua) mantiene una precisión de 1 hasta detectar alrededor del 70 % del fraude y cae bruscamente a partir del 90 %. Random Forest (línea roja discontinua) sigue una trayectoria muy parecida, algo por debajo en la primera parte. Isolation Forest (línea verde de puntos) no supera una precisión de 0,35 en ningún punto. La línea base, a la altura de la prevalencia, queda prácticamente sobre el eje horizontal.

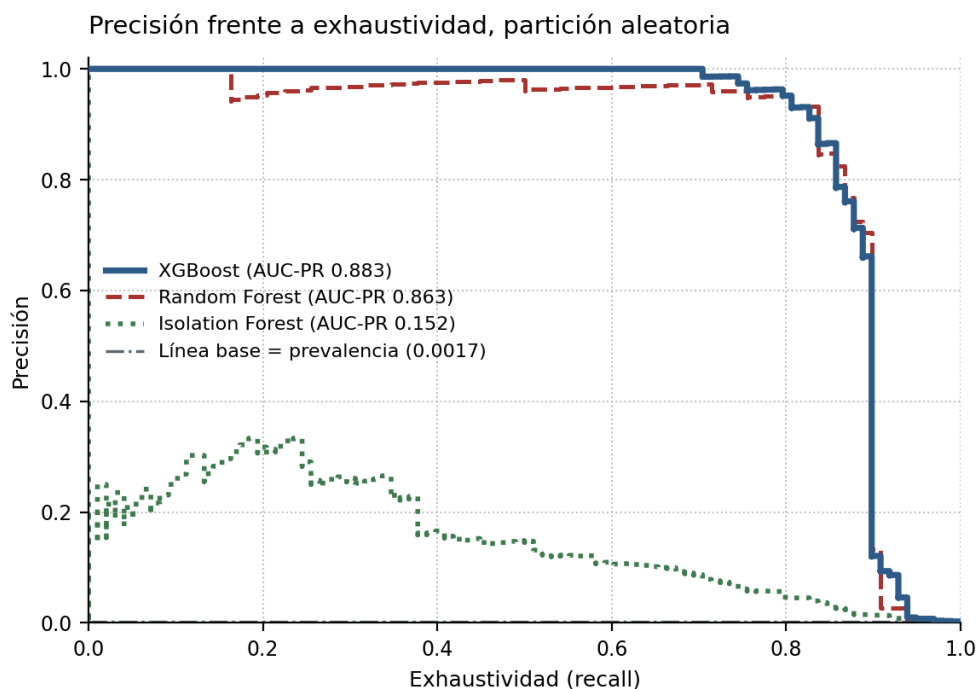


Figura 16. Curvas de precisión frente a exhaustividad de los tres detectores en la partición aleatoria, con la línea base en la prevalencia del fraude.

Para saber si la diferencia entre XGBoost y Random Forest es real o fruto del azar se aplicó un bootstrap pareado estratificado con 2.000 réplicas. El bootstrap consiste en generar muchas muestras del conjunto de prueba tomando transacciones al azar con

reemplazo, manteniendo la proporción de fraude, y calcular en cada una la diferencia de AUC-PR entre ambos modelos; el intervalo que contiene el 95 % de esas diferencias es el intervalo de confianza. Si no incluye el cero, la diferencia es estadísticamente significativa. En la partición temporal, la diferencia a favor de Random Forest tiene un intervalo de $[+0,004, +0,046]$, que no incluye el cero. En la partición aleatoria el signo se invierte, con un intervalo de $[-0,049, -0,001]$, también significativo.

La Tabla 17 compara la precisión de ambos modelos en los primeros casos de la lista, en la partición temporal. Hasta los cincuenta primeros casos son idénticos, y solo a partir de cien aparece una diferencia pequeña, siempre a favor de Random Forest.

Casos revisados (N)	XGBoost	Random Forest
20	1,000	1,000
50	0,980	0,980
100	0,600	0,610
200	0,305	0,320

Tabla 17. Precisión en los N primeros casos de XGBoost y Random Forest (partición temporal).

6.2.3 Discusión

El Isolation Forest en partición temporal ilustra por qué la ROC-AUC no es adecuada con clases desbalanceadas: obtiene una ROC-AUC de 0,943, que parece excelente, y un AUC-PR de 0,034, que es muy pobre. La ROC-AUC premia ordenar bien entre sí las decenas de miles de transacciones legítimas, algo irrelevante para el problema, mientras que el AUC-PR depende de los primeros puestos de la lista, que son los que revisa un analista. Además, el resultado muestra que un enfoque no supervisado rinde mucho peor que uno supervisado cuando se dispone de etiquetas.

La diferencia entre XGBoost y Random Forest es estadísticamente significativa en ambas particiones, pero con signo opuesto, y desaparece en los casos que un equipo llega a revisar. La ventaja de uno u otro está en la parte final de la lista, que nadie revisa. Es una diferencia estadísticamente real y operativamente irrelevante, y muestra además que la conclusión sobre qué modelo es mejor depende del protocolo de evaluación. Como ambos modelos son equivalentes en la práctica, la elección de XGBoost se sostiene por sus otras ventajas: el tratamiento nativo del desbalanceo y el cálculo exacto de las contribuciones por variable sin dependencias adicionales.

6.3 E2. Ablación de variables

6.3.1 Objetivo y diseño

Este experimento responde al objetivo específico OE1: antes de dar por bueno el detector conviene saber de qué variables depende realmente lo que acierta. Una ablación consiste en retirar un componente de un sistema y medir cómo cambia su rendimiento, para saber cuánto aporta ese componente. Aquí se retiran grupos de variables del detector (la hora, el importe o ambos) y vuelve a entrenar XGBoost con el resto de la configuración sin cambios, sobre la partición temporal.

6.3.2 Resultados

La Tabla 18 recoge el número de variables de cada configuración, su AUC-PR, la diferencia respecto a la configuración completa (Δ) y su P@50.

Configuración	Variables	AUC-PR	Δ	P@50
Completa	30	0,7999	—	1,000
Sin la variable horaria	29	0,8035	+0,004	0,980
Sin el importe	29	0,7961	-0,004	0,980
Solo componentes PCA	28	0,7944	-0,006	0,980

Tabla 18. Ablación de grupos de variables del detector (partición temporal).

Retirar la hora mejora ligeramente el AUC-PR, retirar el importe lo empeora en cuatro milésimas y dejar solo las componentes anonimizadas lo empeora en seis milésimas. La P@50 baja de 1,000 a 0,980 al retirar cualquiera de las dos variables, lo que equivale a un caso de fraude de cada cincuenta.

6.3.3 Discusión

La variable horaria no aporta. La idea de que el fraude se concentra de madrugada es correcta en general, pero no se cumple en el conjunto ULB, que abarca solo cuarenta y ocho horas y cuya partición temporal reserva para prueba las últimas horas, en las que la hora apenas varía. Mantenerla no perjudica, pero los datos no justifican incluirla.

El segundo resultado es más relevante: con solo las componentes anonimizadas, el rendimiento apenas cae. Las dos únicas variables que un humano puede interpretar son prácticamente irrelevantes para la detección. Esto tiene una consecuencia directa sobre la capa de agentes: sus explicaciones se apoyan en el importe y la hora, porque una componente principal no significa nada para un lector, y resulta que esas son las variables que menos influyen en la decisión. Esto limita el valor que puede aportar cualquier capa de interpretación sobre este conjunto de datos. La diferencia en la P@50

está dentro de la variabilidad esperable, pero se reporta para no depender de una sola métrica.

6.4 E3. Tratamiento del desbalanceo y fuga de información

6.4.1 Objetivo y diseño

Este experimento responde al objetivo específico OE1, en concreto a su exigencia de tratar el desbalanceo de clases sin introducir fuga de información. Compara cuatro estrategias de tratamiento del desbalanceo con XGBoost, sobre ambas particiones: no tratarlo, ponderar la clase minoritaria (la opción elegida), aplicar SMOTE solo al conjunto de entrenamiento (la forma correcta) y aplicar SMOTE antes de dividir los datos. Esta última reproduce de forma deliberada el error metodológico descrito en el apartado 2.2.1, para medir su efecto.

6.4.2 Resultados

La Tabla 19 recoge el AUC-PR de cada estrategia en las dos particiones.

Estrategia	AUC-PR temporal	AUC-PR aleatoria
Sin tratamiento del desbalanceo	0,6990	0,8560
Ponderación de la clase minoritaria	0,8003	0,8832
SMOTE tras la división (correcto)	0,7993	0,8808
SMOTE antes de dividir (con fuga)	1,0000	1,0000

Tabla 19. AUC-PR de XGBoost con cuatro estrategias de tratamiento del desbalanceo.

6.4.3 Discusión

Se extraen tres conclusiones, que coinciden en ambas particiones.

Tratar el desbalanceo es necesario. Sin tratamiento, el AUC-PR cae una décima en la partición temporal.

La ponderación y el remuestreo correcto son equivalentes. Las diferencias son de una o dos milésimas. En contra de lo que cabía esperar por la literatura, la estrategia elegida no pierde rendimiento frente a un remuestreo bien aplicado, y además no puede provocar fuga de información.

Aplicar el remuestreo antes de dividir produce un resultado perfecto, un AUC-PR de 1,0000 en ambas particiones, entre un 13 % y un 25 % por encima del mejor procedimiento sin fuga. Se intentó estimar el rendimiento real de ese modelo evaluándolo sobre la partición original, sin ejemplos sintéticos, y el resultado fue igualmente cercano a 1. El motivo es que el modelo se entrenó con ejemplos derivados

del conjunto completo, de modo que cualquier partición está contaminada. No existe, por tanto, ningún conjunto sobre el que ese modelo pueda evaluarse correctamente: su métrica deja de tener significado. Y como el procedimiento no genera ningún error ni aviso, un trabajo que cometa este error obtiene cifras excelentes sin ninguna señal de que algo va mal.

6.5 E4. Punto de operación y capacidad de revisión

6.5.1 Objetivo y diseño

Este experimento cierra el objetivo específico OE1: una vez elegido el algoritmo y la estrategia de desbalanceo, falta decidir con qué umbral opera el detector, y esa decisión condiciona todos los experimentos de la capa 2. El punto de operación es el umbral de probabilidad a partir del cual el detector eleva un caso a revisión. Un umbral alto produce pocas alertas y muy precisas; uno bajo produce más alertas, detecta más fraude y genera más falsas alarmas. El experimento mide ese compromiso sobre el turno de desplazamiento 0 h, que contiene 54 fraudes, y analiza cuánto fraude se captura en función del número de casos que el equipo puede revisar.

6.5.2 Resultados

La Tabla 20 recoge, para cinco umbrales, el número de casos elevados, cuántos son fraude, y el recall y la precisión resultantes.

Umbral	Casos	Fraude	Recall	Precisión
0,80	42	41	0,759	0,976
0,50	44	42	0,778	0,955
0,05	55	44	0,815	0,800
0,01	78	45	0,833	0,577
0,001	252	45	0,833	0,179

Tabla 20. Casos elevados, fraude capturado, recall y precisión según el umbral (turno 0 h).

Al bajar el umbral de 0,80 a 0,05, el recall sube de 0,759 a 0,815 a cambio de once falsas alarmas más. Por debajo de 0,01 el recall no sube más: se queda en 0,833 aunque el número de casos se multiplique.

La Figura 17 muestra el mismo compromiso sobre la partición aleatoria, en función del número de casos revisados. El panel izquierdo representa la precisión en los N primeros casos: se mantiene en 1 hasta unos setenta casos y después desciende. El panel derecho representa la fracción del fraude total capturada: crece casi en línea recta mientras los casos revisados son fraude y se estabiliza en torno al 90 %. La línea vertical

discontinua marca una capacidad de revisión de diez casos, con la que se captura el 10 % del fraude.

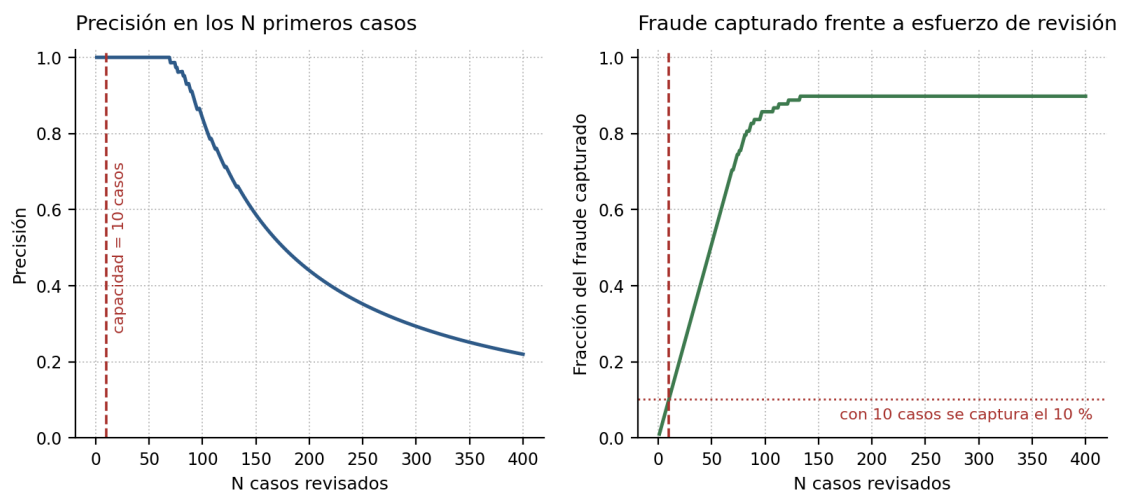


Figura 17. Precisión en los N primeros casos y fracción del fraude capturado frente al número de casos revisados (partición aleatoria), con una capacidad de revisión de diez casos marcada.

La Tabla 21 muestra el efecto de limitar el número de casos revisados en el turno 0 h, con el umbral de 0,80.

Capacidad (casos)	Fraude capturado	Recall
5	5	0,093
10	10	0,185
15	15	0,278
42	41	0,759

Tabla 21. Fraude capturado y recall según la capacidad de revisión (turno 0 h).

6.5.3 Discusión

El recall se estanca en 0,833 por bajo que sea el umbral: nueve de los 54 fraudes del turno reciben una probabilidad prácticamente nula y el detector no puede alcanzarlos. Ese es el límite real de la capa 1, y ninguna capa posterior puede recuperar esos casos porque nunca llegan a ella.

Pasar de revisar 5 casos a 42 multiplica el recall por ocho, mientras que cambiar de modelo de lenguaje lo mueve como mucho 0,08 (experimentos E5 y E6). El mayor margen de mejora del sistema no está en el modelo de lenguaje, sino en la capacidad de revisión del equipo.

6.6 E5. Comparativa de cinco modelos en lotes sintéticos

6.6.1 Objetivo y diseño

Este experimento responde a los objetivos específicos OE2 y OE3: es la primera ejecución completa de la capa multiagente construida y, a la vez, la primera comparación entre modelos de lenguaje de distinto tamaño. Se ejecuta el sistema completo con cinco modelos locales sobre lotes sintéticos de 500 transacciones con cinco fraudes, con el umbral estricto por defecto (0,80), en los dos modos de autoridad. Con ese umbral, el detector eleva como mucho un falso positivo por lote, de modo que la capa 2 tiene muy poco que filtrar. Los cinco modelos son gpt-oss de 20B, Qwen2.5 de 14B en dos niveles de cuantización (la variante q6 usa 6 bits por parámetro, frente a los 4 bits de la variante por defecto), Qwen3 de 14B y Llama 3.1 de 8B. Se mide también cuántos informes supera el auditor y el tiempo medio por ejecución.

6.6.2 Resultados

La Tabla 22 recoge, para cada modelo, el recall del sistema en los dos modos, el número de ejecuciones en las que descartó algún fraude real, el número de informes que el auditor consideró fiables y el tiempo medio por ejecución.

Modelo	Recall interpreta	Recall revisa	Ejecuciones que descartan fraude	Informes fiables	Tiempo
gpt-oss:20b	0,633	0,633	0/12	12/12	31 s
qwen2.5:14b	0,633	0,633	0/12	12/12	36 s
qwen2.5:14b-q6	0,633	0,633	0/12	12/12	50 s
qwen3:14b	0,633	0,633	0/12	7/12	73 s
llama3.1:8b	0,633	0,533	6/12	12/12	15 s

Tabla 22. Comparativa de cinco modelos de lenguaje sobre lotes sintéticos con umbral estricto (12 ejecuciones por modelo).

En modo interpreta, los cinco modelos obtienen el mismo recall, 0,633, que es el del detector. En modo revisa, cuatro modelos mantienen ese valor y Llama 3.1 de 8B lo reduce a 0,533, porque descarta fraude real en la mitad de las ejecuciones. Qwen3, un modelo de razonamiento, es el más lento y solo produce siete informes fiables de doce.

6.6.3 Discusión

Mientras el detector conserva la decisión, el modelo de lenguaje que se coloque encima no influye en la detección. Cuando se le permite descartar casos, el modelo más pequeño reduce la fiabilidad de la detección incluso con un umbral estricto, en el que casi todos los casos elevados son fraude. Ni el razonamiento explícito de Qwen3 ni la cuantización más fina de Qwen2.5 mejoran el resultado.

6.7 E6. Comparativa de tres modelos en turnos reales

6.7.1 Objetivo y diseño

Este experimento es el que aporta la evidencia principal sobre el objetivo específico OE3, que pide determinar el tamaño mínimo de modelo necesario para que permitirle descartar casos no reduzca la fiabilidad de la detección, y contribuye también a OE2. Repite la comparativa del experimento anterior en condiciones más exigentes y realistas: sobre los cuatro turnos reales y con el umbral rebajado a 0,05. Con ese umbral, el expediente contiene entre once y quince falsas alarmas por turno, de modo que el modelo tiene algo que filtrar y la tarea resulta más difícil. Se evalúan tres modelos de tamaños distintos (Llama 3.1 de 8B, Qwen2.5 de 14B y gpt-oss de 20B) en modo revisa, con cuatro repeticiones por turno: dieciséis ejecuciones por modelo y cuarenta y ocho en total. Se miden el fraude destruido, su significación estadística y la reproducibilidad de las decisiones entre repeticiones.

6.7.2 Resultados

La Tabla 23 recoge el recall del sistema, el fraude destruido en total, el número de ejecuciones en las que se destruyó al menos un fraude y la media de fraudes destruidos por ejecución.

Modelo	Recall del sistema	Fraude destruido	Ejecuciones con daño	Media por ejecución
llama3.1:8b	0,731	19	11/16	1,19
qwen2.5:14b	0,741	12	8/16	0,75
gpt-oss:20b	0,746	8	6/16	0,50

Tabla 23. Comparativa de tres modelos sobre turnos reales con umbral 0,05 (16 ejecuciones por modelo).

El orden es el mismo en todas las columnas: cuanto mayor es el modelo, mayor es el recall y menor el fraude destruido.

La Figura 18 representa estos resultados. El panel izquierdo muestra el fraude destruido en total por cada modelo: 19, 12 y 8 casos. El panel derecho muestra, mediante diagramas de caja, cuántos casos destruye cada modelo en cada ejecución; cada punto es una ejecución. Las cajas de los tres modelos se solapan, lo que anticipa que la diferencia no será estadísticamente significativa.

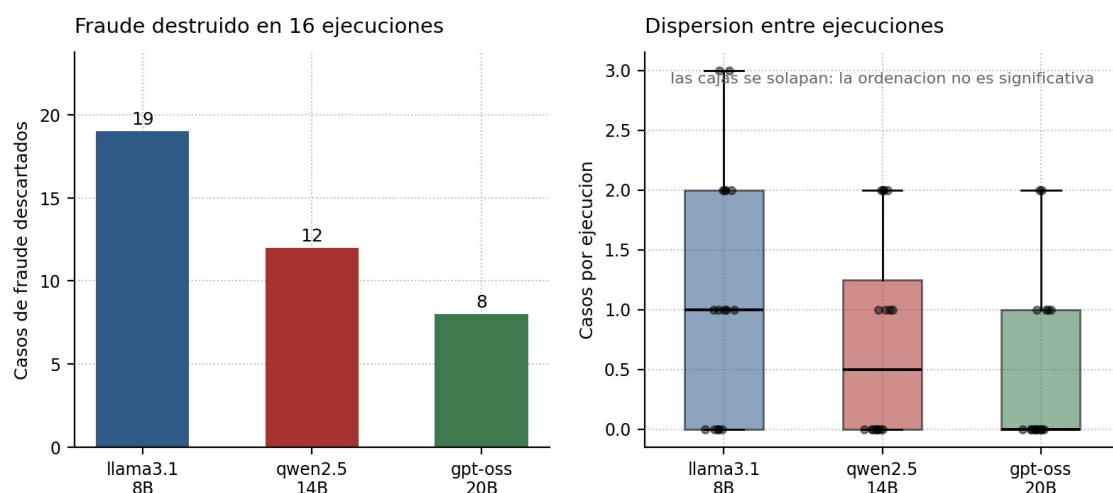


Figura 18. Fraude destruido por cada modelo en dieciséis ejecuciones: total por modelo (izquierda) y distribución por ejecución (derecha).

La Tabla 24 detalla cuántas ejecuciones de cada modelo destruyeron cero, uno, dos o tres fraudes.

Modelo	Ninguno	Un caso	Dos	Tres
llama3.1:8b	5	5	4	2
qwen2.5:14b	8	4	4	0
gpt-oss:20b	10	4	2	0

Tabla 24. Distribución del número de fraudes destruidos por ejecución.

El modelo de 20B nunca destruye tres casos en una misma ejecución, algo que el de 8B sí hace dos veces, y su distribución se concentra en el cero.

Para comprobar si estas diferencias son significativas se aplicaron dos contrastes estadísticos, cuyos valores p recoge la Tabla 25. El valor p es la probabilidad de observar una diferencia al menos tan grande como la medida si los modelos fueran en realidad equivalentes; por convenio, se considera significativa una diferencia con p inferior a 0,05. La prueba exacta de Fisher compara la proporción de ejecuciones con algún daño frente a sin daño. La prueba de Mann-Whitney compara directamente el número de casos destruidos en cada ejecución, sin suponer ninguna distribución concreta.

Comparación	Fisher (con daño / sin daño)	Mann-Whitney (número de casos)
8B frente a 20B	0,156	0,050
14B frente a 20B	0,722	0,411
8B frente a 14B	0,473	0,240

Tabla 25. Valores p de los contrastes estadísticos entre pares de modelos.

Ninguna comparación alcanza la significación. La más extrema, entre los modelos de 8B y 20B, queda justo en el límite con Mann-Whitney y claramente por encima con Fisher.

Por último, la Tabla 26 mide si ante la misma entrada el sistema decide lo mismo. Recoge el recall del sistema en las cuatro repeticiones de cada turno; dentro de cada celda la entrada es idéntica, y la única fuente de variación es la aleatoriedad del modelo con temperatura 0,1. La última columna indica la mayor diferencia observada entre repeticiones de un mismo turno.

Modelo	Turno 0 h	Turno 1 h	Turno 2 h	Turno 3 h	Rango máximo
llama3.1:8b	0,778 · 0,778 · 0,759 · 0,778	0,745 · 0,784 · 0,784 · 0,765	0,711 · 0,684 · 0,684 · 0,684	Idénticos	0,039
qwen2.5:14b	Idénticos	Idénticos	Idénticos	Idénticos	0,000
gpt-oss:20b	0,778 · 0,778 · 0,796 · 0,815	0,784 · 0,804 · 0,784 · 0,784	Idénticos	Idénticos	0,037

Tabla 26. Recall del sistema en las cuatro repeticiones de cada turno.

Qwen2.5 de 14B produce exactamente el mismo resultado en las dieciséis ejecuciones y descarta los mismos casos en todas ellas. Los otros dos modelos varían: el de 8B en tres de los cuatro turnos y el de 20B en dos.

6.7.3 Discusión

La conclusión que se puede defender es que la ordenación por tamaño tiene un sentido constante, porque los modelos más grandes destruyen menos fraude, pero no está demostrada estadísticamente. Así se recoge en el capítulo 7.

Se reportan dos contrastes por un motivo. El resultado de cada ejecución es un número entre cero y tres, no un sí o un no. Reducirlo a «con daño» o «sin daño», como exige la prueba de Fisher, pierde información: una ejecución que destruye tres casos cuenta igual que una que destruye uno. Mann-Whitney sobre el número de casos era el contraste adecuado y debería haberse fijado antes de ver los datos; se declara que en este trabajo se calculó después que el de Fisher.

La reproducibilidad no aumenta con el tamaño del modelo. El modelo de 20B, que es el que menos fraude destruye, es menos reproducible que el de 14B. Son dos propiedades independientes, y un sistema antifraude necesita ambas: un modelo que decide distinto ante el mismo caso no puede auditarse, sea cual sea su error medio. El experimento E7 muestra, no obstante, que la reproducibilidad perfecta del modelo de 14B depende de la información que recibe.

Este experimento también parece contradecir al E5, donde Qwen2.5 de 14B no descartó ningún fraude y aquí lo hace en la mitad de las ejecuciones. No hay contradicción: cambia el punto de operación. Con un umbral estricto, casi todo lo elevado es fraude y un modelo mediano no se equivoca. Al rebajar el umbral para detectar más fraude, el expediente se llena de casos dudosos y el mismo modelo empieza a equivocarse. El tamaño de modelo necesario depende, por tanto, de la dificultad de los casos que se le presentan, y no tiene sentido fijar un número mínimo de parámetros sin especificar el umbral.

6.7.4 Resultados corregidos durante el trabajo

Varias cifras de este experimento se corrigieron durante su desarrollo, y se documentan porque ilustran errores que no producen ningún aviso.

Una versión anterior reportaba treinta y dos ejecuciones por modelo, con 37, 27 y 13 fraudes destruidos y contrastes significativos ($p = 0,024$ y $p = 0,002$). Esas cifras procedían de combinar tres ficheros de resultados obtenidos en días distintos. Al revisarlos se comprobó que veinticuatro de las cuarenta y ocho combinaciones de modelo, turno y repetición estaban duplicadas: eran repeticiones de la misma ejecución, no réplicas nuevas. El tamaño de muestra estaba inflado y, con él, la significación. Al corregir el recuento, el resultado deja de ser significativo.

Con ocho ejecuciones por modelo, el de 20B causaba daño en una sola, lo que sugería un comportamiento distinto al de los demás. Con dieciséis, su tasa se sitúa en torno a un tercio de las ejecuciones: aquella primera tanda era excepcional. La afirmación no era falsa con los datos de ese momento, pero sí prematura.

Una versión anterior ilustraba la falta de reproducibilidad del modelo de 8B con recalls de 0,60, 0,40 y 0,80 ante la misma entrada. La cifra es real, pero procede de los lotes sintéticos con cinco fraudes, en los que un solo caso que cambie de veredicto mueve el recall en 0,20. Sobre turnos reales, con entre veintiséis y cincuenta y cuatro fraudes, la variación del mismo modelo está entre 0,019 y 0,039. La conclusión se mantiene, pero la magnitud aparente dependía del número de fraudes del lote. También se retiraron un contraste de Fisher ($p = 0,0191$) y uno de Wilcoxon ($p = 0,0235$) de esa versión, porque no se reproducen con los datos actuales.

6.8 E7. Efecto de la capa 2 y ablación del expediente

6.8.1 Objetivo y diseño

Este experimento es el que resuelve el objetivo específico OE2 y, con él, la pregunta central del trabajo: si la capa multiagente aporta algo sobre el detector. Responde a dos preguntas. La primera es si la capa 2, trabajando con el umbral permisivo de 0,05, mejora al detector funcionando solo con su umbral estricto, que es la línea base. La segunda es si, al descartar casos, el modelo razona sobre los datos o se limita a seguir la puntuación que el propio expediente le entrega.

Para responder a la segunda se realizó una ablación del expediente. El dossier de cada caso incluía la probabilidad de fraude y el nivel de riesgo, este último calculado con el mismo umbral que decide qué casos se elevan. El experimento se repitió con tres niveles de información: completo, con probabilidad y nivel de riesgo; sin nivel, solo con la probabilidad; y ciego, sin ninguna de las dos. En los tres niveles se mantienen las contribuciones por variable, el importe y la hora. Se realizaron cuarenta y ocho ejecuciones con Qwen2.5 de 14B: dieciséis por nivel, sobre los cuatro turnos con cuatro repeticiones cada uno.

6.8.2 Resultados

La Tabla 27 compara, en cada turno, el recall del detector solo con umbral estricto y el recall del sistema con la capa 2 y umbral permisivo.

Turno	Recall sin capa 2	Recall con capa 2
0 h	0,759	0,778
1 h	0,784	0,784
2 h	0,711	0,711
3 h	0,692	0,692

Tabla 27. Recall del detector con umbral estricto frente al sistema con la capa 2 y umbral 0,05.

La diferencia media es de +0,005 en recall y de -0,001 en precisión: la capa 2 deja el sistema prácticamente donde lo dejaba un umbral más estricto.

La Tabla 28 recoge los resultados de la ablación. Para cada nivel se indica qué información ve el modelo, el número medio de casos que descarta por ejecución, cuántos de esos descartes son correctos (es decir, eran falsas alarmas) y la precisión del descarte, que es la proporción de descartes correctos.

Nivel	Qué ve el modelo	Descarta	Acierta	Precisión del descarte
Completo	Probabilidad y nivel de riesgo	9,9	9,2	92 %
Sin nivel	Solo probabilidad	9,4	8,5	90 %
Ciego	Ni probabilidad ni nivel	3,5	2,9	82 %

Tabla 28. Ablación del expediente: descartes y su acierto según la información disponible (medias por ejecución).

La Figura 19 representa gráficamente la ablación. El panel izquierdo compara, para cada nivel, el volumen de descartes, en azul y expresado como porcentaje del nivel completo (100, 95 y 35), con el porcentaje de descartes acertados, en verde (92, 90 y 82). El panel derecho muestra en cuántos de los cuatro turnos las cuatro repeticiones dieron exactamente el mismo resultado.

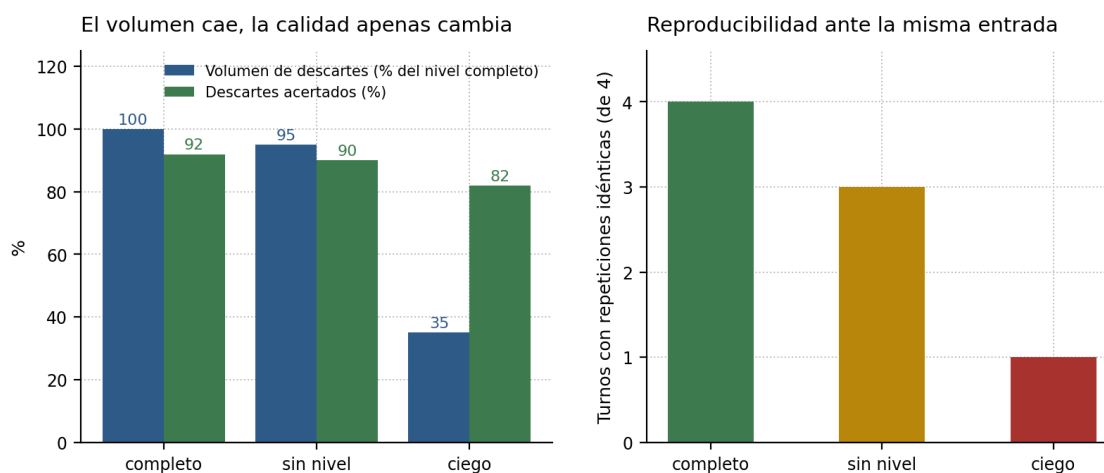


Figura 19. Efecto del nivel de información del expediente sobre el volumen de descartes, su acierto y la reproducibilidad entre ejecuciones.

La Tabla 29 recoge esa reproducibilidad en forma de tabla.

Nivel	Turnos con las cuatro repeticiones idénticas
Completo	4 de 4
Sin nivel	3 de 4
Ciego	1 de 4

Tabla 29. Turnos con las cuatro repeticiones idénticas según el nivel de información.

6.8.3 Discusión

La capa 2 no mejora al detector: con umbral permisivo, el sistema completo obtiene prácticamente el mismo resultado que el detector solo con un umbral estricto.

La ablación separa dos magnitudes que el experimento E6 no distinguía. El volumen de descartes cae mucho al retirar la puntuación: de 9,9 casos a 3,5, un 65 % menos. La calidad de esos descartes, en cambio, apenas baja: el modelo acierta el 82 % de sus

descartes frente al 92 %. En la proporción de falsas alarmas del expediente, elegir al azar acertaría en torno al 29 %, así que el modelo sigue siendo unas 2,8 veces mejor que el azar sin la puntuación, frente a 3,1 veces con ella.

La interpretación es que la puntuación del detector no le da al modelo capacidad para distinguir un fraude de una falsa alarma, sino confianza para decidir. El modelo extrae información útil de las contribuciones por variable por sí solo; lo que le falta sin la puntuación no es criterio, sino seguridad, y pasa a confirmar casi todo lo que se le presenta.

Esa información útil tiene un alcance limitado. Un 82 % de precisión en los descartes es muy superior al azar, pero insuficiente para operar: incluso con el 92 % del nivel completo, la capa 2 destruyó doce fraudes reales a lo largo de dieciséis ejecuciones.

La reproducibilidad es el resultado de mayor alcance práctico. La reproducibilidad perfecta de Qwen2.5 de 14B, que el experimento E6 señalaba como su principal ventaja, no era una propiedad del modelo: dependía de recibir un número en el enunciado. Sin la puntuación, el modelo decide distinto ante la misma entrada, igual que el modelo de 8B. Para un sistema que debe poder auditarse, la conclusión es de diseño: la trazabilidad de las decisiones no procede del modelo de lenguaje, sino de haberlas anclado a la salida determinista del detector.

6.9 E8. Operación en continuo

6.9.1 Objetivo y diseño

Este experimento aborda el objetivo específico OE2 en su vertiente operativa: comprobar si la capa construida puede funcionar de forma continua y a qué coste. Los experimentos anteriores evalúan el sistema por lotes: recibe un turno completo de cuatro horas y produce un informe. Un departamento antifraude real recibe las transacciones de forma continua. Este experimento reproduce ese funcionamiento: acumula las transacciones de una ventana corta, las analiza, emite el informe y pasa a la siguiente ventana. Se comparan ventanas de 5, 10 y 30 minutos con el turno de 4 horas, midiendo el número de casos por ventana, el coste de cómputo, la ocupación de la máquina y la espera media hasta que un caso aparece en un informe.

6.9.2 Resultados

La Tabla 30 recoge, para cada tamaño de ventana, el número medio de transacciones y de casos elevados, y el porcentaje de ventanas en las que no se eleva ningún caso.

Ventana	Transacciones	Casos elevados	Ventanas sin ningún caso
5 min	683	0,9	52 %
10 min	1.366	1,8	33 %
30 min	4.099	5,2	0 %
4 h	32.791	42,0	0 %

Tabla 30. Transacciones y casos elevados según el tamaño de ventana.

El coste por ventana resultó proporcional al número de tareas, con un tiempo por tarea muy estable, entre seis y ocho segundos. El número de tareas es el número de casos investigados más dos fijas, las del agente que presenta los casos y la del que redacta el informe. Con ventanas de diez minutos, medido con el modelo real, el sistema ocupa el 3,9 % del tiempo disponible y ninguna ventana queda sin procesar a tiempo.

La Tabla 31 recoge la espera media desde que se produce una transacción hasta que aparece en un informe.

Ventana	Espera media hasta la alerta
4 h	120 min
30 min	13,2 min
10 min	6,0 min
5 min	2,9 min

Tabla 31. Espera media hasta la alerta según el tamaño de ventana.

La Figura 20 representa ambos resultados. El panel izquierdo muestra, en escalas logarítmicas, que la espera media crece en proporción al tamaño de la ventana, desde unos 3 minutos con ventanas de 5 hasta 120 minutos con turnos de 4 horas. El panel derecho muestra la ocupación de cómputo medida en cada ejecución: reducir la ventana no aumenta el coste.

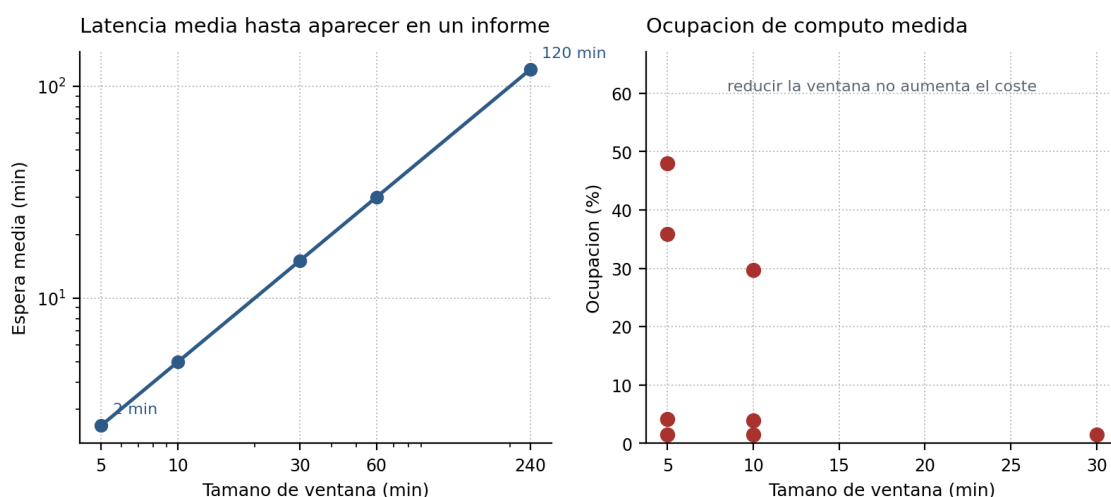


Figura 20. Espera media hasta aparecer en un informe frente al tamaño de ventana (izquierda) y ocupación de cómputo medida en las ejecuciones registradas (derecha).

6.9.3 Discusión

La capacidad de cómputo no es el factor limitante. Reducir la ventana de cuatro horas a cinco minutos divide la espera por cuarenta sin un coste de cómputo apreciable. En este dominio la diferencia es importante: el patrón de fraude más frecuente consiste en un cargo de importe mínimo seguido de uno mayor, de modo que el momento de la alerta decide si el segundo cargo llega a producirse.

El experimento reveló además un defecto que no aparecía con turnos de cuatro horas. La selección de casos incluía una regla de respaldo para que el informe nunca saliera vacío: si no había suficientes casos por encima del umbral, se completaba el expediente con los siguientes de la lista. Con turnos de cuatro horas esa regla nunca se activa, porque siempre hay decenas de casos de riesgo alto. Con ventanas de cinco minutos se habría activado en más de la mitad de ellas, y su implementación completaba el expediente hasta la capacidad máxima del equipo en lugar de hasta el mínimo previsto. Una ventana con un solo caso real habría elevado sesenta, de los cuales cincuenta y nueve serían transacciones legítimas. Es el ejemplo más claro del trabajo de una decisión razonable en un modo de funcionamiento que resulta perjudicial en otro, y de un defecto que ninguna prueba sobre el modo original podía descubrir.

6.10 E9. Comparación con un modelo de mayor tamaño

6.10.1 Objetivo y diseño

El objetivo específico OE4 pide cuantificar qué se pierde por emplear modelos que caben en la memoria de una tarjeta de consumo frente a modelos de mayor tamaño. La opción más directa, usar un modelo comercial a través de internet, habría contradicho la

restricción de partida del trabajo, que exige no enviar datos a terceros. Por eso se ejecutó en local un modelo de 72B, Qwen2.5-72B, cuantizado a 4 bits.

Un modelo de ese tamaño no cabe en una tarjeta de 16 GB. Para ejecutarlo se utilizó la biblioteca AirLLM [56], que aplica la carga por capas: un modelo de lenguaje está formado por una secuencia de capas que se aplican una tras otra, y AirLLM mantiene en la memoria de la tarjeta una sola capa cada vez y lee las demás desde el disco a medida que las necesita. La memoria necesaria pasa a depender del tamaño de una capa y no del modelo completo, a cambio de una velocidad mucho menor.

Se aisló el Investigador, que es el único agente que emite veredictos. Ejecutar el sistema completo habría consumido horas en agentes que no influyen en lo que se mide. El guion de este experimento reproduce el enunciado exacto del Investigador y mide con la misma función que el sistema completo. Antes de usarlo se comprobó que reproduce el comportamiento del sistema: con Qwen2.5 de 14B sobre los casos 44 a 55 del turno 0 h, destruye los mismos dos fraudes que el sistema completo en el experimento E6.

A una hora y cincuenta minutos por caso, no era posible analizar el expediente entero. Los últimos casos del expediente, los de menor probabilidad, son casi todos transacciones legítimas que todos los modelos descartan, y no permiten distinguir entre modelos. Por eso la comparación se dirigió a los dos únicos casos de esa zona que son fraude real, el 50 y el 55, y se añadieron dos casos legítimos como control. Es una selección deliberada: el resultado no es una tasa, sino la respuesta a si el modelo mayor salva esos dos casos concretos.

6.10.2 Resultados

La Figura 21 muestra la ejecución con AirLLM. A la izquierda, la herramienta nvidia-smi indica que la memoria de la tarjeta está casi vacía (1,7 de 16 GB) y que su utilización es de un solo dígito. A la derecha, la consola muestra la carga de las capas del modelo. La consola no muestra nada nuevo hasta que termina cada caso, porque cada caso es una única llamada que tarda casi dos horas, de modo que la prueba de que el proceso está trabajando es la utilización que indica nvidia-smi.

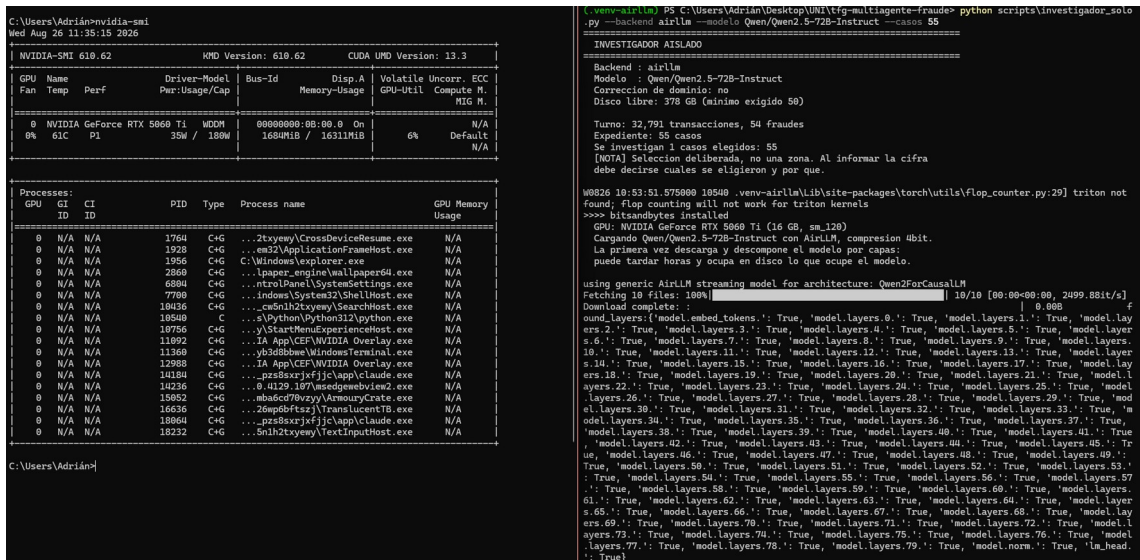


Figura 21. Ejecución con AirLLM junto a nvidia-smi: memoria de vídeo casi vacía (1,7 de 16 GB) y utilización de GPU de un solo dígito mientras el modelo de 72B procesa un caso.

La Tabla 32 recoge los veredictos de ambos modelos sobre los cuatro casos.

Caso	Clase real	Qwen2.5 14B	Qwen2.5 72B
50	Fraude	Descartado	Descartado
53	Legítima	Descartado	Descartado
54	Legítima	Descartado	Descartado
55	Fraude	Descartado	Descartado

Tabla 32. Veredictos de Qwen2.5 de 14B y de 72B sobre los cuatro casos seleccionados del turno 0 h.

Los veredictos son idénticos: el modelo de 72B destruye los mismos dos fraudes que el de 14B.

El coste, en cambio, es muy distinto, como muestra la Tabla 33.

	Qwen2.5 14B	Qwen2.5 72B, cuantizado a 4 bits
Segundos por caso	3,7	6.622,7
Expediente de 55 casos	3,4 min	6.071 min (extrapolado)
Utilización de GPU	—	9 % a 25 %

Tabla 33. Coste de ejecución de los modelos de 14B y 72B.

El modelo de 72B es 1.790 veces más lento. Analizar un turno de cuatro horas llevaría algo más de cuatro días.

Para entender por qué se descartan los dos fraudes se examinó la explicación escrita del modelo de 72B. Sobre el caso 55, un fraude real de 2,22 euros, el modelo escribió que, dado que el nivel de riesgo es bajo y el importe muy reducido, es probable que se trate de un falso positivo. Sobre el caso 50, un fraude real de 1,00 euro, escribió que, dado el bajo importe y el equilibrio entre factores, la transacción parece más bien

una anomalía sin intención fraudulenta clara. Ambos se descartan porque el importe es bajo.

La Tabla 34 muestra la distribución de importes del turno evaluado para cada clase.

	Fraude	Legítima
Número de transacciones	54	32.737
Importe mediano	2,11 EUR	19,99 EUR
Hasta 1 EUR	40,7 %	10,2 %
Hasta 2 EUR	50,0 %	17,4 %
Hasta 5 EUR	59,3 %	25,1 %

Tabla 34. Distribución de importes por clase en el turno 0 h.

La mitad de los fraudes del turno no supera los 2,11 euros, frente a 19,99 euros en las transacciones legítimas. Una transacción fraudulenta tiene 2,4 veces más probabilidad de ser de cinco euros o menos que una legítima.

6.10.3 Discusión

El modelo de mayor tamaño no decide mejor sobre estos casos, y su coste lo hace inviable. La utilización de GPU observada indica además que el cuello de botella es la lectura de las ochenta capas del modelo desde disco para cada token generado, y no el cálculo, de modo que una tarjeta más potente no lo resolvería. La cuantización a 4 bits desactiva además la lectura anticipada de capas de la biblioteca, lo que impide solapar la lectura y el cálculo.

El fallo no se debe a la falta de capacidad del modelo, sino a una creencia equivocada. La idea de que un importe bajo indica una operación inocente es razonable en general, pero en este dominio es justo al revés: el patrón más frecuente del fraude con tarjeta es el cargo mínimo de prueba antes del cargo grande, como se explicó en el apartado 2.1, y el modelo lo interpreta como señal de inocencia. El caso 54 muestra que el modelo no ignora los datos: ante una transacción legítima de 912 euros, razona correctamente sobre las contribuciones de las variables y la descarta con acierto. Lee las contribuciones y después las anula con una creencia que en estos datos es falsa. El experimento E10 estudia qué ocurre al corregir esa creencia.

Son solo cuatro casos, por lo que no es posible ningún contraste estadístico; es una demostración y así debe leerse. Una primera ejecución se invalidó porque AirLLM no aplicaba la plantilla de conversación del modelo, es decir, el formato que indica al modelo qué parte del texto es una petición del usuario. Sin ella, el modelo no interpretaba el texto como una petición, sino como un documento que debía continuar.

Este error se describe en el apartado 6.14. El resultado más sólido del experimento es el coste, que no depende de la selección de casos.

6.11 E10. Corrección de la premisa del importe

6.11.1 Objetivo y diseño

Este experimento responde a los objetivos específicos OE2 y OE5: interviene sobre el funcionamiento de la capa de agentes y, al comparar lo que el modelo escribe con lo que decide, valida sus salidas más allá de lo que comprueba la auditoría automática. El experimento E9 mostró que los modelos descartan fraude alegando que el importe es bajo, una premisa que en estos datos es falsa. Cabía esperar que el problema tuviera una solución sencilla: advertir al modelo en el enunciado.

Se añadió al enunciado del Investigador un párrafo, denominado corrección de dominio, que advierte de que el cargo mínimo de prueba es habitual en el fraude con tarjeta y de que un importe reducido no basta por sí solo para descartar un caso. Se evitó a propósito la formulación contraria, que el importe bajo indica fraude, porque habría dado la respuesta al modelo y habría reproducido el defecto que obligó a repetir la ablación del expediente (fase 11). La corrección retira una premisa falsa, pero no introduce la opuesta.

La medición abarca los cuatro turnos, los doce últimos casos del expediente de cada turno y dos modelos (Llama 3.1 de 8B y Qwen2.5 de 14B), con y sin corrección: 192 veredictos en total. Además de las métricas habituales, se cuenta en cuántos casos la explicación menciona el patrón del cargo de prueba y en cuántos de ellos el modelo descarta el caso a pesar de mencionarlo.

El experimento permite evaluar las dos propiedades de una explicación definidas en el apartado 2.5. La plausibilidad se refleja en si la explicación incorpora un conocimiento correcto del dominio, el patrón del cargo de prueba. La fidelidad se refleja, de forma indirecta, en si la decisión es coherente con lo que dice la explicación.

6.11.2 Resultados

La Figura 22 y la Figura 23 muestran el experimento en el demostrador sobre el caso 55 (transacción 1885), un fraude real de 2,22 euros, con Qwen2.5 de 14B. Sin la corrección, la explicación usa el importe bajo como argumento y el veredicto es descartado. Con la corrección, la explicación dice que la transacción «podría ser una prueba inicial antes de realizar cargos más grandes», que es exactamente el patrón de

fraude, y el veredicto sigue siendo descartado. En ambos casos el demostrador advierte en rojo de que el caso era fraude real.

Departamento antifraude

Detector tabular con una capa de agentes encima. Todo se ejecuta en local.

✓ conjunto de datos

✓ detector entrenado

✓ Ollama (10 modelos)

Turno activo: 4 h desde +0 h

umbral 0.05

capacidad 60

55 casos en el expediente (numerados del 1 al 55)

44 son fraude real

1 Turno

2 Caso

3 Investigación

4 Informe

3. Investigación de un caso

El Investigador produce dos salidas: un párrafo explicativo y un veredicto. Se muestran por separado a propósito, porque el resultado central de este trabajo es que no siempre se corresponden.

Caso

Modelo

Tokens

55

qwen2.5-14b-ctx16k:latest

220

☐ Corrección de dominio

Investigar

PÁRRAFO EXPLICATIVO

VEREDICTO

CASO 55

El detector considera sospechosa la transacción por su importe bajo de 2.22 EUR, que contrasta significativamente con el importe medio del lote de 84.25 EUR. Además, las variables PCA V14 y V4 muestran valores altos que contribuyen hacia una clasificación de fraude, mientras que V1 y V7 presentan valores que tienden a indicar transacciones legítimas, pero no son suficientes para contrarrestar completamente la sospecha generada por las otras variables.

DESCARTADO

6.31 s · qwen2.5-14b-ctx16k:latest · 135 tokens

Este caso **era fraude real** y el modelo lo ha retirado del informe. Es exactamente el fallo que la memoria cuantifica: el sistema detecta menos con la capa 2 que sin ella.

► Ver el enunciado completo enviado al modelo

Figura 22. Investigación del caso 55 sin la corrección de dominio: el modelo usa el importe bajo como argumento y descarta el fraude.

Departamento antifraude

Detector tabular con una capa de agentes encima. Todo se ejecuta en local.

✓ conjunto de datos

✓ detector entrenado

✓ Ollama (10 modelos)

Turno activo: 4 h desde +0 h

umbral 0.05

capacidad 60

55 casos en el expediente (numerados del 1 al 55)

44 son fraude real

1 Turno

2 Caso

3 Investigación

4 Informe

3. Investigación de un caso

El Investigador produce dos salidas: un párrafo explicativo y un veredicto. Se muestran por separado a propósito, porque el resultado central de este trabajo es que no siempre se corresponden.

Caso

Modelo

Tokens

55

qwen2.5-14b-ctx16k:latest

220

☒ Corrección de dominio

Investigar

PÁRRAFO EXPLICATIVO

VEREDICTO

CASO 55

El detector considera sospechoso el CASO 55 debido a que, aunque el importe de la transacción es bajo (2.22 EUR), las variables PCA V14 y V4 muestran valores altos que apuntan hacia un posible fraude, con aportes significativos de +3.004 y +2.180 respectivamente. Además, el importe medio del lote es considerablemente mayor (84.25 EUR), lo que sugiere que esta transacción podría ser una prueba inicial antes de realizar cargos más grandes.

DESCARTADO

6.47 s · qwen2.5-14b-ctx16k:latest · con corrección de dominio · 140 tokens

Este caso **era fraude real** y el modelo lo ha retirado del informe. Es exactamente el fallo que la memoria cuantifica: el sistema detecta menos con la capa 2 que sin ella.

► Ver el enunciado completo enviado al modelo

Figura 23. Investigación del caso 55 con la corrección de dominio: la explicación reconoce el patrón del cargo de prueba y el veredicto descarta igualmente el fraude.

La Tabla 35 recoge los resultados agregados. Para cada modelo y condición se indican el número de casos confirmados, la precisión y el recall sobre estos casos, el

número de explicaciones que mencionan el patrón del cargo de prueba y, de ellas, cuántas acaban igualmente en descarte.

Modelo	Corrección	Confirmaciones	Precisión	Recall	Menciona el patrón	Y descarta igual
llama3.1:8b	No	13 de 48	0,15	0,40	0	—
llama3.1:8b	Sí	36 de 48	0,08	0,60	20	3 (15 %)
qwen2.5:14b	No	6 de 48	0,33	0,40	0	—
qwen2.5:14b	Sí	12 de 48	0,17	0,40	23	14 (61 %)

Tabla 35. Efecto de la corrección de dominio sobre 192 veredictos (48 por modelo y condición).

La corrección se incorpora al texto de forma sistemática: sin ella, el patrón del cargo de prueba no aparece ni una vez en noventa y seis casos; con ella, aparece en cuarenta y tres. La decisión, en cambio, no mejora en ningún modelo: la precisión cae a la mitad en ambos y el fraude recuperado en total pasa de seis casos a cinco.

El comportamiento difiere según el modelo. En Qwen2.5 de 14B, la explicación y el veredicto se separan: el modelo nombra el patrón, lo reconoce en el caso concreto y, aun así, descarta el caso en el 61 % de las ocasiones. Sobre un caso escribe que podría ser un intento inicial de verificación previo a una transacción mayor, y lo descarta. Sobre otro, que el bajo importe se asocia habitualmente a intentos preliminares de uso fraudulento, y lo descarta. Su recall no cambia. En Llama 3.1 de 8B ocurre lo contrario: la explicación y el veredicto van juntos, con solo un 15 % de contradicciones, pero el modelo pasa a confirmar treinta y seis de cuarenta y ocho casos. Recupera un fraude más, pero deja de filtrar: confirmar tres de cada cuatro casos equivale a no aportar nada sobre la salida del detector.

La mención del patrón se detecta de forma automática, exigiendo que la explicación contenga a la vez dos grupos de expresiones: uno relacionado con la idea de prueba o tanteo y otro con un cargo posterior de mayor importe. Cualquiera de ellos por separado aparece en contextos que no tienen que ver con el patrón, porque el modelo menciona el importe continuamente. Este criterio es deliberadamente estricto y deja fuera menciones más vagas que un lector humano sí contaría. Esos casos no contados hacen que la tasa de contradicción real sea, si acaso, mayor que la medida, de modo que la conclusión no se ve favorecida por el criterio.

6.11.3 Discusión

La corrección aumenta la plausibilidad de la explicación, que pasa a incluir conocimiento válido del dominio, pero no su fidelidad a la decisión, que sigue siendo la

misma. Una tasa alta de contradicción demuestra que la explicación y el veredicto no van juntos, pero no demuestra por qué mecanismo interno ocurre. El montaje experimental solo da acceso a lo que el modelo escribe y a lo que decide, no a su proceso interno.

Los modelos de razonamiento quedan fuera de esta comparación por el motivo descrito en el apartado 6.14: agotan el límite de tokens antes de emitir la respuesta, y solo una de ocho ejecuciones completó los doce veredictos.

Este trabajo no compara sus resultados con modelos comerciales accesibles por internet. La omisión es deliberada. El escenario que se modela es el de una entidad que no puede enviar datos de transacciones a un tercero, y una cifra obtenida con un modelo remoto no sería utilizable en ese escenario. El objetivo OE4 se resolvió por ello con la carga por capas del experimento E9.

6.12 E11. Extensión exploratoria con variables de negocio

6.12.1 Objetivo y diseño

Este experimento extiende los objetivos específicos OE1 y OE2 a un segundo conjunto de datos. El experimento E2 mostró que las dos únicas variables interpretables del conjunto ULB apenas influyen en la detección, de modo que lo único que la capa 2 puede explicar en términos comprensibles es casi irrelevante para la decisión. Este experimento comprueba qué ocurre con un conjunto de datos en el que las variables sí tienen significado de negocio. Es una exploración con un único modelo, sin ajuste de hiperparámetros y sin repeticiones, y no pretende sustituir las conclusiones de los experimentos anteriores.

El conjunto se generó con Sparkov [67], el mismo simulador que produjo el conjunto «Credit Card Transactions Fraud Detection Dataset» publicado en Kaggle. Se generó en lugar de descargarse porque así se puede fijar el periodo y el número de clientes: 1.000 clientes simulados durante todo el año 2024, con una semilla fija para que sea reproducible. Es un conjunto simulado, no de transacciones reales, y por eso este experimento se considera exploratorio.

A cambio, cada transacción incluye el comercio, su categoría, las coordenadas del domicilio del titular y del comercio, la población de la ciudad, la profesión y la fecha de nacimiento del titular. A partir de esos campos se derivaron seis variables de negocio (importe, hora del día, día de la semana, edad del titular, distancia en kilómetros entre domicilio y comercio, y población de la ciudad) más la categoría del comercio codificada en catorce variables binarias. Al abarcar un año completo, permite una partición

temporal con volumen real: entrenamiento con los diez primeros meses y prueba con los dos últimos.

La Tabla 36 recoge el tamaño de ambos conjuntos y su proporción de fraude.

	Filas	Fraude	Prevalencia
Entrenamiento	653.869	8.161	1,248 %
Prueba	192.881	1.321	0,685 %

Tabla 36. Composición del conjunto Sparkov en entrenamiento y prueba.

Se entrenó solo XGBoost, con la misma ponderación de la clase minoritaria que en el sistema principal y sin búsqueda de hiperparámetros. Después se ejecutó el Investigador, con Qwen2.5 de 14B, en dos pruebas: sobre los cinco fraudes con mayor puntuación, y en una ablación sobre nueve casos con dos versiones del mismo dossier.

6.12.2 Resultados

La Tabla 37 recoge las métricas del detector sobre el conjunto de prueba.

AUC-PR	ROC-AUC	P@50	P@100	P@200
0,078	0,723	0,420	0,400	0,345

Tabla 37. Métricas del detector XGBoost sobre el conjunto Sparkov.

Las cifras no son comparables directamente con las del conjunto ULB, cuyo detector sí pasó por una búsqueda de hiperparámetros completa. Lo relevante es la parte alta de la lista: entre los cincuenta casos con mayor puntuación, el 42 % son fraude, frente a una prevalencia del 0,685 %, es decir, unas sesenta y una veces más que el azar, aunque muy por debajo del 0,98 del sistema principal. Las métricas con umbral fijo en 0,5 no son informativas (precisión de 0,009 con recall de 0,948), porque la ponderación de la clase minoritaria desplaza hacia arriba la mayoría de las probabilidades; por eso el punto de operación se fija con la P@N.

La diferencia principal está en las contribuciones por variable, que citan conceptos que un analista reconoce. La Tabla 38 muestra las contribuciones de un fraude del comercio «Kozey-Kuhlman», de la categoría de cuidado personal, de 744,54 dólares, a 109,7 km del domicilio del titular y con probabilidad 1,0000. Las variables que más empujan hacia fraude son la categoría del comercio, la hora y el importe.

Variable	Aporte
categoria_personal_care	+5,109
hora	+3,606
importe	+3,285
edad	+1,247
categoria_home	+0,365
dia_semana	+0,282

Tabla 38. Contribuciones por variable de un caso de fraude del conjunto Sparkov.

En la primera prueba, el Investigador confirmó correctamente los cinco fraudes. Con probabilidad 1,0000 en los cinco, confirmarlos no requiere discriminar nada difícil; lo relevante es el texto. Sobre un cargo de 902,19 dólares en el comercio Medhurst, Cartwright and Ebert, de cuidado personal, el modelo escribe que el importe es alto para ese tipo de establecimiento, que la operación ocurre a las 22:00 y que el titular, de 85 años, vive a 120,7 km del comercio, lo que «dificulta la verosimilitud de la transacción». Sobre un cargo de viaje de 866,48 dólares en Mosciski Group, sopesa la distancia y la baja población de la ciudad del titular frente al importe y la categoría, y concluye que estos últimos «superan estas posibles anomalías geográficas».

Para separar el efecto de que las variables tengan nombre del efecto de cambiar de conjunto de datos, se realizó una ablación sobre los mismos nueve casos con dos versiones del dossier. La versión completa incluye comercio, categoría, edad, distancia y población con su nombre real. La versión reducida anonimiza esas cinco variables como var_1 a var_5, igual que ocurre en el conjunto ULB, y solo conserva con su nombre el importe y la hora. Las contribuciones numéricas son idénticas en ambas versiones. Los casos se eligieron por ser aquellos en los que el detector más se equivoca: los seis fraudes con menor probabilidad de todo el conjunto de prueba (entre 0,0018 y 0,0205) y las tres transacciones legítimas con la probabilidad más alta (1,0000). Son los casos en los que un criterio propio del modelo tendría más ocasión de notarse.

La Tabla 39 recoge los veredictos en ambas versiones.

Casos	Clase real	Importe	Dossier completo	Dossier reducido
1 a 6	Fraude	9,67 a 156,52 USD	Descartados	Descartados
7 a 9	Legítima	955 a 2.135 USD	Confirmados	Confirmados

Tabla 39. Veredictos del Investigador con el dossier completo y con el reducido sobre los mismos nueve casos.

El veredicto es idéntico en las dos versiones en los nueve casos. Dar nombre a las variables no recupera ningún fraude ni corrige ninguna falsa alarma. El modelo

reproduce exactamente el error del detector: confirma las tres transacciones legítimas que el detector puntúa alto y descarta los seis fraudes que puntúa bajo.

El texto sí cambia de forma sistemática. En la versión completa, el modelo nombra el comercio, comenta la categoría con sus propias palabras («servicios de transporte de gas», «categoría de comida y bebida») y pondera la edad o la distancia. En la reducida, la misma estructura de argumentos se construye sobre «var_1» y «var_2» sin perder coherencia.

6.12.3 Discusión

El experimento permite comprobar dos cosas. La primera es que la limitación del apartado 4.2.1, según la cual las explicaciones no pueden referirse al negocio porque los datos no lo permiten, es del conjunto de datos y no del sistema: con otro conjunto, las mismas herramientas producen expedientes legibles y explicaciones que los aprovechan, sin cambiar ninguna línea de la capa 2. La segunda es que esa legibilidad no basta: saber qué significa cada variable no cambió ningún veredicto en los casos en que el detector más se equivoca. El modelo no necesita entender una variable para basar su veredicto en su valor, y entenderla tampoco le hace decidir distinto. La legibilidad cambia lo que el texto puede argumentar, no lo que el sistema decide.

Se trata de un único modelo, sin ajuste de hiperparámetros, sobre datos simulados y con una sola semilla. Catorce casos en total no permiten ningún contraste estadístico, de modo que es un resultado cualitativo.

6.13 E12. Ajuste fino con QLoRA

6.13.1 Objetivo y diseño

Este experimento responde al objetivo específico OE2, explorando la última de las tres vías consideradas para modificar el comportamiento de la capa de agentes. El experimento E10 mostró que corregir la premisa del importe en el enunciado cambia la explicación, pero no la decisión. Queda una alternativa: incorporar esa corrección a los pesos del modelo mediante ajuste fino, de modo que deje de ser un consejo que el modelo puede ignorar en cada llamada y pase a formar parte de su comportamiento.

Se entrenó un adaptador LoRA de rango 16 sobre Qwen2.5-14B-Instruct cuantizado a 4 bits, es decir, con la técnica QLoRA descrita en el apartado 2.3 [28, 29]. El rango indica el tamaño del adaptador: cuanto mayor, más parámetros entrena. Se usaron 540 ejemplos contruidos solo con transacciones anteriores al corte de la partición temporal; el guion comprueba en cada ejemplo que su fecha es anterior a ese corte, para que

ningún caso de la evaluación se haya visto durante el entrenamiento. El conjunto de ejemplos equilibra fraudes y falsas alarmas, e incluye una proporción mayor de fraudes de importe bajo, que es precisamente el patrón que el modelo no reconocía. Las respuestas de los ejemplos se generaron con una plantilla a partir de las contribuciones reales del detector, no se redactaron a mano. El entrenamiento tardó 29,6 horas, mucho más de lo previsto, porque la tarjeta de 16 GB se compartía con el resto del sistema operativo, y terminó con una pérdida de 0,086.

Se comparan tres condiciones sobre los mismos doce casos del turno 0 h usados en los experimentos E9 y E10, con el mismo enunciado: el modelo base, el modelo base con la corrección en el enunciado y el modelo con el adaptador y sin corrección en el enunciado.

6.13.2 Resultados

La Tabla 40 recoge, para cada condición, cuántos de los doce casos confirma, qué fraudes destruye y qué casos confirma.

Condición	Confirma	Fraude destruido	Casos confirmados
Base	0 de 12	2 (casos 50 y 55)	Ninguno
Base con corrección	1 de 12	2 (casos 50 y 55)	44
Con adaptador	1 de 12	2 (casos 50 y 55)	44

Tabla 40. Veredictos del modelo base, con corrección en el enunciado y con el adaptador QLoRA sobre doce casos.

Los veredictos del modelo con adaptador son idénticos, caso por caso, a los del modelo base con la corrección en el enunciado. El adaptador no salva los fraudes 50 y 55, y confirma el mismo caso de control, el 44, que no es fraude.

Lo que cambia por completo es el texto. Frente a los párrafos largos de las otras dos condiciones, el modelo adaptado responde siempre con la misma estructura, breve: cita la probabilidad del expediente, dos variables a favor de la sospecha y dos en contra con sus valores, y cierra con una conclusión. Sobre el caso 55 escribe: «el detector eleva el CASO 55 con probabilidad 0,0673 y un importe de 2,22 EUR. Empujan hacia fraude V14 (+3,004) y V4 (+2,180). En sentido contrario, V1 (-2,603) y V7 (-1,660). El conjunto de aportes no sostiene la sospecha: el patrón es compatible con una operación legítima». Sobre el caso 44, el único que confirma, escribe: «el importe reducido no es un atenuante: el cargo mínimo de prueba es un patrón habitual de fraude, y las contribuciones dominantes lo respaldan».

6.13.3 Discusión

La probabilidad que cita el modelo (0,0673 en el caso 55, 0,1286 en el 50 y 0,6081 en el 44) es la que le entrega el expediente, y coincide siempre con el sentido del veredicto: con probabilidad baja descarta y con probabilidad más alta confirma. Es la misma dependencia de la puntuación del detector que mostró el experimento E7, ahora sin cambios en el enunciado. El modelo no aprendió a razonar de otra forma sobre el importe; aprendió una plantilla que resume fielmente las contribuciones y decide según la puntuación del expediente. El uso de respuestas generadas por plantilla es la explicación más probable de que el estilo se haya vuelto tan uniforme.

Por tanto, la separación entre explicación y decisión no depende de que la corrección llegue por el enunciado o por los pesos. En ambos casos el modelo dispone de la misma información y decide con el mismo criterio, que no es el importe sino la puntuación del detector.

Son doce casos y un adaptador entrenado una sola vez, por lo que no es posible ningún contraste estadístico. Un conjunto de entrenamiento con respuestas más variadas podría dar otro resultado, y no se ha medido si el adaptador generaliza a otros fraudes de importe bajo.

6.14 Errores silenciosos detectados

Este apartado cierra el objetivo específico OE5, en su parte de documentación de los errores observados. A lo largo de los experimentos se detectaron varios errores que no producían ninguna excepción ni aviso, sino resultados de apariencia correcta. Se denominan en esta memoria errores silenciosos. Este apartado los reúne porque son una aportación práctica del trabajo para quien construya sistemas similares.

La Figura 24 muestra uno de ellos reproducido en el demostrador. Un modelo de razonamiento, gpt-oss de 20B, con un límite de 220 tokens, dedica todos los tokens a razonar y devuelve una respuesta vacía. El demostrador muestra un aviso en ámbar; sin ese aviso, el veredicto ausente sería indistinguible de un dato.

Departamento antifraude

Detector tabular con una capa de agentes encima. Todo se ejecuta en local.

✓ conjunto de datos ✓ detector entrenado ✓ Ollama (10 modelos)

Turno activo: 4 h desde +0 h umbral 0.05 capacidad 60 55 casos en el expediente (numerados del 1 al 55) 44 son fraude real

1 Turno 2 Caso 3 Investigación 4 Informe

3. Investigación de un caso

El Investigador produce dos salidas: un párrafo explicativo y un veredicto. Se muestran por separado a propósito, porque el resultado central de este trabajo es que no siempre se corresponden.

Caso

Modelo

Tokens

55

gpt-oss-20b-ctx16klatest

☐ Corrección de dominio

220

Investigar

El modelo agota los 220 tokens razonando y no llega a emitir respuesta. Es un modelo de razonamiento: gasta el presupuesto pensando antes de escribir. Sube el límite.

PÁRRAFO EXPLICATIVO

VEREDICTO

sin texto

no reconocido

12.34 s · gpt-oss-20b-ctx16klatest · 220 tokens · respuesta truncada

► Ver el enunciado completo enviado al modelo

Figura 24. Error silencioso reproducido en el demostrador: el modelo de razonamiento agota los 220 tokens pensando y devuelve una respuesta vacía.

La Tabla 41 resume los siete errores silenciosos principales, qué ocurrió en cada uno y cómo se corrigió.

Error	Qué ocurrió	Corrección
Identificadores inventados	Un informe recomendaba bloquear cinco transacciones inexistentes	Expediente determinista
Invencción de conceptos	El informe atribuía país, comercio e historial que no existen en los datos	Auditoría de conceptos prohibidos
Truncamiento del contexto	El tamaño de contexto por defecto cortaba el expediente sin avisar	Ampliación explícita del contexto
Pérdida al sintetizar	La investigación estaba completa, pero el informe solo recogía una parte de los casos	Auditoría separada por etapa
Cambio de idioma	El informe salió en inglés y los patrones del auditor, en castellano, no detectaban nada	Instrucción de idioma y patrones en ambos idiomas
Parámetro no propagado	Un parámetro que limita el tamaño del expediente no llegaba a los agentes: se informaba de un caso mientras se investigaban sesenta	Propagar el parámetro por toda la cadena
Estimación presentada como medida	La instrumentación calculaba el coste con una constante supuesta y lo mostraba junto a valores medidos, sin distinguirlos	Modelo de coste ajustado sobre medidas y etiquetado explícito

Tabla 41. Errores silenciosos detectados durante el desarrollo y su corrección.

Los cuatro primeros producen texto falso. El quinto es el más sutil de los que afectan al sistema: produce texto correcto, pero deja sin efecto al auditor.

Los dos últimos afectan a la herramienta de medida y no al sistema evaluado, lo que los hace más peligrosos: un defecto del sistema puede descubrirse midiendo, pero un defecto en la medida no. El sexto apareció al medir la ocupación en el experimento E8: la instrumentación informaba correctamente del número de casos elevados, pero el sistema procesaba sesenta, porque el límite del expediente se aplicaba en el recuento y no en la información entregada a los agentes. Ninguna cifra resultaba absurda por separado; solo al comparar el número de tareas ejecutadas con el número de casos declarados se hizo visible. En el séptimo, la primera versión de la constante estaba equivocada en un orden de magnitud, y la ocupación resultante, que parecía una medición, era veinte veces menor que la real. La lección es que hay que auditar también la instrumentación, y que la separación entre lo medido y lo estimado debe ser explícita en cada valor mostrado.

Además de los recogidos en la tabla, se detectaron otros errores del mismo tipo:

- Aplicar SMOTE antes de dividir el conjunto produce un AUC-PR de 1,0000, una cifra que solo una revisión metodológica identifica como imposible (experimento E3).
- Omitir en el enunciado la descripción de la salida esperada hizo que el modelo escribiera el veredicto con un formato ligeramente distinto, y el programa no extrajo ningún veredicto de diez, con todas las métricas a cero.
- Al combinar las salidas de varios casos, un encabezado sin contenido se emparejaba con el veredicto del caso siguiente y se perdía el último caso. En la ejecución afectada, el caso perdido era el único fraude, y la cifra publicada de fraude destruido fue cero cuando el valor real era uno: el error favorecía al sistema.
- Los modelos de razonamiento agotan el límite de tokens pensando antes de responder, y la respuesta llega vacía o cortada sin veredicto. Se observó primero con Qwen3 y el demostrador lo reprodujo con gpt-oss, de otra familia. Con el límite común a los demás modelos no respondían; al cuadruplicarlo, solo una de ocho ejecuciones completó los doce veredictos, y las ejecuciones con veredictos incompletos se mezclaron sin aviso con las completas en una tabla comparativa, porque el guion solo descartaba las ejecuciones sin ningún veredicto.
- AirLLM no aplicaba la plantilla de conversación del modelo, que interpretaba el texto como un documento que continuar y no como una petición (experimento E9).
- Combinar ficheros de experimentos con ejecuciones duplicadas elevó el tamaño de muestra de dieciséis a treinta y dos ejecuciones por modelo y convirtió en significativos dos contrastes que no lo son (experimento E6).

Por último, la Tabla 42 muestra un límite detectado en la síntesis del informe. Para cada tamaño de expediente se indica qué porcentaje de casos cubre la investigación, qué porcentaje cubre el informe final y cuántos casos cubre este en promedio.

Expediente (casos)	Cobertura de la investigación	Cobertura del informe	Casos cubiertos en el informe
55	100 %	26 %	14,2
53	100 %	27 %	14,5
42	100 %	35 %	14,5
32	100 %	45 %	14,5

Tabla 42. Cobertura de la investigación y del informe final según el tamaño del expediente.

Aunque el expediente varía entre 32 y 55 casos, el informe final siempre cubre unos catorce. Dividir la investigación en una tarea por caso resolvió la cobertura de la investigación, pero trasladó el límite a la redacción del informe. Por eso es importante que la tabla de casos del informe se ensamble sin intervención del modelo, como se describe en el apartado 4.1: garantiza que todos los casos elevados figuren en el informe aunque el texto redactado no los comente todos.

7. Conclusiones

Este capítulo valora el grado de cumplimiento de los objetivos, resume las conclusiones principales, sitúa el trabajo respecto a la literatura revisada y recoge sus limitaciones y las líneas de trabajo futuro.

7.1 Grado de consecución de los objetivos

La Tabla 43 resume el grado de consecución de cada objetivo específico y los experimentos o apartados que lo sustentan. La correspondencia es la misma que recoge la columna de objetivos de la Tabla 15, en el apartado 6.1.2. Se detalla a continuación.

Objetivo	Grado de consecución	Dónde se sustenta
OE1. Detector de fraude	Alcanzado	Capítulo 4; experimentos E1 a E4
OE2. Capa multiagente local	Alcanzado en su construcción; sin mejora de la detección	Capítulo 4; experimentos E5 a E8 y E10 a E12
OE3. Comparación de modelos locales	Alcanzado; diferencias no significativas	Experimentos E5 y E6
OE4. Comparación con un modelo de mayor tamaño	Alcanzado en local	Experimento E9
OE5. Validación automática y errores	Alcanzado	Apartados 4.6 y 6.14; experimento E10

Tabla 43. Grado de consecución de los objetivos específicos.

OE1. Construir y evaluar un detector de fraude. Alcanzado. El detector basado en XGBoost obtiene un AUC-PR de 0,883 en partición aleatoria y de 0,797 en partición temporal, con una precisión en los cincuenta casos de mayor puntuación de 1,00 y 0,98 respectivamente. En la partición aleatoria detecta el 83,7 % del fraude del conjunto de prueba. Se compararon tres algoritmos y cuatro estrategias de tratamiento del desbalanceo, se ajustaron los hiperparámetros y se comprobó que la estrategia elegida no provoca fuga de información ni pierde rendimiento frente al remuestreo bien aplicado. El Isolation Forest, sin usar etiquetas, obtiene 0,152 y 0,034 de AUC-PR, lo que cuantifica la ventaja de disponer de ellas.

OE2. Diseñar una capa multiagente local. Alcanzado en cuanto a la construcción, pero sin beneficio en la detección. El sistema de seis agentes en CrewAI produce informes en castellano, con cobertura completa de los casos elevados y sin invenciones detectables por la auditoría automática. Sin embargo, no mejora la fiabilidad de la detección: en el mejor de los casos la deja igual, y cuando se le permite descartar casos la reduce.

OE3. Comparar modelos de lenguaje locales. Alcanzado. Se compararon cinco modelos en lotes sintéticos y tres en turnos reales. Los modelos más grandes destruyen menos fraude, pero la diferencia no es estadísticamente significativa, y el tamaño necesario depende del umbral de operación.

OE4. Cuantificar la pérdida frente a un modelo de mayor tamaño. Alcanzado sin romper la restricción de ejecución local. Mediante carga por capas se ejecutó un modelo de 72.000 millones de parámetros en una tarjeta de 16 GB. Emite los mismos veredictos que el de 14.000 millones con un coste 1.790 veces mayor.

OE5. Validar las salidas y documentar los errores. Alcanzado. Cada informe se audita automáticamente frente al expediente y frente a una lista de conceptos que no existen en los datos, se dispone de un banco de pruebas que mide el detector y el sistema completo con la misma función, y se han documentado los errores silenciosos detectados. Todos los datos experimentales están versionados junto al código, de modo que cualquier cifra de esta memoria puede rastrearse hasta la ejecución que la produjo.

7.2 Conclusiones principales

7.2.1 El detector es la contribución principal

El trabajo ha construido un detector de fraude con un rendimiento dentro del rango publicado para el conjunto de referencia y, sobre todo, con una evaluación rigurosa. Se evalúa con partición aleatoria y temporal, se miden la precisión en los primeros casos y los contrastes estadísticos, y se ha comprobado experimentalmente que el error metodológico más común en esta literatura, aplicar el remuestreo antes de dividir los datos, produce métricas perfectas sin ningún aviso. La ponderación de la clase minoritaria resulta tan eficaz como el remuestreo bien aplicado y no puede provocar fuga de información.

El trabajo muestra además que la elección entre modelos depende del protocolo: XGBoost y Random Forest difieren de forma significativa en AUC-PR, con signo distinto según la partición, pero son idénticos en los cincuenta primeros casos, que son los que revisa un equipo. Y muestra que el límite del sistema lo marca el detector: nueve de los 54 fraudes del turno evaluado reciben probabilidad prácticamente nula, y ninguna capa posterior puede recuperarlos.

7.2.2 El tamaño del modelo de lenguaje no es el factor limitante

La hipótesis inicial era que los modelos que caben en una tarjeta de consumo son insuficientes y que un modelo mayor decidiría mejor. Los datos no la respaldan.

Sobre los dos fraudes de la zona final del expediente, Qwen2.5 de 72B emite exactamente los mismos veredictos que Qwen2.5 de 14B: descarta ambos. El coste difiere en un factor de 1.790, con 6.623 segundos por caso frente a 3,7. La utilización de GPU, entre el 9 % y el 25 %, indica que el cuello de botella es la lectura de las capas desde disco, de modo que una tarjeta más potente no lo resolvería.

Entre los modelos que sí caben en memoria, los más grandes destruyen menos fraude (19, 12 y 8 casos en dieciséis ejecuciones para los modelos de 8B, 14B y 20B), pero la diferencia no alcanza significación estadística: la prueba de Fisher da $p = 0,156$ y la de Mann-Whitney, $p = 0,050$. La tendencia es sugerente, pero no está demostrada.

7.2.3 Corregir la premisa cambia la explicación, no la decisión

Los modelos descartaban fraudes alegando que el importe era bajo. En estos datos esa premisa es falsa: en el turno evaluado, el importe mediano de un fraude es de 2,11 euros frente a 19,99 de una transacción legítima, y el 59,3 % de los fraudes no llega a cinco euros, frente al 25,1 % de las legítimas. La relación se mantiene sobre el conjunto completo, con menor intensidad: mediana de 9,25 frente a 22,00 euros, y 45,1 % frente a 24,2 % por debajo de cinco euros.

Al advertir al modelo de que un importe bajo no descarta el fraude, su explicación incorpora la advertencia (el patrón del cargo de prueba pasa de no aparecer nunca a aparecer en 43 de 96 casos), pero su decisión no mejora. En Qwen2.5 de 14B, la explicación reconoce el patrón y el veredicto lo contradice en el 61 % de esos casos. En Llama 3.1 de 8B, la explicación y la decisión van juntas, pero el modelo pasa a confirmar casi todo y deja de filtrar. Ajustar el modelo con QLoRA para incorporar la corrección a sus pesos produce exactamente los mismos veredictos que la corrección en el enunciado: cambia el estilo del texto, no la decisión.

En los términos de la literatura revisada en el apartado 2.5, la corrección aumenta la plausibilidad de la explicación sin aumentar su fidelidad. La consecuencia práctica es que una explicación bien redactada y coherente con el dominio no demuestra que la decisión se haya tomado por esas razones, y no debe usarse como prueba para auditar al sistema que la produce. Este trabajo propone medir la proporción de casos en que la explicación reconoce un indicio y la decisión lo ignora. No es una medida de fidelidad en sentido estricto, porque el proceso interno del modelo no es observable, sino de consistencia entre explicación y decisión, y tiene la ventaja de poder comprobarse en un sistema en funcionamiento sin instrumentación adicional.

7.2.4 La puntuación del detector aporta confianza, no criterio

La ablación del expediente muestra que, al retirar la puntuación del detector, el número de descartes cae un 65 %, mientras que su acierto solo baja del 92 % al 82 %. La reproducibilidad entre ejecuciones pasa de cuatro turnos de cuatro a uno. La puntuación no da al modelo capacidad para distinguir un fraude de una falsa alarma: le da seguridad para decidir y estabilidad para repetir la decisión. El ajuste fino con QLoRA confirma este mecanismo, porque el modelo adaptado decide según la probabilidad que le entrega el expediente.

El experimento con el conjunto Sparkov lo confirma desde otro ángulo. Dar nombre real a las variables de negocio o anonimizarlas no cambió ningún veredicto en los nueve casos en que el detector más se equivoca: el modelo reproduce el error del detector en todos ellos. La legibilidad mejora lo que el texto puede argumentar, pero no lo que el sistema decide.

7.2.5 El cuello de botella operativo es la capacidad de revisión

El detector eleva más casos de los que un equipo puede revisar, y es esa limitación, y no la calidad del modelo de lenguaje, la que determina cuánto fraude se detecta: pasar de 5 a 42 casos revisados multiplica el recall por ocho. La operación en continuo lo confirma: con ventanas de diez minutos la ocupación de cómputo es del 3,9 %, de modo que sobra capacidad de cálculo y falta capacidad humana. La espera media hasta la alerta cae de 120 minutos con turnos de cuatro horas a unos 6 minutos con ventanas de diez, sin coste apreciable.

7.2.6 Los errores del sistema no generan avisos

Ninguno de los errores detectados durante el desarrollo produjo una excepción. Todos generaron resultados de apariencia correcta: métricas perfectas por fuga de información, informes con transacciones inventadas, veredictos perdidos que favorecerían al sistema, contrastes significativos por ejecuciones duplicadas o costes estimados presentados como medidos. Un sistema de este tipo exige, por tanto, validar por programa sus salidas y desconfiar de los resultados que confirman lo esperado.

La construcción del demostrador web confirmó la misma conclusión desde otro ángulo. Se validó con un plan de catorce pruebas que encontró seis defectos en él, y ninguno producía un error visible: todos generaban pantallas de apariencia correcta. El estado del turno se guardaba en el servidor y el navegador lo olvidaba al recargar la página, de modo que la interfaz pedía casos de un expediente que creía tener mientras el servidor trabajaba con otro, y el mensaje de error resultante («el caso 55 no existe») era cierto pero engañoso, porque no explicaba la causa. El auditor recibía identificadores

reales de transacción cuando el informe citaba números de caso, de modo que un informe válido aparecía con cobertura nula y cincuenta y cinco citas marcadas como inexistentes. Y una respuesta cortada por el límite de tokens era indistinguible de una respuesta en la que el modelo hubiera omitido el veredicto, hasta que se incorporó la lectura del motivo de parada que informa el servidor de modelos. Que una herramienta auxiliar de cuatro pantallas reproduzca el mismo tipo de fallo que el sistema al que sirve refuerza la conclusión: en estos sistemas el modo de fallo por defecto no es la excepción, sino la salida plausible.

7.3 Posicionamiento respecto a la literatura

Una vez presentados los resultados, la Tabla 44 compara el trabajo con la literatura revisada en el capítulo 2. La columna central recoge únicamente afirmaciones comprobadas en los trabajos leídos.

Aspecto	Literatura	Este trabajo
Métrica del detector	Entre 0,78 y 0,97 de AUC-PR sobre el conjunto ULB	0,883 de AUC-PR en partición aleatoria y 0,797 en partición temporal
Tratamiento del desbalanceo	Remuestreo frecuente, con fuga documentada	Ponderación de la clase minoritaria, sin remuestreo
Partición	Aleatoria casi siempre	Aleatoria y temporal
Papel del modelo de lenguaje	Clasificar, o explicar atribuciones en lenguaje natural	Capa de interpretación medida frente a una línea base
Autoridad del modelo	Explica, no decide	Se le permite descartar casos y se mide el efecto
Infraestructura	Modelos comerciales a través de internet	Local, hardware de consumo, coste cero
Evaluación del sistema de agentes	Comparación entre configuraciones del propio modelo, sin contraste estadístico	Comparación con el detector solo, con ablaciones y contrastes estadísticos
Errores del sistema	Poco documentados	Catálogo de errores silenciosos, con su corrección
Reproducibilidad entre ejecuciones	Rara vez medida	Medida ante entradas idénticas
Evaluación de la explicación	Fidelidad y plausibilidad, propuestas como criterios	Medida de consistencia entre explicación y decisión sobre un sistema en funcionamiento

Tabla 44. Posicionamiento del trabajo respecto a la literatura revisada.

La tabla resume el posicionamiento, pero conviene extraer de ella las conclusiones concretas que dan valor a la propuesta frente a cada grupo de trabajos revisados.

Frente a los detectores publicados, la aportación no está en la cifra sino en que la cifra sea interpretable. El 0,883 de AUC-PR obtenido aquí queda por debajo del 0,97 de FraudX AI [10] y se sitúa en la zona media del rango publicado para este conjunto de

datos. Sin embargo, de los trabajos revisados ninguno acompaña su métrica de las tres cosas que sí acompañan a esta: una partición temporal que evalúe el modelo sobre transacciones posteriores a las que usó para aprender, la precisión en los primeros casos, que es la métrica que corresponde a la capacidad real de revisión de un equipo, y una comprobación experimental propia de cuánto infla la métrica el error metodológico más extendido del dominio. El contraste con Taha y Malebary [13] lo ilustra: su 98,40 % de exactitud no es comparable con nada, porque con un 0,17 % de fraude ese valor se obtiene clasificando bien lo fácil, y su propio F1 del 56,95 % lo confirma. Un valor de 0,883 del que se conoce el protocolo exacto es más útil que un 0,97 del que no.

Frente a los trabajos que han documentado la fuga de información, este añade la comprobación desde dentro y la alternativa. Kabane [16] y Magnier [17] examinan la literatura publicada y muestran que los resultados extremos se explican por el orden del preprocesamiento. Aquí el fenómeno se reproduce sobre la propia tubería de datos del trabajo, con un AUC-PR de 1,0000 al aplicar SMOTE antes de dividir, y se da un paso más: se comprueba que ponderar la clase minoritaria rinde igual que el remuestreo correctamente aplicado. La advertencia metodológica se convierte así en una recomendación accionable, porque hacer las cosas bien no cuesta rendimiento.

Frente a Carcillo y otros [14], este trabajo aporta la magnitud del fenómeno sobre el conjunto de referencia público. Ellos observan que las puntuaciones de anomalía solo ayudan con una granularidad concreta y solo en AUC-PR; aquí se cuantifica el punto de partida de esa discusión: el Isolation Forest, sin etiquetas, obtiene 0,152 y 0,034 de AUC-PR frente a los 0,883 y 0,797 del modelo supervisado. La detección no supervisada no es una alternativa a las etiquetas en este dominio, sino, como mucho, un complemento.

Frente a los sistemas multiagente financieros, la diferencia es la línea base, y es la que da sentido al resultado negativo. MASCA [36], FinCon [37] y TradingAgents [38] comparan una configuración del modelo de lenguaje con otra configuración del mismo modelo, de modo que pueden responder si un sistema multiagente es mejor que un agente único, pero no si el modelo de lenguaje aporta algo frente a no utilizarlo. Este trabajo toma como línea base el detector funcionando solo y responde a esa segunda pregunta. La respuesta es que no aporta fiabilidad: en el mejor de los casos la deja intacta y, cuando se le concede autoridad para descartar casos, la reduce. Un resultado negativo frente a una línea base que la literatura revisada no utiliza delimita el alcance del enfoque, y es más informativo que una mejora medida contra una referencia complaciente.

Frente a la literatura sobre explicabilidad, la aportación es una medida practicable. Bilal, Ebert y Lin [35] proponen la fidelidad y la plausibilidad como criterios, pero los trabajos que recogen se limitan a traducir a lenguaje natural la salida de un método de atribución, sin que el modelo decida nada. Aquí el modelo decide, y eso permite medir algo que esos trabajos no pueden medir: la proporción de casos en que la explicación reconoce un indicio y el veredicto lo ignora, el 61 % en Qwen2.5 de 14B. No es fidelidad en sentido estricto, porque el proceso interno del modelo no es observable, pero sí es una medida de consistencia que puede calcularse sobre un sistema en funcionamiento sin instrumentación adicional.

Frente a todos ellos, por último, el trabajo aporta una cifra sobre la viabilidad de la ejecución local, que los trabajos revisados no abordan porque emplean modelos comerciales a través de internet. El sistema completo funciona en una tarjeta de consumo de 16 GB con coste económico nulo, y el precio de esa restricción está medido: un modelo de 72.000 millones de parámetros, ejecutado por carga de capas, emite exactamente los mismos veredictos que uno de 14.000 millones con un coste 1.790 veces mayor. Para quien deba decidir si un sistema de este tipo puede operar sin enviar datos a un tercero, esa equivalencia es un dato aprovechable.

La aportación del trabajo es, en suma, doble. Por un lado, un detector de fraude evaluado con una metodología que evita la fuga de información e incluye la partición temporal y la precisión en los primeros casos. Por otro, una medición, bajo la restricción de ejecución local, de lo que aporta una capa de agentes sobre ese detector: no mejora la fiabilidad de la detección, su resultado no depende del tamaño del modelo dentro del rango evaluado, y sus explicaciones no siempre se corresponden con sus decisiones.

7.4 Limitaciones

- El sistema no puede recuperar los fraudes que el detector no eleva. El recall máximo medido es de 0,837 en partición aleatoria y de 0,760 en temporal.
- La comparación entre los modelos que caben en memoria no alcanza significación estadística.
- Las repeticiones dentro de un mismo turno, con temperatura 0,1, son casi idénticas, y los turnos se solapan parcialmente. El número de observaciones realmente independientes es cercano a cuatro, no al número de ejecuciones.
- La comparación con el modelo de 72B se realizó sobre cuatro casos elegidos de forma deliberada. Es una demostración, no una serie estadística.
- La ablación del expediente se realizó con un solo modelo, de modo que no puede afirmarse que la dependencia de la puntuación sea general.

- Las dos únicas variables interpretables del conjunto apenas influyen en la detección, y las explicaciones de la capa de agentes se apoyan precisamente en ellas.
- Los modelos de razonamiento quedan fuera de las comparaciones principales, porque agotan el límite de tokens antes de responder y no son comparables con el diseño experimental empleado, que fija el coste por caso.
- El conjunto principal está anonimizado y cubre un único periodo de 48 horas y una sola región.
- El experimento con variables de negocio usa un conjunto simulado, un único modelo sin ajuste de hiperparámetros y catorce casos en total, por lo que sus resultados son cualitativos.
- El ajuste con QLoRA se evaluó sobre doce casos, con un único adaptador y con respuestas de entrenamiento generadas por plantilla.
- No se ha medido si las explicaciones de la capa de agentes ayudan a un analista humano; haría falta un estudio con usuarios.

7.5 Trabajo futuro

Aplicar el sistema a un dominio con contenido textual. Es la línea principal. En transferencias o en prevención del blanqueo de capitales existen conceptos, ordenantes, países y relaciones entre operaciones sobre los que un modelo de lenguaje puede razonar. En el conjunto empleado aquí, con veintiocho variables anonimizadas, un importe y una hora, hay poco que interpretar más allá de lo que el detector ya calcula, de modo que la limitación observada podría deberse al dominio y no al enfoque.

Repetir el experimento con variables de negocio sobre transacciones reales y con un detector ajustado. El experimento E11 mostró que, con variables legibles, las explicaciones pasan a citar comercio, categoría, distancia y edad, pero los veredictos no cambian. Queda pendiente repetirlo con datos reales y con un detector que haya pasado por la misma búsqueda de hiperparámetros que el del conjunto ULB. La mediana de probabilidad de 0,95 sobre el conjunto de prueba de Sparkov sugiere que parte del problema es de calibración del detector, y separar ambos efectos requiere ese ajuste.

Mejorar el detector. Dado que el detector es la pieza que determina el rendimiento del sistema, conviene explorar combinaciones como la de FraudX AI, que promedia XGBoost y Random Forest, o la incorporación de las puntuaciones del Isolation Forest como variables de entrada, siguiendo a Carcillo y otros.

Repetir el ajuste fino con respuestas más variadas. El experimento E12 dejó abierta la posibilidad de que el resultado se deba a que las respuestas de entrenamiento se

generaron con una plantilla, lo que pudo enseñar al modelo a resumir el expediente en lugar de a decidir distinto. Repetirlo con respuestas redactadas o revisadas por una persona permitiría separar ambas explicaciones.

Retirar el veredicto a la capa de agentes. En la zona de menor probabilidad del expediente ningún modelo discrimina. Limitar la intervención del modelo a la explicación de los casos, sin permitirle descartarlos, conservaría el beneficio sin reducir la fiabilidad de la detección.

Medir el valor de las explicaciones con analistas reales. El trabajo mide si la capa de agentes filtra bien, y no lo hace, pero no mide si sus explicaciones ayudan a una persona a decidir más rápido o mejor, que es el motivo por el que se incorporó. A la vista de la conclusión 7.2.3, ese estudio debería tener en cuenta que una explicación convincente puede no corresponderse con el motivo real de la decisión.

Bibliografía

- [1] European Banking Authority y European Central Bank (2025). 2025 Report on Payment Fraud. Nota de prensa y resumen: <https://www.ecb.europa.eu/press/pr/date/2025/html/ecb.pr251215~e133d9d683.en.html>
- [2] Machine Learning Group, Université Libre de Bruxelles, y Worldline (2018). Credit Card Fraud Detection [conjunto de datos]. Kaggle. <https://www.kaggle.com/datasets/mlg-ulb/creditcardfraud>
- [3] Chen, T. y Guestrin, C. (2016). XGBoost: A Scalable Tree Boosting System. En Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (pp. 785-794). ACM.
- [4] Pedregosa, F., et al. (2011). Scikit-learn: Machine Learning in Python. Journal of Machine Learning Research, 12, 2825-2830.
- [5] CrewAI (2026). CrewAI documentation. <https://docs.crewai.com>
- [6] Ollama (2026). Ollama documentation. <https://ollama.com>
- [7] FastAPI (2026). FastAPI documentation. <https://fastapi.tiangolo.com>
- [8] Dal Pozzolo, A., Caelen, O., Johnson, R. A. y Bontempi, G. (2015). Calibrating Probability with Undersampling for Unbalanced Classification. En 2015 IEEE Symposium Series on Computational Intelligence (pp. 159-166). IEEE.
- [9] Saito, T. y Rehmsmeier, M. (2015). The Precision-Recall Plot Is More Informative than the ROC Plot When Evaluating Binary Classifiers on Imbalanced Datasets. PLOS ONE, 10(3), e0118432.
- [10] Baisholanova, K., et al. (2025). FraudX AI: An Interpretable Machine Learning Framework for Credit Card Fraud Detection on Imbalanced Datasets. Computers, 14(4), 120. <https://doi.org/10.3390/computers14040120>
- [11] Shapley, L. S. (1953). A Value for n-Person Games. En H. W. Kuhn y A. W. Tucker (Eds.), Contributions to the Theory of Games II (pp. 307-317). Princeton University Press.
- [12] Lundberg, S. M. y Lee, S.-I. (2017). A Unified Approach to Interpreting Model Predictions. En Advances in Neural Information Processing Systems 30 (pp. 4765-4774).
- [13] Taha, A. A. y Malebary, S. J. (2020). An Intelligent Approach to Credit Card Fraud Detection Using an Optimized Light Gradient Boosting Machine. IEEE Access, 8, 25579-25587. <https://doi.org/10.1109/ACCESS.2020.2971354>
- [14] Carcillo, F., Le Borgne, Y.-A., Caelen, O., Kessaci, Y., Oblé, F. y Bontempi, G. (2021). Combining unsupervised and supervised learning in credit card fraud detection. Information Sciences, 557, 317-331.

- [15] Chawla, N. V., Bowyer, K. W., Hall, L. O. y Kegelmeyer, W. P. (2002). SMOTE: Synthetic Minority Over-sampling Technique. *Journal of Artificial Intelligence Research*, 16, 321-357.
- [16] Kabane, S. (2024). Impact of Sampling Techniques and Data Leakage on XGBoost Performance in Credit Card Fraud Detection. *arXiv:2412.07437*.
- [17] Magnier, B. (2025). Data Leakage and Deceptive Performance: A Critical Examination of Credit Card Fraud Detection Methodologies. *Mathematics*, 13(16), 2563. <https://doi.org/10.3390/math13162563>
- [18] Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł. y Polosukhin, I. (2017). Attention Is All You Need. En *Advances in Neural Information Processing Systems* 30.
- [19] Grattafiori, A., et al. (2024). The Llama 3 Herd of Models. *arXiv:2407.21783*.
- [20] Qwen Team (2024). Qwen2.5 Technical Report. *arXiv:2412.15115*.
- [21] OpenAI (2025). gpt-oss-120b & gpt-oss-20b Model Card. *arXiv:2508.10925*.
- [22] Patil, S. G., Mao, H., et al. (2025). The Berkeley Function Calling Leaderboard (BFCL): From Tool Use to Agentic Evaluation of Large Language Models. En *Proceedings of the 42nd International Conference on Machine Learning (ICML 2025)*.
- [23] Sun, H., Zhang, S., Niu, L., Ren, L., Xu, H., Fu, H., Zhao, F., Yuan, C. y Wang, X. (2025). Collab-Overcooked: Benchmarking and Evaluating Large Language Models as Collaborative Agents. En *Proceedings of EMNLP 2025* (pp. 4890-4919). <https://doi.org/10.18653/v1/2025.emnlp-main.249>
- [24] Qin, J., Xi, Y., Huang, J., Rui, R., Yin, D., Liu, W., Yu, Y., Zhang, W. y Sun, X. (2025). APTBench: Benchmarking Agentic Potential of Base LLMs During Pre-Training. *arXiv:2510.24397*.
- [25] Brown, T. B., et al. (2020). Language Models are Few-Shot Learners. En *Advances in Neural Information Processing Systems* 33 (pp. 1877-1901).
- [26] Wei, J., Wang, X., Schuurmans, D., Bosma, M., Ichter, B., Xia, F., Chi, E., Le, Q. y Zhou, D. (2022). Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. En *Advances in Neural Information Processing Systems* 35.
- [27] Kojima, T., Guo, S. T., Reid, M., Matsuo, Y. y Iwasawa, Y. (2022). Large Language Models are Zero-Shot Reasoners. En *Advances in Neural Information Processing Systems* 35.
- [28] Hu, E. J., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., Wang, L. y Chen, W. (2022). LoRA: Low-Rank Adaptation of Large Language Models. En *International Conference on Learning Representations (ICLR 2022)*.
- [29] Dettmers, T., Pagnoni, A., Holtzman, A. y Zettlemoyer, L. (2023). QLoRA: Efficient Finetuning of Quantized LLMs. En *Advances in Neural Information Processing Systems* 36.

- [30] Chandola, V., Banerjee, A. y Kumar, V. (2009). Anomaly Detection: A Survey. *ACM Computing Surveys*, 41(3), 1-58.
- [31] Li, A., Zhao, Y., Qiu, C., Kloft, M., Smyth, P., Rudolph, M. y Mandt, S. (2024). Anomaly Detection of Tabular Data Using LLMs. *arXiv:2406.16308*.
- [32] Vapnik, V. N. (1998). *Statistical Learning Theory*. Wiley.
- [33] Tsai, C.-P., Teng, G., Wallis, P. y Ding, W. (2025). AnoLLM: Large Language Models for Tabular Anomaly Detection. En *International Conference on Learning Representations (ICLR 2025)*.
- [34] Tan, X., Ma, Y. y Zhang, X. (2025). Understanding Structured Financial Data with LLMs: A Case Study on Fraud Detection. *arXiv:2512.13040*.
- [35] Bilal, A., Ebert, D. y Lin, B. (2025). LLMs for Explainable AI: A Comprehensive Survey. *arXiv:2504.00125*.
- [36] Bhattacharya, G., et al. (2025). MASCA: LLM based-Multi Agents System for Credit Assessment. *arXiv:2507.22758*.
- [37] Yu, Y., Yao, Z., Li, H., Deng, Z., et al. (2024). FinCon: A Synthesized LLM Multi-Agent System with Conceptual Verbal Reinforcement for Enhanced Financial Decision Making. En *Advances in Neural Information Processing Systems 37*.
- [38] Xiao, Y., Sun, E., Luo, D. y Wang, W. (2024). TradingAgents: Multi-Agents LLM Financial Trading Framework. *arXiv:2412.20138*.
- [39] Rodríguez, J. R. (2014). *Gestión de proyectos informáticos: métodos, herramientas y casos (2ª ed.)*. Editorial UOC.
- [40] Breiman, L. (2001). Random Forests. *Machine Learning*, 45(1), 5-32.
- [41] Liu, F. T., Ting, K. M. y Zhou, Z.-H. (2008). Isolation Forest. En *2008 Eighth IEEE International Conference on Data Mining (pp. 413-422)*. IEEE.
- [42] Python Software Foundation (2026). Python 3.12 documentation. <https://docs.python.org/3.12/>
- [43] NVIDIA (2026). CUDA Toolkit documentation. <https://docs.nvidia.com/cuda/>
- [44] Harris, C. R., et al. (2020). Array programming with NumPy. *Nature*, 585, 357-362.
- [45] McKinney, W. (2010). Data Structures for Statistical Computing in Python. En *Proceedings of the 9th Python in Science Conference (pp. 56-61)*.
- [46] Joblib developers (2026). Joblib: running Python functions as pipeline jobs. <https://joblib.readthedocs.io>
- [47] Lemaître, G., Nogueira, F. y Aridas, C. K. (2017). Imbalanced-learn: A Python Toolbox to Tackle the Curse of Imbalanced Datasets in Machine Learning. *Journal of Machine Learning Research*, 18(17), 1-5.

- [48] Virtanen, P., et al. (2020). SciPy 1.0: Fundamental Algorithms for Scientific Computing in Python. *Nature Methods*, 17, 261-272.
- [49] Hunter, J. D. (2007). Matplotlib: A 2D Graphics Environment. *Computing in Science & Engineering*, 9(3), 90-95.
- [50] ggml-org (2026). llama.cpp: LLM inference in C/C++ [software]. <https://github.com/ggml-org/llama.cpp>
- [51] Pydantic (2026). Pydantic documentation. <https://docs.pydantic.dev>
- [52] BerriAI (2026). LiteLLM documentation. <https://docs.litellm.ai>
- [53] Paszke, A., et al. (2019). PyTorch: An Imperative Style, High-Performance Deep Learning Library. En *Advances in Neural Information Processing Systems* 32.
- [54] Wolf, T., et al. (2020). Transformers: State-of-the-Art Natural Language Processing. En *Proceedings of EMNLP 2020: System Demonstrations* (pp. 38-45).
- [55] Hugging Face (2026). bitsandbytes documentation. <https://huggingface.co/docs/bitsandbytes>
- [56] Li, G. (2026). AirLLM: 70B inference with single 4GB GPU [software]. <https://github.com/lyogavin/airllm>
- [57] Mangrulkar, S., Gugger, S., Debut, L., Belkada, Y., Paul, S. y Bossan, B. (2022). PEFT: State-of-the-art Parameter-Efficient Fine-Tuning methods [software]. <https://github.com/huggingface/peft>
- [58] Lhoest, Q., et al. (2021). Datasets: A Community Library for Natural Language Processing. En *Proceedings of EMNLP 2021: System Demonstrations* (pp. 175-184).
- [59] Uvicorn (2026). Uvicorn documentation. <https://www.uvicorn.org>
- [60] WHATWG (2026). HTML Living Standard. <https://html.spec.whatwg.org>
- [61] W3C (2026). CSS Snapshot. <https://www.w3.org/TR/CSS/>
- [62] Ecma International (2025). ECMA-262: ECMAScript Language Specification. <https://tc39.es/ecma262/>
- [63] Bray, T. (Ed.) (2017). The JavaScript Object Notation (JSON) Data Interchange Format. RFC 8259. IETF.
- [64] Chacon, S. y Straub, B. (2014). *Pro Git* (2^a ed.). Apress. <https://git-scm.com/book>
- [65] GitHub (2026). GitHub documentation. <https://docs.github.com>
- [66] Ellson, J., Gansner, E., Koutsofios, L., North, S. C. y Woodhull, G. (2002). Graphviz: Open Source Graph Drawing Tools. En *Graph Drawing (GD 2001)*, LNCS 2265 (pp. 483-484). Springer.
- [67] Harris, B. (2020). Sparkov Data Generation [software]. https://github.com/namebrandon/Sparkov_Data_Generation

Apéndices

A. Manual de instalación y reproducción

La Figura 25 muestra la organización del repositorio. El código determinista de la capa 1 aparece en azul, el estocástico de la capa 2 en rojo, los guiones de experimentación en verde, y los datos y la documentación en gris.

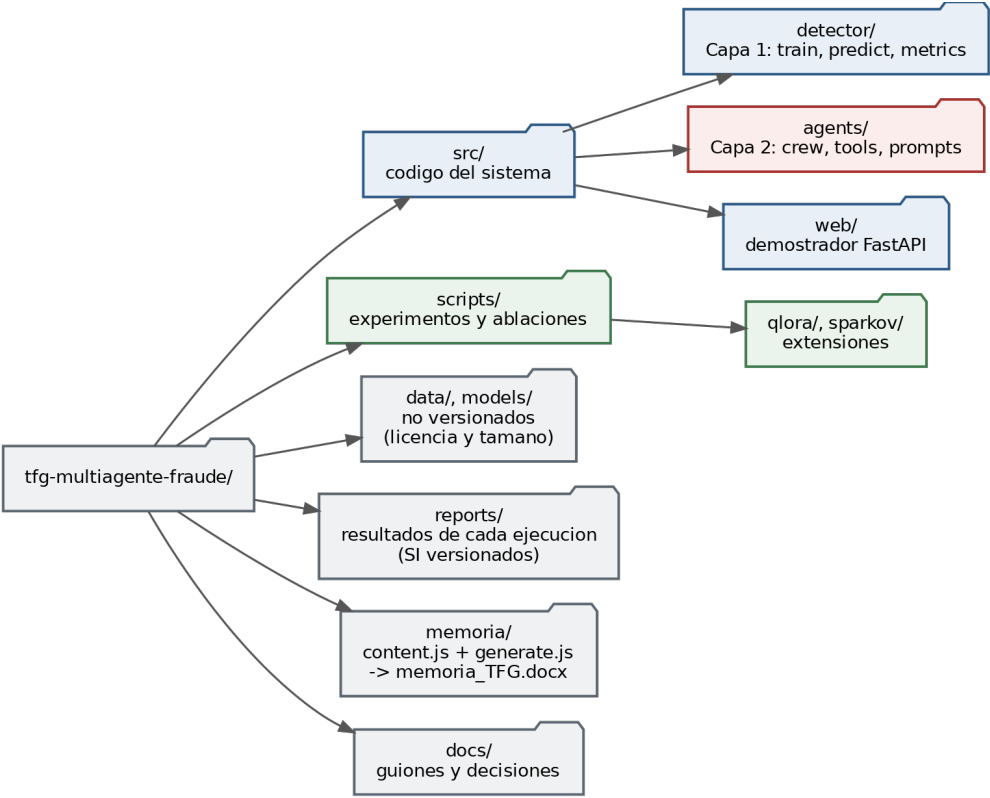


Figura 25. Estructura del repositorio. En azul, el código determinista de la capa 1; en rojo, el estocástico de la capa 2; en verde, los guiones de experimentación; en gris, los datos y la documentación.

La Tabla 45 describe los cuatro directorios principales del repositorio.

Directorio	Contenido
src/detector	Carga de datos, entrenamiento y puntuación, sin dependencias de la capa 2
src/agents	Definición de los agentes, sus herramientas y los fragmentos de enunciado compartidos
scripts	Un guion por experimento, cada uno con su justificación en la cabecera
reports	Resultados de todas las ejecuciones, versionados

Tabla 45. Directorios principales del repositorio.

La reproducción completa requiere Windows o Linux con Python 3.12, el servidor de modelos Ollama y el conjunto de datos, que no se distribuye con el repositorio por su

tamaño y su licencia y debe descargarse de su fuente original [2] y guardarse como data/creditcard.csv. Los pasos, en PowerShell, son los siguientes:

```
git clone https://github.com/adrmurbau/tfg-multiagente-fraude.git
cd tfg-multiagente-fraude
py -3.12 -m venv .venv
.\.venv\Scripts\Activate.ps1
pip install -r requirements.txt
ollama pull llama3.1:8b
python scripts\check_entorno.py
```

El último guion verifica el entorno, como se muestra en la Figura 13. Los modelos del detector se entrenan en segundos con el módulo de entrenamiento del directorio src/detector, y cada experimento del capítulo 6 se reproduce ejecutando su guion del directorio scripts.

Todos los resultados experimentales están bajo control de versiones junto al código que los generó, de modo que cualquier cifra de esta memoria puede rastrearse hasta la ejecución concreta que la produjo y hasta la versión del código vigente en ese momento. Los guiones registran en cada fichero de salida los parámetros con que se invocaron, incluidos los que afectan a la comparabilidad entre tandas, como el límite de tokens por caso.

La comparación con el modelo de 72B (experimento E9) requiere un entorno de Python separado, con una versión de PyTorch compilada para la arquitectura de la tarjeta gráfica, AirLLM y bitsandbytes. Se instala aparte para no alterar el entorno principal; las instrucciones figuran en el fichero requirements.txt.

B. Código fuente

El código completo está disponible en el repositorio del proyecto:

<https://github.com/adrmurbau/tfg-multiagente-fraude>

El repositorio contiene el historial de desarrollo completo: veintidós registros, 2.173 líneas de código en los módulos del sistema y 3.637 en los guiones de experimentación, además de 131 ficheros de resultados.

La Figura 26 muestra, como ejemplo del estilo del código, la función que selecciona los casos del expediente. Los comentarios documentan las decisiones de diseño, como el uso de una ordenación estable para que el número de caso que ve el modelo apunte siempre a la misma transacción.

```

201 # -----
202 def seleccionar_casos(self, df: pd.DataFrame, capacidad: int = None,
203                        umbral: float = None, minimo: int = None) -> pd.DataFrame:
204
205     from src.config import CASOS_MAX, CASOS_MIN
206     if minimo is None:
207         minimo = CASOS_MIN
208
209     tope = capacidad if capacidad is not None else CASOS_MAX
210     p = self.puntuar(df)
211     # kind="mergesort" (estable): hay probabilidades empatadas exactas, y
212     # el quicksort por defecto las desempata de forma no reproducible. Sin
213     # esto, el numero de caso que ve el LLM puede apuntar a una transaccion
214     # distinta en cada ejecucion, y la evaluacion contra etiquetas cruzaria
215     # veredictos con las filas equivocadas.
216     if umbral is not None:
217         altos = p[p["prob_fraude"] >= umbral].sort_values(
218             "prob_fraude", ascending=False, kind="mergesort")
219     else:
220         altos = p[p["riesgo"] == "ALTO"].sort_values(
221             "prob_fraude", ascending=False, kind="mergesort")
222
223     if len(altos) >= minimo:
224         return altos.head(tope)
225     return p.nlargest(max(minimo, min(tope, len(p))), "prob_fraude")
226
227
228 # -----

```

Figura 26. Función de selección de casos del detector, con la documentación de sus decisiones de diseño.

Los guiones se agrupan en tres familias, y cada uno declara en su cabecera la pregunta a la que responde. La Tabla 46 recoge los guiones del sistema en funcionamiento, la Tabla 47 los de experimentos y ablaciones, con el experimento del capítulo 6 al que corresponden, y la Tabla 48 las pruebas que no invocan al modelo de lenguaje.

Guion	Función
check_entorno.py	Verificación del entorno antes de cualquier ejecución
run_departamento.py	Ejecución completa del sistema sobre un turno
run_streaming.py	Operación en continuo por ventanas cortas (E8)
tune_detector.py	Búsqueda de hiperparámetros del detector

Tabla 46. Guiones del sistema en funcionamiento.

Guion	Pregunta que responde
comparar_detectores.py	¿Difieren XGBoost y Random Forest de forma significativa? (E1)
ablacion_variables.py	¿Cuánto contribuye cada grupo de variables? (E2)
ablacion_desbalanceo.py	¿Qué aporta cada estrategia frente al desbalanceo? (E3)
experimento.py	Comparativa entre modelos y modos (E5 a E7)
analisis.py	Análisis estadístico de los resultados anteriores
investigador_solo.py	Comparación con modelos de mayor tamaño (E9)
ablacion_dominio.py	¿Cambia el veredicto al corregir la premisa, o solo la explicación? (E10)

Tabla 47. Guiones de experimentos y ablaciones.

Guion	Qué comprueba
test_detector.py	El detector como herramienta
test_tools.py	Las herramientas de construcción del expediente
test_auditor.py	Regresión del auditor de informes
test_crew_minimo.py	Un agente y una herramienta, para acotar fallos de integración
reauditar.py	Aplica el auditor actual a informes antiguos

Tabla 48. Pruebas que no invocan al modelo de lenguaje.

Que cinco de los dieciséis guiones sean pruebas que no invocan al modelo de lenguaje es deliberado. Como muestra el apartado 6.14, los errores de este tipo de sistema no se manifiestan como excepciones, de modo que la única defensa es comprobar por programa lo que el modelo devuelve.

C. Manual de usuario del demostrador

Este apéndice explica cómo usar el demostrador web descrito en el apartado 4.7. Algunas capturas ya han aparecido a lo largo de la memoria y se repiten aquí para que el manual pueda leerse de forma independiente.

C.1 Puesta en marcha

Con el entorno instalado según el apéndice A y Ollama en ejecución, el servidor se inicia desde la raíz del repositorio con la orden siguiente:

```
python -m uvicorn src.web.app:app --reload --port 8000
```

La Figura 27 muestra la consola una vez iniciado el servidor. A continuación se abre en el navegador la dirección <http://localhost:8000>.

```
(.venv) PS C:\Users\Adrián\Desktop\UNI\tfg-multiagente-fraude> python -m uvicorn src.web.app:app --reload --port 8000
INFO: Will watch for changes in these directories: ['C:\\Users\\Adrián\\Desktop\\UNI\\tf-g-multiagente-fraude']
INFO: Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
INFO: Started reloader process [15284] using WatchFiles
INFO: Started server process [18248]
INFO: Waiting for application startup.
INFO: Application startup complete.
INFO: 127.0.0.1:53509 - "GET / HTTP/1.1" 200 OK
INFO: 127.0.0.1:53509 - "GET /estilo.css HTTP/1.1" 200 OK
INFO: 127.0.0.1:52429 - "GET /app.js HTTP/1.1" 200 OK
INFO: 127.0.0.1:53509 - "GET /favicon.ico HTTP/1.1" 404 Not Found
INFO: 127.0.0.1:52429 - "GET /api/estado HTTP/1.1" 200 OK
```

Figura 27. Inicio del servidor del demostrador desde la consola.

La pantalla inicial, en la Figura 28, muestra en la parte superior tres indicadores que deben aparecer en verde: el conjunto de datos cargado, el detector entrenado y el número de modelos disponibles en Ollama. Debajo aparecen las cuatro pestañas del recorrido (Turno, Caso, Investigación e Informe), que deben usarse en ese orden. Mientras no se construya un turno, las pestañas siguientes avisan de que no hay ningún turno cargado.

Figura 28. Pantalla inicial del demostrador con los indicadores del entorno.

C.2 Pantalla 1: construir un turno

En la pestaña Turno se indican cuatro valores: la duración del turno en horas, el desplazamiento respecto al inicio del periodo de prueba, el umbral de probabilidad y la capacidad de revisión en número de casos. Los valores empleados en la memoria son 4, 0, 0,05 y 60. Al pulsar Detectar, el detector puntúa el turno y muestra, como en la Figura 29, el número de transacciones, los fraudes del turno, los casos elevados, cuántos de ellos son fraude y el tiempo de detección. Debajo aparece la tabla del expediente, con el número de caso, el identificador de la transacción, la probabilidad, el nivel de riesgo, el importe, la hora y la clase real. Al pulsar una fila se abre ese caso en la pantalla 2.

Departamento antifraude

Detector tabular con una capa de agentes encima. Todo se ejecuta en local.

✓ conjunto de datos ✓ detector entrenado ✓ Ollama (10 modelos)

Turno activo: 4 h desde +0 h umbral 0.05 capacidad 60 55 casos en el expediente (numerados del 1 al 55) 44 son fraude real

1 Turno 2 Caso 3 Investigación 4 Informe

1. Construir un turno y detectar

La capa 1 puntúa todas las transacciones del periodo y eleva a revisión las que superan el umbral, hasta agotar la capacidad del equipo.

Duración (h)	Desplazamiento (h)	Umbral	Capacidad (casos)	Detectar
4	0	0,05	60	

32.791

transacciones

54

fraudes en el turno

55

casos elevados

44

de ellos, fraude

264.2 ms

detección

#	IDENTIFICADOR	PROBABILIDAD	RIESGO	IMPORTE	HORA	CLASE REAL
1	15904	1.0000	ALTO	1.00 €	18.2 h	FRAUDE
2	1867	1.0000	ALTO	1.18 €	16.6 h	FRAUDE
3	15854	1.0000	ALTO	1.00 €	18.2 h	FRAUDE
4	24929	1.0000	ALTO	0.77 €	19.3 h	FRAUDE
5	2631	1.0000	ALTO	94.82 €	16.7 h	FRAUDE
6	16003	1.0000	ALTO	1.00 €	18.2 h	FRAUDE
7	20150	1.0000	ALTO	51.37 €	18.7 h	FRAUDE
8	23632	1.0000	ALTO	0.77 €	19.2 h	FRAUDE
9	24279	1.0000	ALTO	0.77 €	19.2 h	FRAUDE
10	7799	1.0000	ALTO	122.68 €	17.2 h	FRAUDE
11	16159	1.0000	ALTO	1.00 €	18.3 h	FRAUDE

La columna de clase real no la ve el sistema en ningún momento: se muestra sólo para poder juzgar sus decisiones. Pulsa una fila para inspeccionar el caso.

Figura 29. Pantalla 1: construcción y puntuación de un turno.

C.3 Pantalla 2: inspeccionar un caso

En la pestaña Caso se indica el número de caso y el nivel de información del expediente (completo, sin nivel o ciego) y se pulsa Ver caso. A la izquierda aparece lo que calcula el detector: probabilidad, importe, hora, clase real y un gráfico de barras con las contribuciones por variable, en rojo las que empujan hacia fraude y en azul las que empujan hacia legítima. A la derecha aparece el texto exacto que recibirá el modelo. La Figura 30 muestra el caso 1, un fraude evidente con probabilidad 1,0000. Cambiar el nivel de información permite ver qué datos se retiran en el experimento E7.

Departamento antifraude

Detector tabular con una capa de agentes encima. Todo se ejecuta en local.

✓ conjunto de datos ✓ detector entrenado ✓ Ollama (10 modelos)

Turno activo: 4 h desde +0 h umbral 0.05 capacidad 60 55 casos en el expediente (numerados del 1 al 55) 44 son fraude real

1 Turno 2 Caso 3 Investigación 4 Informe

2. El caso, tal como lo ve el modelo

A la izquierda, las cifras que calcula el detector. A la derecha, el texto exacto que se incrusta en el enunciado. El desplegable reproduce la ablación de la sección 5.7: retira información y observa qué deja de ver el modelo.

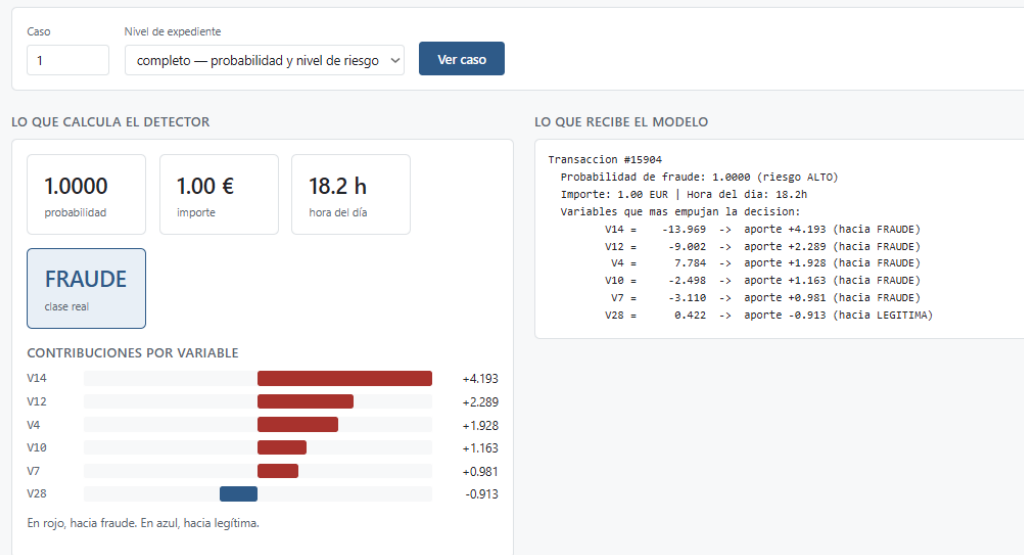


Figura 30. Pantalla 2: inspección del caso 1 con el expediente completo.

C.4 Pantalla 3: investigar un caso

En la pestaña Investigación se indican el número de caso, el modelo de lenguaje, si se aplica la corrección de dominio del experimento E10 y el límite de tokens, y se pulsa Investigar. Tras unos segundos, la pantalla muestra en dos columnas el párrafo explicativo y el veredicto (confirmado en verde o descartado en rojo), junto con el tiempo empleado y los tokens generados. Si el modelo descarta un caso que era fraude real, aparece un aviso rojo. Si el modelo agota los tokens antes de responder, aparece un aviso ámbar, como en la Figura 24; en ese caso debe aumentarse el límite de tokens o elegirse un modelo que no sea de razonamiento. La Figura 31 muestra el caso 1 confirmado, y las figuras Figura 22 y Figura 23 muestran el caso 55 sin y con corrección de dominio.

Departamento antifraude

Detector tabular con una capa de agentes encima. Todo se ejecuta en local.

✓ conjunto de datos ✓ detector entrenado ✓ Ollama (10 modelos)

Turno activo: 4 h desde +0 h umbral 0.05 capacidad 60 55 casos en el expediente (numerados del 1 al 55) 44 son fraude real

1 Turno 2 Caso 3 Investigación 4 Informe

3. Investigación de un caso

El Investigador produce dos salidas: un párrafo explicativo y un veredicto. Se muestran por separado a propósito, porque el resultado central de este trabajo es que no siempre se corresponden.

Caso	Modelo	Tokens	
1	qwen2.5-14b-ctx16k:latest	220	☐ Corrección de dominio Investigar

PÁRRAFO EXPLICATIVO

CASO 1

El detector marca esta transacción como sospechosa debido a su importe extremadamente bajo de solo 1,00 EUR en comparación con el importe medio del lote que es de 84,25 EUR. Además, las variables PCA anonimizadas V14, V12 y V4 presentan valores anormales que contribuyen significativamente al riesgo de fraude, sugiriendo patrones atípicos en los datos. Estas anomalías indican un comportamiento poco común que eleva el nivel de riesgo a ALTO.

VEREDICTO

CONFIRMADO

3.68 s - qwen2.5-14b-ctx16k:latest - 137 tokens

► Ver el enunciado completo enviado al modelo

Figura 31. Pantalla 3: investigación del caso 1, confirmado por el modelo.

C.5 Pantalla 4: generar y auditar el informe

En la pestaña Informe se elige el modelo y el modo de autoridad (interpreta o revisa) y se pulsa Ejecutar departamento. La ejecución tarda varios minutos, porque intervienen los seis agentes. Al terminar, la pantalla muestra el resultado del auditor, en verde si el informe es fiable o en ámbar si está incompleto, junto con la cobertura del expediente, las métricas del turno y del sistema, la tabla de casos y el informe redactado. La Figura 32 muestra un informe fiable en modo interpreta, y la Figura 33 la tabla de casos y el comienzo del informe.

Departamento antifraude

Detector tabular con una capa de agentes encima. Todo se ejecuta en local.

✓ conjunto de datos ✓ detector entrenado ✓ Ollama (10 modelos)

Turno activo: 4 h desde +0 h umbral 0.05 capacidad 60 55 casos en el expediente (numerados del 1 al 55) 44 son fraude real

1 Turno 2 Caso 3 Investigación 4 Informe

4. Informe del departamento

Ejecuta los seis agentes sobre el expediente completo y audita el resultado. Tarda varios minutos: el coste está en la capa 2, no en la detección.

Modelo

qwen2.5-14b-ctx16klatest

Modo

revisa — puede descartar casos

Ejecutar departamento

INCOMPLETO según el auditor automático · 344.9 s · qwen2.5-14b-ctx16klatest · modo revisa

- Cobertura del expediente: 98 % — 54 de 55 casos citados en el informe.
- 21 casos quedaron sin veredicto en modo revisa: el modelo no se pronunció sobre ellos y conservan la decisión del detector.

32791 transacciones del turno	54 fraudes reales	55 casos elevados	44 fraude en el expediente	0.8148 recall del detector	0.8 precisión del detector
34 confirmados por el modelo	0 descartados por el modelo	0 FRAUDE DESTRUIDO	0 descartes acertados	0.8148 recall del sistema	0.8 precisión del sistema

TABLA DE CASOS (ENSAMBLADA SIN MODELO)

Caso	Importe (EUR)	Hora	Veredicto	Acción recomendada
1	1.00	18.2h	Confirmado	Bloquear tarjeta
2	1.18	16.6h	Confirmado	Bloquear tarjeta
3	1.00	18.2h	Confirmado	Bloquear tarjeta
4	0.77	19.3h	Confirmado	Bloquear tarjeta
5	94.82	16.7h	Confirmado	Bloquear tarjeta
6	1.00	18.2h	Confirmado	Bloquear tarjeta
7	51.37	18.7h	Confirmado	Bloquear tarjeta
8	0.77	19.2h	Confirmado	Bloquear tarjeta
9	0.77	19.2h	Confirmado	Bloquear tarjeta
10	122.68	17.2h	Confirmado	Bloquear tarjeta
11	1.00	18.3h	Confirmado	Bloquear tarjeta
12	94.82	19.2h	Confirmado	Bloquear tarjeta
13	0.77	19.1h	Confirmado	Bloquear tarjeta
14	0.00	17.1h	Confirmado	Bloquear tarjeta
15	0.00	18.7h	Confirmado	Bloquear tarjeta
16	0.76	17.2h	Confirmado	Bloquear tarjeta
17	0.77	19.6h	Confirmado	Bloquear tarjeta

Figura 32. Pantalla 4: informe marcado como fiable por el auditor, con las métricas del turno y del sistema.

INFORME REDACTADO			
# Resumen Ejecutivo			
El lote de análisis consta de 55 transacciones consideradas de alto riesgo (rojo) por el detector M3Boost. Todas las transacciones tienen un nivel de probabilidad de fraude del 100% o muy cercano, con importes que varían desde 0.00 EUR hasta 230.00 EUR. La mayoría de los casos presentan valores atípicos en variables como V14, V12 y V6 que empujan la decisión hacia el fraude.			
# Tabla Priorizada			
Caso	Importe (EUR)	Veredicto	Acción Recomendada
[-----]			
CASO 1	1.00	CONFIRMADO	Bloquear tarjeta
CASO 2	1.18	CONFIRMADO	Bloquear tarjeta
CASO 3	1.00	CONFIRMADO	Bloquear tarjeta
CASO 4	0.77	CONFIRMADO	Bloquear tarjeta
CASO 5	94.82	CONFIRMADO	Bloquear tarjeta
CASO 6	1.00	CONFIRMADO	Bloquear tarjeta
CASO 7	13.37	CONFIRMADO	Bloquear tarjeta
CASO 8	0.77	CONFIRMADO	Bloquear tarjeta
CASO 9	0.77	CONFIRMADO	Bloquear tarjeta
CASO 10	120.00	CONFIRMADO	Bloquear tarjeta
CASO 11	1.00	CONFIRMADO	Bloquear tarjeta
CASO 12	94.82	CONFIRMADO	Bloquear tarjeta
CASO 13	0.77	CONFIRMADO	Bloquear tarjeta
CASO 14	0.00	CONFIRMADO	Bloquear tarjeta
CASO 15	0.00	CONFIRMADO	Bloquear tarjeta
CASO 16	0.76	CONFIRMADO	Bloquear tarjeta
CASO 17	0.77	CONFIRMADO	Bloquear tarjeta
CASO 18	4.97	CONFIRMADO	Bloquear tarjeta
CASO 19	0.00	CONFIRMADO	Bloquear tarjeta
CASO 20	0.77	CONFIRMADO	Bloquear tarjeta
CASO 21	1.00	CONFIRMADO	Bloquear tarjeta
CASO 22	18.78	CONFIRMADO	Bloquear tarjeta
CASO 23	1.18	CONFIRMADO	Bloquear tarjeta
CASO 24	6.74	CONFIRMADO	Bloquear tarjeta
CASO 25	0.81	CONFIRMADO	Bloquear tarjeta
CASO 26	35.00	CONFIRMADO	Bloquear tarjeta
CASO 27	230.00	CONFIRMADO	Bloquear tarjeta
CASO 28	58.00	CONFIRMADO	Bloquear tarjeta
CASO 29	2.00	CONFIRMADO	Bloquear tarjeta
CASO 30	1.00	CONFIRMADO	Bloquear tarjeta
CASO 31	1.59	CONFIRMADO	Bloquear tarjeta
CASO 32	1.59	CONFIRMADO	Bloquear tarjeta
CASO 33	1384.40	CONFIRMADO	Bloquear tarjeta
CASO 34	0.76	CONFIRMADO	Bloquear tarjeta
CASO 35	0.77	CONFIRMADO	Bloquear tarjeta
CASO 36	17.73	CONFIRMADO	Bloquear tarjeta
CASO 37	9.29	CONFIRMADO	Bloquear tarjeta
CASO 38	0.00	CONFIRMADO	Bloquear tarjeta
CASO 39	0.77	CONFIRMADO	Bloquear tarjeta
CASO 40	0.76	CONFIRMADO	Bloquear tarjeta
CASO 41	247.80	CONFIRMADO	Bloquear tarjeta
CASO 42	1.00	CONFIRMADO	Bloquear tarjeta
CASO 43	996.27	CONFIRMADO	Bloquear tarjeta
CASO 44	1.00	CONFIRMADO	Bloquear tarjeta
CASO 45	1809.28	CONFIRMADO	Bloquear tarjeta
CASO 46	1.00	CONFIRMADO	Bloquear tarjeta
CASO 47	0.00	CONFIRMADO	Bloquear tarjeta
CASO 48	0.76	CONFIRMADO	Bloquear tarjeta
CASO 49	375.00	CONFIRMADO	Bloquear tarjeta
CASO 50	680.80	CONFIRMADO	Bloquear tarjeta
CASO 51	1.00	CONFIRMADO	Bloquear tarjeta
CASO 52	0.00	CONFIRMADO	Bloquear tarjeta
CASO 53	913.00	CONFIRMADO	Bloquear tarjeta
CASO 54	2.22	CONFIRMADO	Bloquear tarjeta
# Explicación del Investigador			
CASO 1	El detector identifica esta transacción como sospechosa debido a un importe muy bajo (1.00 EUR) y valores atípicos en las variables V14, V12 y V6 que empujan la decisión hacia el fraude. Además, la variable V10 también contribuye de manera significativa al riesgo.		
VEREDICTO:	CONFIRMADO		
CASO 2	Esta transacción se considera sospechosa debido a un importe bajo (1.18 EUR) y valores atípicos en las variables V14, V12 y V6 que empujan la decisión hacia el fraude. La variable V10 también contribuye de manera significativa al riesgo.		
VEREDICTO:	CONFIRMADO		
CASO 3	El detector identifica esta transacción como sospechosa debido a un importe muy bajo (1.00 EUR) y valores atípicos en las variables V14, V12 y V6 que empujan la decisión hacia el fraude. La variable V10 también contribuye de manera significativa al riesgo.		
VEREDICTO:	CONFIRMADO		
CASO 4	Esta transacción se considera sospechosa debido a un importe muy bajo (0.77 EUR) y valores atípicos en las variables V14, V12 y V6 que empujan la decisión hacia el fraude. La variable V10 también contribuye de manera significativa al riesgo.		

Figura 33. Pantalla 4: tabla de casos ensamblada sin modelo y comienzo del informe redactado.

C.6 Problemas frecuentes

- Si algún indicador de la pantalla inicial aparece en rojo, debe comprobarse que el fichero data/creditcard.csv existe, que el detector está entrenado y que Ollama está en ejecución.
- Si al recargar el navegador las pantallas 2 a 4 indican que no hay turno, debe volver a construirse el turno en la pantalla 1.
- Si el veredicto aparece como «no reconocido», el modelo ha agotado el límite de tokens; debe aumentarse el límite o cambiarse de modelo.

D. Glosario

AUC-PR. Área bajo la curva de precisión y exhaustividad. Con clases muy desbalanceadas es preferible al área bajo la curva ROC, porque depende sobre todo de los primeros puestos de la lista, que son los que revisa un analista.

Ablación. Experimento que retira un componente de un sistema para medir cuánto aporta.

Agente. Modelo de lenguaje con un papel, un objetivo y un conjunto de herramientas.

Cadena de pensamiento. Forma de pedir una tarea a un modelo de lenguaje en la que se le solicita que razone paso a paso antes de responder.

Carga por capas. Técnica que mantiene en la memoria de la tarjeta gráfica una sola capa del modelo cada vez y lee las demás desde disco. Permite ejecutar modelos que no cabrían en la memoria disponible, a costa de la velocidad.

Cero disparos. Forma de pedir una tarea a un modelo de lenguaje sin darle ningún ejemplo resuelto.

Cuantización. Representación de los parámetros de un modelo con menos bits para reducir la memoria que ocupa.

Error silencioso. Error que no produce ninguna excepción ni aviso y genera resultados de apariencia correcta.

Expediente. Conjunto de casos que el detector eleva a revisión en un turno, con la información que acompaña a cada uno.

Fidelidad. Grado en que una explicación representa el proceso de decisión que el modelo siguió realmente.

Fraude destruido. Número de fraudes reales elevados por el detector que la capa de agentes descarta en modo revisa.

Fuga de información. Uso, durante el entrenamiento, de información del conjunto con el que después se evalúa el modelo. Produce métricas infladas.

Herramienta. Función del programa que un agente puede pedir que se ejecute y cuyo resultado recibe.

Línea base. Configuración de referencia contra la que se compara una propuesta. En este trabajo, el detector funcionando sin la capa de agentes.

LoRA y QLoRA. Técnicas de ajuste fino que entrenan un pequeño adaptador en lugar de modificar todos los parámetros del modelo; QLoRA lo hace sobre un modelo cuantizado a 4 bits.

Marco de orquestación. Biblioteca que define los agentes y sus tareas, fija el orden en que intervienen y les hace llegar los resultados de los anteriores. En este trabajo, CrewAI.

Modo interpreta y modo revisa. Los dos modos de autoridad de la capa 2. En el primero explica sin decidir; en el segundo puede descartar casos elevados por el detector.

Partición temporal. División entre entrenamiento y prueba según el orden cronológico: se entrena con el pasado y se evalúa con el futuro.

Plausibilidad. Coherencia lógica de una explicación y su consistencia con el conocimiento del dominio. Una explicación puede ser plausible sin ser fiel.

Precisión en los N primeros casos (P@N). Proporción de fraudes reales entre los N casos de mayor puntuación. N representa la capacidad de revisión del equipo.

Prueba de importe mínimo. Patrón de fraude con tarjeta consistente en un cargo pequeño para comprobar que la tarjeta funciona, previo a un cargo mayor.

scale_pos_weight. Parámetro de XGBoost que aumenta el peso de la clase minoritaria durante el entrenamiento. Alternativa al remuestreo que no puede provocar fuga de información.

SHAP. Método de atribución basado en los valores de Shapley que reparte la predicción entre las variables de entrada. Aquí se calcula de forma exacta sobre los árboles del modelo.

SMOTE. Técnica que genera ejemplos sintéticos de la clase minoritaria por interpolación. Aplicada antes de dividir el conjunto, produce fuga de información.

Temperatura. Parámetro que controla la aleatoriedad con la que un modelo de lenguaje elige cada token.

Token. Fragmento de texto, normalmente parte de una palabra, en que un modelo de lenguaje divide la entrada y la salida.

Turno. Conjunto de transacciones de una ventana temporal que el sistema procesa como una unidad; cuatro horas salvo indicación contraria.

E. Uso de herramientas de inteligencia artificial

Este apéndice declara las herramientas de inteligencia artificial generativa empleadas durante la realización del trabajo, y con qué finalidad se ha usado cada una. No se refiere a los modelos de lenguaje que forman parte del sistema desarrollado, que son objeto de estudio del propio trabajo y se describen en el capítulo 5, sino a las herramientas utilizadas como apoyo durante el desarrollo y la redacción.

La Tabla 49 resume esas herramientas.

Herramienta	Tipo	Uso en el trabajo
Claude (Claude Code y Cowork)	Asistente de programación y de redacción	Generación y revisión de código, apoyo a la depuración, y revisión y corrección de la memoria
Gemini	Asistente conversacional	Consultas técnicas y apoyo puntual a la programación
opencode	Asistente de programación en línea de órdenes	Generación y modificación de código sobre el repositorio
Cursor	Entorno de desarrollo con asistencia de IA	Edición del código con autocompletado y sugerencias
NotebookLM	Asistente de consulta sobre documentos	Lectura y consulta de los artículos del estado del arte

Tabla 49. Herramientas de inteligencia artificial empleadas durante el trabajo.

El uso se ha ajustado a los criterios siguientes:

- Ninguna cifra de esta memoria procede de una herramienta de inteligencia artificial. Todos los resultados provienen de ejecuciones del sistema registradas en fichero y versionadas junto al código, como se describe en el apartado 4.6.3.
- Todo el código generado con ayuda de estas herramientas ha sido revisado, ejecutado y probado por el autor. El comportamiento del sistema se comprueba además mediante las pruebas descritas en el apéndice B.
- Las referencias bibliográficas se han obtenido y verificado en sus fuentes originales. Las afirmaciones atribuidas a cada trabajo proceden de su lectura, y no de resúmenes automáticos.
- Los textos redactados o revisados con ayuda de estas herramientas han sido leídos, corregidos y asumidos por el autor, que es el responsable del contenido íntegro de la memoria.